7

DIVING DEEPER INTO PASTA

EXPLORING THE SEVEN STAGES AND EMBEDDED THREAT MODELING ACTIVITIES

> "Knowing your own darkness is the best method for dealing with the darkness[es] of other people."
>
> *Carl Gustav Jung, Swiss Psychiatrist*

Knowledge is power. This is greatly exemplified in developing good and reliable software. At a basic level, poorly developed software compromises generally excludes an adequate SLDC process. As the SLDC process aims to ensure that requirements and design patterns are incorporate, PASTA aims to ensure that those are devoid of risk. Power comes from the knowledge of knowing what coding errors exist prior to a production release. Ignorance is not knowing what weaknesses and vulnerabilities are actually exploitable via abuse cases. As hindsight is always 20/20 in the world of insecure software, PASTA provides the ability to create security foresight.

The following section will now walk-through the PASTA process in the context of a newly forming application going through a generic, waterfall, SDLC methodology. For this example, we will assume a simplified version of Agile SDLC methodology. Regardless of the flavor of SDLC used, threat modeling applications should run parallel to such a process in order to integrate into the generic definition, design, development, and testing phases of software development. The following depiction

Risk Centric Threat Modeling: Process for Attack Simulation and Threat Analysis, First Edition.
Tony UcedaVélez and Marco M. Morana.
© 2015 John Wiley & Sons, Inc. Published 2015 by John Wiley & Sons, Inc.

of the various stages of PASTA is intended to highlight what efforts should take place in conjunction with a developing application. PASTA can also be applied in other circumstances that do not relate to full SDLC efforts; however, they are not covered at this time since much of the activities mapping to a full SDLC development effort can be applied and tailored as desired by a security organization.

The objective of this section is to provide a detailed walk-through of the PASTA process alongside software development activities, normally conducted via a standard SDLC process. As part of this walk-through, roles depicted under the aforementioned RACI diagram will be discussed along with the deliverables that serve as possible inputs/outputs of the activities for each stage of PASTA. This walk-through is not meant to be absolute in the number of activities that are conducted under each stage of PASTA, but provides a suite of core activities that should be performed in support of the listed objectives and goals for each of PASTA's seven steps. This walk-through is also intended to be simply an example to many possible examples in successfully applying PASTA as a threat modeling methodology.

Stage I of PASTA – Defining Objectives (DO)

Business objectives for a given application provide the key ingredients to ultimately define a functional system. Information technology and Information Security leaders are now tasked with understanding those objectives and correlating existing and new governance measures in order to support business objectives with both technology and security measures. This has been a recent message that is long overdue and bears reminding; after all, today's state of application security is the blatant absence of security requirements. A key factor is that security is not properly matched up to business requirements. Since security is looked upon as something that disables business applications versus something that protects and even enables them, as security professionals, we currently find ourselves in a very antagonistic state in the eyes of our business counterparts. For this very reason, it is important for any threat modeling methodology to lead off with well-defined business objectives that are supported with governance artifacts that (1) make sense, (2) are in line with the capabilities of the business and technology groups, and (3) address inherent risks related to both the regulatory landscape of the data being managed by the application environment as well as the inherent risks of the application architecture (mobile, ATM, traditional web, etc.).

Central to PASTA's application threat modeling process are understanding business, financial, and operational objectives that legitimize an application's existence. From a business perspective, discovering why an application was developed, who the intended users are, and how and why features were developed is very important. Essentially, all of these questions map to answers, or more specifically, requirements in any type of SDLC methodology. These requirements give way to use cases. It is these *use cases* that PASTA's stage I seeks to extract from business objectives. Being able to enumerate and inventory use cases has never been done in other security efforts

and therefore inhibits the ongoing use of security evaluations. Beyond this culture shift in application assessment, the use cases found in stage I reveal detailed functionality that would not be captured in dynamic application analysis, source code reviews, risk assessments, application audits, or other types of application evaluations. Knowledge of application use cases enables security groups to bridge the schism that has traditionally separated productive collaboration between business and security professionals. Security professionals have long been categorized as being completely ignorant of business terminology and understanding. Through threat modeling, this notion can be disrupted and give way to productive and realistic discussions on how to apply security features to application functions and features.

From the knowledge of how use cases fulfill business objectives, security professionals are able to hone in on the business impact of adverse security threats affecting the functionality of the application and its underlying data. Understanding how an application fulfills business objectives will help in understanding how adverse threats affect not only the risk level around the application itself, but also the service level agreements, legal agreements, regulatory requirements, and more that are binding to the business. In summary, understanding business objectives behind applications provide the following value-added components to the PASTA threat modeling stage I:

- Knowledge of business drivers/motivators around the application and its core functionality.
- Understanding of the business impact around the application if objectives are not fulfilled or adversely affected by security threats or agents.
- Understanding of possible business liabilities if certain security considerations are not incorporated into the design and requirement building activities for the application. For example, applications operating in a highly regulated environment may have to retrofit compliance requirements if not properly considered within this stage, as it is a business objective to avert compliance requirements that may be costly postimplementation or development.

The inclusion of business objectives at the start of an asset- or risk-centric approach to threat modeling helps build relevant and meaningful context later when attacks, weaknesses, vulnerabilities, and exploits are addressed further in PASTA. Similar to how IT and IS governance begins with exercises to understand business objectives, PASTA initiates threat activities with these foundational concepts so that threats are more relatable and meaningful in stage VII.

The following is a brief and simple representation that helps capture how business objectives could be analyzed into certain prerequisites for factoring security into the process. Examples as to how defined objectives can pave the road to security governance, such as policies, standards, and guidelines, are as follows.

The right-hand side of Table 7.1 reveals examples of governance considerations that could support some of the business objectives that are reflected on the left side of the table.

TABLE 7.1 Relating Business Objectives to Security Requirements

Business Objectives	Security Requirements
High availability for app to process simultaneous requests	• DoS mitigation techniques, • High availability (HA) architecture
Ensuring client/customer confidentiality	• Input/output validation, • Strong authentication requirements • Client data segregation in database • Data encryption (in transit/at rest)
How reliance on data integrity (e.g. financial trading apps/content)	• Use of digital signatures with digital certificates • Proper logging and monitoring at system and DB level
Adherence to stringent privacy laws (per country)	• Proper authentication model • Secure role-based access control model for application access
Legal (e.g. client driven) and regulatory risks	• Translate security requirements to controls for mitigating regulatory and legal risks

Requirement Scope Creeps and Security

Beyond building a primer of procedural- and technology-based governance measures, defining business objectives in stage I provides other intrinsic benefits to the threat modeling process. It may be difficult to consider that business requirements can lead to security features or failures for many, but understanding this relationship requires the ability to have experienced objectives that give way to functional requirements that go horribly wrong. Consider poorly defined business objectives that beget functional requirements for the application environment. Many of those business objectives may be germane to support the business while some are loosely and/or poorly defined and, as a result, give way to poorly or loosely defined application functional requirements. Ultimately, as the application progresses over multiple builds in its life cycle, functional requirements that were defined based on business objectives that did not have clear relevance to the company's overall objectives may be susceptible to being neglected or even forgotten over time. Many on both the user and development sides have said, "Oh yes, I forgot this app could do that."

It is within the realm of those functional, orphaned use cases that dysfunction could arise via misuse cases of the application environment. For every functional aspect of the application, dysfunction can exist, particularly if that part of the application codebase is not well kept. Take, for example, an online B2C portal for sports jerseys. Many of the functionalities may be related to payment processing, account creation, cart management, or other functionalities that support selling sports jerseys online. However, a separate business objective of interfacing with newly formed partner sites (such as footwear) may have been introduced and ultimately neglected as that aspect

of the business disintegrates over time. If APIs that were initially triggered from the online sports jersey store were poorly designed for security (such as anonymous web service requests to a web service namespace or end point), an ideal attack vector related to information leakage could be exploited after having been discovered on the B2C sports jersey web portal. It is for such cases that stage I allows parallel efforts challenge whether or not some of the business objectives defined by the SDLC are extraneous to the application's mission.

Stage I Activities

There are several activities to complete under stage I. Each activity within this stage is intended to correlate to activities that relate to *Defining Requirements* within a generic SDLC methodology.

From the perspective of stage I of the PASTA threat model, the activities that should take place here are aimed at the following:

1. Defining key business objectives to that application.
2. Understanding impact of application to business and thereby impact of functional features to business.
3. Developing a risk profile for the application.

Stage I: Activity 1 (S1:A1) Define Business Requirements

Now we will begin to look at the specific activities that make up stage I of the PASTA methodology. Cross references to the appendix RACI diagram will be helpful throughout this chapter in order to identify what roles are associated with each of the stage activities.

Much of the information relevant to completing S1:A1 already is addressed by some SDLC steps. Although distinctly different in their approach, Waterfall and Agile SDLC methodologies do gather business objectives and requirements that can fulfill this stage of the PASTA process. For the purpose of a risk-based application threat model, S1:A1 prescribes specific information gathering activities that need to be conducted in order to ensure that forward facing PASTA activities have a relevant anchor of context from which to build the overall risk analysis.

Figure 7.1 provides a representation on how obtaining business requirements becomes relevant in building a contextually sound threat model.

The arrow above reflects that use cases can originate in various phases of the SDLC process. For this reason, the process may need to be repeated if use cases are found to be different from earlier efforts as part of a repetitive requirement gathering process.

Rogue use cases may stem from poorly established SDLC practices where developers are left to decipher what requirements a product application should have with the resulting requirements devoid of any input or loopback from the business. Another way rogue use cases come about is via requirement gathering activities that are lost in translation and never become reconciled through repetitive development efforts.

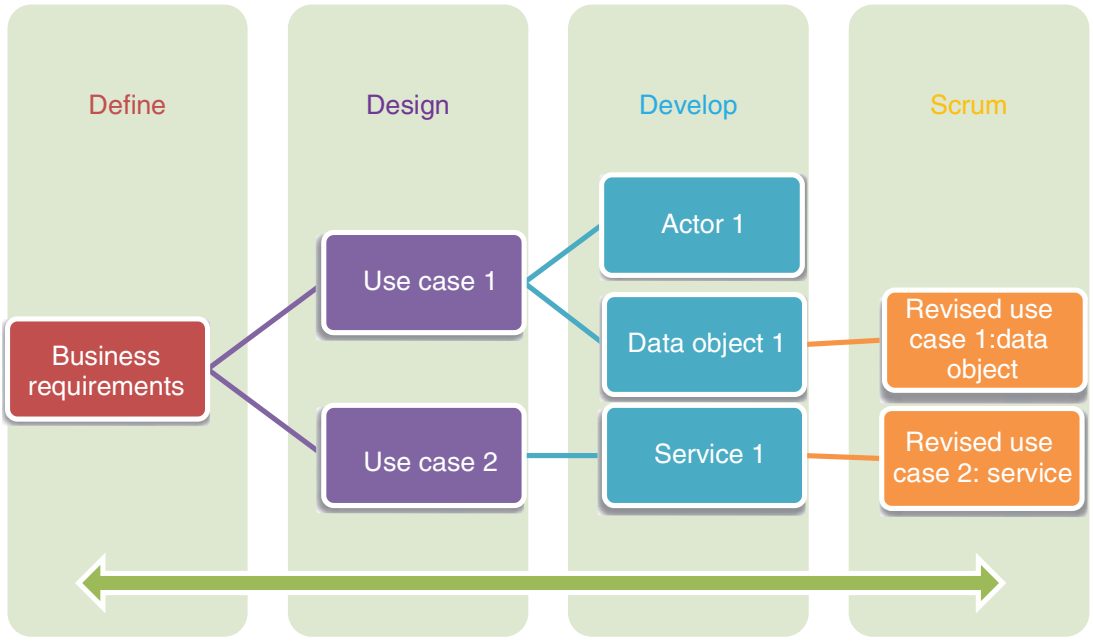


Figure 7.1 Deriving Use Cases from Business Objectives

If repetitive development steps do not catch possible rogue or orphaned use cases, activities in both stages I and IV will help make certain that there is a good mapping that supports identified business requirements. During these steps, requirements received by application development teams may be audited by those responsible for accomplishing threat modeling in order to reconcile whether the translated use cases truly fulfill some of the defined requirements for the application.

Where the information may not be captured or exist presently, a simple artifact can be used to capture the responses needed to complete this activity in S1:A1. The following is a sample artifact that could be used/developed.

In Table 7.2, the highlighted use case derived from the sample business objective does not tie into fulfilling the goal of the developed product application. This simple exercise does not necessarily mean that the use case will be excluded from development; however, it may be trimmed.

At the conclusion of this activity, the threat modeling team should have a strong understanding of what are the key functional requirements for the application. This may easily lend from the DEFINE stage of an SDLC. Furthermore, the end of this activity should provide for a good understanding as to what the key requirement(s) for the application is. In knowing this information, defining business impact in the next activity will be far easier to accomplish.

Stage I: Activity 2 (S1:A2) Define Security/Compliance Requirements

This stage is all about meeting business objectives. It is important to recognize that many businesses actually have both security and compliance as included areas in their overall objectives. The absence of security in their customer facing applications or products may spell trouble for many businesses. Similarly, ignoring or faltering on

TABLE 7.2 Enumeration of Business Requirements to Understood Use Cases

Business Requirement Example	Use Case Example
Become an industry leader in providing a paperless, mobile solution for collecting basic PHI	A mobile app on an iOS tablet is used to capture vital PHI by the physician (blood pressure, height, weight, age, temperature, etc.) upon check-in.
	Allow for PHI records to be sent via multiple protocols (RF, Bluetooth, e-mail, HTTPS, etc.) to avoid the need to print.
	Incorporate GPS location capabilities to allow doctors to inform patients about the location of a pharmacy near their home.
Grow a market presence in Western Europe where mobile tablet solutions are proliferating	Allow the user to select a default language from a wide range of languages found in Western Europe.
	Build in translation capabilities for the 15 most prevalent languages in Western Europe.

regulatory requirements for information security may introduce loss of accreditation or introduce fines – both of which would be vehemently avoided by most companies, particularly small to mid-size start-ups. Activity 3 of stage I provides the opportunity to address both regulatory compliance and security requirements for the application being developed. This provides an integration point for Governance and Compliance processes, artifacts, or deliverables that may have already been developed by the company. From a business point of view, this incorporation of security and compliance is mitigating potential business or reputational controls earlier in a product or software development process.

Working with compliance and privacy officers in this stage, this threat modeling activity can quickly derive what scope of both internal security requirements and external regulatory requirements is related to the application architecture model and most importantly, the data model. Resulting from an assessment of where compliance, privacy, and security overlaps, requirements that are key to implement during this stage are especially those where overlaps occur. Figure 7.2 attempts to portray this visually.

Maintaining topicality to the risk-centric approach of PASTA, this early adoption of security, compliance, and privacy requirements provides for both security and compliance risks to become mitigated earlier in the SDLC process. This allows for preemptive threat mitigation via hardening techniques against system, application, and/or even network components. Working alongside security governance groups, a review of IT and security standards can be applied to the application environment in order to provide a base layer of security. As previously mentioned, many external guidelines and security best practices from sources such as CISecurity.org

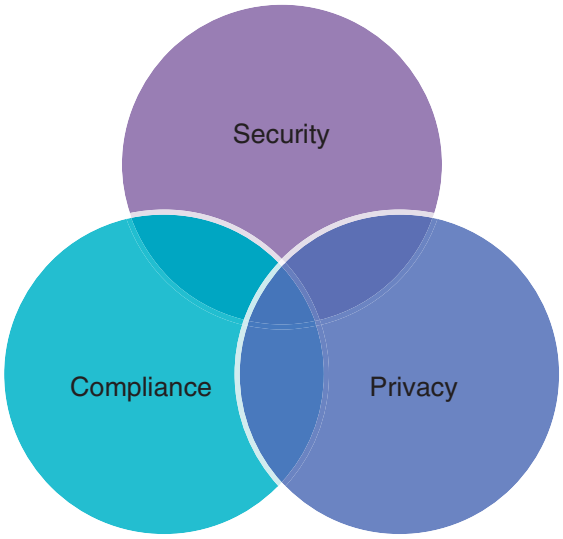


Figure 7.2 Converging Security, Compliance, and Privacy Requirements in Stage I

or OWASP.org can provide an ample external reference point if internal guides and security standards are nonexistent or lacking.

As an example, we can use a health-care product manufacturer. The product is an on-premises client-server solution that captures PHI data for new mothers and their newborns. From a security standpoint, let us assume that the health-care IT organization has internal security standards that are used to ensure that data is securely stored by flat file and relational databases. Internal security audit groups use these security standards as a benchmark for measurement when performing internal assessments. If the application is being threat modeled as part of an SDLC process, this security control would be presented as a requirement to be fulfilled. In doing so, the control has a greater certainty to be adopted and implemented in the DEFINE or DESIGN phase of the SDLC versus introduced, postimplementation.

Similarly, regulatory requirements from HIPAA/HITECH may potentially force the protection of PHI by a covered entity and in such a case, applying cryptographic controls[1] earlier in the process is far easier than having to consider impacts to data calls made by various actors or processes within the application.

Examples of controls that span across privacy, compliance, and security that may be referenced from internal security standards and/or guidelines are the following:

1. Standards for cryptographic controls.
2. Secure coding standards for mobile applications.
3. Network security standards (areas that govern network administration, allowed protocols and services, account hardening).
4. Password Complexity Standard (addresses use of password length, complexity, expiration, etc.).

[1]45 CFR §§ 164.312(a)(2)(iv) and 164.312(e)(2)(ii).

5. Logging Standard (specifies what types of events are logged).
6. Authentication Standard (determines what acceptable authentication mechanisms need to be used).
7. Server Hardening Guide for [X] (includes requirements for hardening an Operating System based on security best practices and OEM recommendations).
8. Security Architecture Guide (outlines the secure architecture models for various types of applications – web, client-server, mobile, and cloud).
9. User Accounts Standard (defines how user accounts are to be governed by credential stores such as AD, LDAP, IdM solutions, etc.).
10. Data Security Standard (identified required controls that should be used to govern security for data at rest and data in transport).

Security Governance Artifacts for Stage I

In forging ahead with this part of stage I, it may be opportune to not assume that all readers understand what is meant by governance, standards, policies, guidelines, and other terms that may or may not be related to actual security governance. It first bears mentioning that there are both technology and security governances. Technology governance has predated security governance, and they traditionally have been managed apart from each other. This is somewhat dysfunctional since a separate group is responsible for configuring a system based on factors such as speed, efficiency, and reliability, among other factors that relate to performance of the application environment. In turn, security groups look to harden the same system environment, motivated primarily to "lock down" the same application environment so that it can reduce its threat landscape. For financial reasons and those related to improved security strategy, both governance groups should create hybrid documents in order to address the same audience and come away with agreed upon functional and security governance material that sustains both functional application requirements for the overlaying application as well as security requirements for CIA.

A suite of governance artifacts used within the early stages of the PASTA process are revealed in Table 7.3. They provide further evidence on how security and compliance requirements can be factored into the PASTA process early on.

Other terms such as guides, cheat sheets, recommendations, or lists may contain invaluable governance material. The aforementioned terms in Table 7.3 are commonly associated and recognized governance terminology that conveys security information for various audience members (end users, system/database administrators, network engineers, architects, etc.). Governance-like material may also stem from blog posts, instructional videos, whitepapers, books, or articles that basically convey what someone should do to a piece of technology, software, or code to improve its security state against any given number of attacks or threats. The act of security governance is not simply the production of standards and the like, but ingesting what sort of obligatory measures are going to be applied to the network, system platform, data repository, supportive network services, and software application so that attacks are less likely to succeed. This is why governance can be leveraged in stage I of the

TABLE 7.3 Governance Artifacts Relevant to Stage I of PASTA

Artifact	Definition
Plan	Details on how policies, procedures, and other governance documentation are to be enforced and made actionable by the audience for whom the plan is designed for. Common plans in governance include Business Continuity, Incident Response, and Disaster Recovery. Depending on the objectives of stage I of PASTA for a given application, these artifacts may or may not be used. They may also be relevant at different stages (e.g. – Incident Response plan may point to the high-level details on how threat feeds are consumed and used, thereby useful in knowing where threat intelligence sources may be found as part of stage IV – Threat Analysis).
Policy	Collection of policy statements that define required actions by personnel and contractors to ensure physical and information security, as well as data privacy. Truly relevant to PASTA's stage I if the policy supports a business objective. Also relevant to PASTA's stage II if the policy can serve as a process based or high-level technical control for which *blind threat modeling* or inherent threat mitigation techniques can be applied.
Policy statement	Succinct, yet clear statements around a specific area that people or processes need to adhered to, in support of company wide security measures. Provides high-level summations of policy goals that may pertain to stage I or II of the PASTA process.
Procedures	Steps or instructions to describe how to achieve the goals defined by a policy's set of policy statements. Provides the actual details on how procedures can serve as threat mitigation controls within stage II of the PASTA process. This allows for inherent risk to be mitigated within the early stages of any given SDLC process (primarily within the DEFINE or DESIGN stage of an SDLC process).
Standard	Requirements for systems or software in order to sustain functional and continuous technology within an application environment and organizational enterprise. This will be most applicable after stage II, since technical objectives/requirements will be defined. Given the technical nature of standards, these governance artifacts provide the most relevant controls to further reduce inherent risk and allow for architects and developers, along with system administrators to conduct blind threat modeling or inherent threat mitigation based on the technology scope or even the process scope (if a human process is an intricate component to the product application). This truly fosters the idea of Building Security-In by including such type of governance artifacts. Building in standards also has a beneficial by-product that exceptions or waivers to implementing controls are reduced by their inclusion and consideration to the system or application being threat modeled.

TABLE 7.3 (*Continued*)

Artifact	Definition
Guideline	Suggested or recommended technical/security measures that could be implemented based on industry expert recommendations on similar environments of application make and type. Given that guidelines have a more technical scope, they will be mostly leveraged during stage II of the PASTA process. Since they are not required (by their nature of being a guideline), these artifacts are irrelevant to stage I. Similar to Standards, they provide further risk reduction when applying against known inherent risk issues caused by identified architectural weaknesses, software vulnerabilities, or previously unmitigated attack vectors.

PASTA model; it provides an initial acknowledgement by the business and technology groups that they are going to develop this new application on the security foundation of hardened networks, architecture, software, and server environments.

The image that follows provides a simple visual representation depicting some of the interrelationships between security governance, technology governance, and ultimately the business objectives defined within this stage of the threat modeling process.

Reflecting on Figure 7.3, the governance materials, which, by definition, are aimed to support technology and security objectives, indirectly support the business objectives that represent the pinnacle of the objectives hierarchy shown. At this point, much of the governance introduced in stage I of the PASTA methodology is more precisely selected and applied and/or adhered to as the scope of technology assets are selected. Essentially, this means that in a heterogeneous ecology of systems, platforms, and software, generic governance material may be cited during stage I. If there is more homogeneity across employed web servers, application servers, proxies, and network devices used as part of the subject application environment, then stage II of the PASTA methodology will provide more specific governance material that conforms to the types of technology that are selected.

Stage I: Activity 3 (S1:A3) Define Business Impact

Deriving business impact for an application environment all centers on adverse events that prevent or limit the predefined business objective from being fulfilled fully or in a timely manner. One already existent source in larger enterprises is the use of a Business Impact Analysis or BIA. BIA exercises are common, especially in the financial industry sector, and allow business objective, self-assessment questions to be captured as well as other types of questions that establish impact criteria for the application as a whole, and in some cases for unique features or functions that the application supports. They can be time-consuming, but it all hinges on the amount

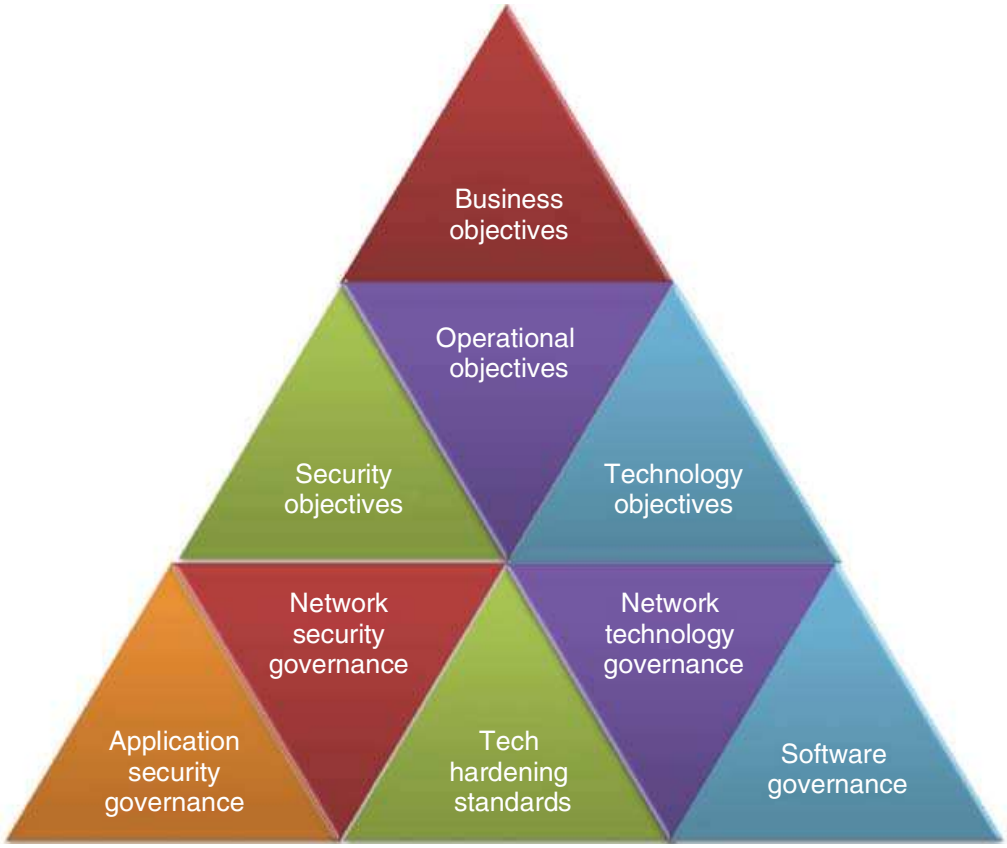


Figure 7.3 Hierarchy of Objectives Addressed by PASTA

of questions gathered as part of that process. For smaller organizations just embarking on threat modeling and not exposed to BIAs previously, a good recipe is simply to develop questions around CIA (Confidentiality, Integrity, and Availability) plus considerations for regulatory Compliance (the other "C"). For any organization, key questions that may be presented are listed as follows:

- Is the data used by the application classified, nonpublic, sensitive, or regulated data (e.g. ePHI (electronic patient health information), PII (personal identifiable information), PCI-DSS related (cardholder data)?.
- Is there a classification assigned to this data? (Internal classification assigned to data may elevate the consequences in the event the data is compromised.)
- What service level agreements (SLAs) to clients/customers are material in terms of loss of revenue, reimbursements, charge backs, and what use cases or data sources would serve as a trigger to these breaks in SLA?
- Can the application's function affect the health of one or more individuals (e.g. medical application in the Emergency Room, Bulk Energy System (BES) (in the Utility sector), direct or indirect operation of an automobile, civilian aircraft, or military device/vehicle)?
- What is the process for gaining access to the application?

- What compliance regulations are relevant based on the data managed by the application and relevant regulatory requirements?
- Does the application's function affect financial reporting for the organization or the integrity of any aspect of financial reporting vehicles?

Monetizing impact is extremely important in this activity and requires some financial analysis. Fortunately, financial numbers are very easy to be obtained from financial officials, particularly when attempting to explain the end goal quantifying business impact. Financial costs are needed in order to convey how an application risk can materialize to business costs, ideally at a unit level. The costs help to convey financial impact to business-minded roles and illustrate to them how technical risks can translate to business impact. Historical events that take place and have associated costs are best to exemplify costs. The following are a list of sample events from which financial costs can be extracted and analyzed.

- Security breaches.
- Litigation costs.
- Historical class action law suit averages.
- Credit monitoring fees.
- Postbreach audit costs.
- Newly required controls to be implemented.
- Loss of accreditation (HIPAA).
- Regulatory related fines (PCI-DSS).
- Opportunity costs associated with downtime.
- Intellectual property loss estimates (via valuation models).
- Direct costs related to downtime or deprecated functionality (annually = annual loss expectancy; per incident = single loss expectancy).

All of the impact considerations listed earlier can be unitized using a gross amount of potential revenue or estimated costs. For example, quarter revenues directly or indirectly related to the product application can be simplistically unitized by a time period (hour, day, month, etc.). In terms of costs, fees related to audit, litigation, and other types of postincident–related costs can be estimated and created into unit costs for different types of events. This will help to provide some degree of financial impact attributed to adverse events from the systems. This type of information and analysis will help to portray a risk picture that is ultimately conveyed in stage VII.

Most BIAs will be able to provide some level of financial context to impact, which is important for this activity. Leveraging the BIA is a powerful, yet underutilized process that generally fails in execution, mostly due to high levels of qualitative speculation with little to not substantiating evidence for financial loss numbers. The BIA process is extremely valuable for understanding risk as it represents the value of a given application to an operational area of a business. The by-product of the BIA process has not been adequately leveraged by other governance processes, such as

risk assessments and business continuity planning efforts, particularly in mid-size and smaller organizations. Textbook BIAs are integrated in more mature governance efforts; however, even in those instances they factor very little into the overall risk evaluation. Related to the PASTA threat modeling methodology (particularly in the context of its use alongside SDLC efforts), BIAs provide a qualitative, baseline description of risk for the application environment. This is important as it may help divide the application environment into components and assets that require varying forms of countermeasures for risk mitigation – all based on the impact that could be sustained by a given attack.

Identifying impact in this stage's activity allows PASTA to have relevant context when addressing threats, vulnerabilities, and attacks against the application. This contextual understanding to threats, attack patterns, and vulnerabilities provides for a rational correlation to defined business objectives. The context for the following exercise will follow the development or rewrite of an entire application environment or substantial portion of said environment. Without yet addressing any aspect of the threat model or encompassed attack surface for the application environment, many of these governance-related aspects will still be able to provide the following value.

This book is not focused on teaching about BIAs; however, they are very important to substantiating a threat model's value under PASTA. Candidly, not having one does inhibit the risk analysis that PASTA attempts to emulate collectively across all seven stages; however, a rapid BIA analysis could be done in order to capture some of the main answers posed by some of the questions exemplified in the last page. The following is a quick example of such an approach in the context of a health-care scenario (Table 7.4).

Building quick questionnaires around simple yet comprehensive security themes (such as Confidentiality, Integrity, Availability, and Regulatory) can easily provide a rapid list of questions on what and how sensitive the business impact would be if threats to those areas were successfully launched. For example, how impacted would a business be if PII were compromised? The key to gathering these answers is to obtain some degree of precision around who, what, and especially by how much an impact would affect a business entity. The precision on the responses will help to ensure that general answers are not factored into the impact analysis, which will undermine the validity of the impact analysis.

It should also be considered that it may be a bit presumptuous to assume that any business impacted by PII leakage would be adversely affected. The reality is that impacts in such a case can greatly vary across industry, business size, country, and so on. Some would be impacted more if their services are focused on managing such data, compared to other business models where such a role is more secondary than principal. An example is whether a government agency in the social services sector experiences a PII breach versus that of an auto store that keeps basic personal data on its customers. The impact will be substantially different, even if the volume of data compromised is the same. Adding elements of government regulations as an overlay, as well as country, will greatly affect the compounded impact of regulatory involvement and fines.

TABLE 7.4 Considerations for Factoring Business Impact

Considerations	Samples
What is the business goal supported by the product app?	*Example:* Increase adoption and overall market utilization of a new product application for physicians who treat diabetic patients.
What is the goal of the app?	*Example:* Facilitate metrics, trending, and overall analysis around insulin shots given to diabetic patients.
What is the regulatory landscape that affects this product app? *(List all the apply)*	*Example:* HIPAA, HITECH (seek UCF guidance).
What is the most critical aspect of the application?	*Example:* Availability, Confidentiality, and Integrity of patient health information.
What is the criticality if the product app was not available over the course of *x*?	*Example:* The application does not have a critical uptime as insulin could still be distributed and quantities and patient data could be captured in the patient module of ACME Health.
What is the existing risk profile associated with the product app?	*Example:* Moderate Risk (4/15/2014).
What is the intended user base of the production application?	*Example:* Health-care practitioners, Loan officers, Tellers, Government employees, Legal, and so on.
Is the data used by the application classified, nonpublic, sensitive, or regulated data?	*Example:* ePHI (electronic patient health information), PII (personal identifiable information), PCI-DSS related (cardholder data)
Is there a classification assigned to this data?	*Example:* Internal classification assigned to data may elevate the consequences in the event the data is compromised
What service level agreements (SLAs) to clients/customers are in place related to the application?	*Example:* Loss of revenue due to SLA violations, reimbursements, and client charge backs.
Can the application's function affect the health of one or more individuals?	*Example: M*edical application in the Emergency Room, Bulk Energy System (BES) (in the Utility sector), direct or indirect operation of an automobile, civilian aircraft, or military device/vehicle, etc.)
What is the process for gaining access to the application?	*Example:* Self-enrollment application, access control workflow, and so on.
What compliance regulations are relevant based on the data managed by the application and relevant regulatory requirements?	*Example:* PCI-DSS, HIPAA, GLBA, NERC CIP, and so on.
Does the application's function affect financial reporting for the organization or the integrity of any aspect of financial reporting vehicles?	*Example:* SOX, FFIEC, NCUA, and so on.

Another thought is to classify impact into various groups, particularly if a bulk financial estimate around each one of those groups can be made. This can simplify the financial analysis so that impactful events are categorized into various financial classes. Organizing impact into various "classes may follow a CIA approach in order to have predefined costs associations for confidentiality, integrity, and availability. This breakdown does not have to follow simply a CIA approach, but any other approach that simply organizes impact scenarios into manageable groups of a similar type. In either case, the best starting point (devoid of a more formal or mature BIA process) is to simply ask questions that all tie to affecting a defined business objective for the application.

1. What are the key goals for the application (e.g. content propagation, provide robust B2C commerce site, employee enrollment, customer bill pay, Cloud-based file service manager, online CRM)?
2. How long can the application be down for?
   a. What would the impact be to existing customers? New potential customers?
3. What legal requirements is the application bound to in terms of service uptime?
4. What regulatory requirements affect the application in the wake of a breach or break in continuity?
5. What impact is felt by cost centers that support the business (e.g. accounting applications, A/R applications, ordering/shipping capabilities, phone lines)?
6. How does reputation factor into the ability for the company to sell its goods/services? Is customer trust a big factor for sustaining repeat business?

More advanced BIAs will have more poignant questions on technology and data use and will typically encompass third-party services, infrastructure, or solution providers that have an integral role in the application's maintenance. Additionally, impacts from various regulations at play should be considered, in order to define supplemental objectives that help to avoid audit risks or customer compliance mandates.

Regulatory Compliance and Business Impact

Faltering against regulatory requirements and mandates can provide their own level of business impact to an organization. Regulatory compliance gaps can negate the worth of an application if the application is found to have material weaknesses against a compliance regulation. This is why compliance is material to a risk-centric approach as it can affect the business objective of a product application. Remember that risk is only relevant if it undermines the objective of a product application. In some instances, even achieving the compliance requirements is not good enough to ward off business impact. In the last quarter of 2013, Target Stores revealed a massive breach that has introduced a massive backlash from both merchant banks, credit card companies, and the PCI Council. This does not include the hundreds of civil suits that have been filed against the company. All of this took place to a

major retailer in the United States that was compliant with the Level 1 PCI-DSS security requirements. For this reason, it is important not to solely rely on simply a compliance driven objective for security, but one that looks at true security analysis, as revealed by a threat model. From a stage I perspective, requirements from security, compliance, and privacy can be adequately converged in order that the proper preventative, detective, and reactive controls can be put into place.

As mentioned in the prior list, threat modeling leaders will also have to consider how application use cases warrant change given any data, security, and/or privacy requirements stemming from compliance regulations. For this reason, the focus on data types are very important for stage I as they help see if the data usage is tied to prerequisites around data security across various compliance regulations. As part of this stage's activity, helpful white-boarding exercises among Compliance, Information Security Officers (ISOs), Business Analysts, and Security Engineers would help define what inherent system or application technology can be leveraged in order to build compliance into the product application. Figure 7.4 depicts a high-level triage representation of the relationship business objectives, application data types, and compliance requirements for a given application. Note how the ≈ symbol represents *resistors* or countermeasures to the risks introduced. The *resistors* in the following figure are supported by controls or countermeasures that may already exist in the environment and may be attributed to administrative, technical, or process-based mitigating controls. Labels on the outside of the arrows represent inherent risks while labels on the inside of the arrows represent mitigating controls (Figure 7.4).

For larger enterprises, sometimes impact is attributed to the lack of compliance to internal controls. Reverting to the first activity for stage I, the threat modeler may find that internal factors may trigger some level of impact that is either direct or indirect. For example, for publicly traded companies, adhering to internal controls around SOX becomes a key objective in ensuring integrity of financial numbers. As such, it provides an opportunity for such an impact to be noted as well as an opportunity to see if there are any internal governance standards that can help introduce controls around logging, authentication, validation of financial transactions, and separation of duties for the application being threat modeled. As part of the third activity in stage I, some

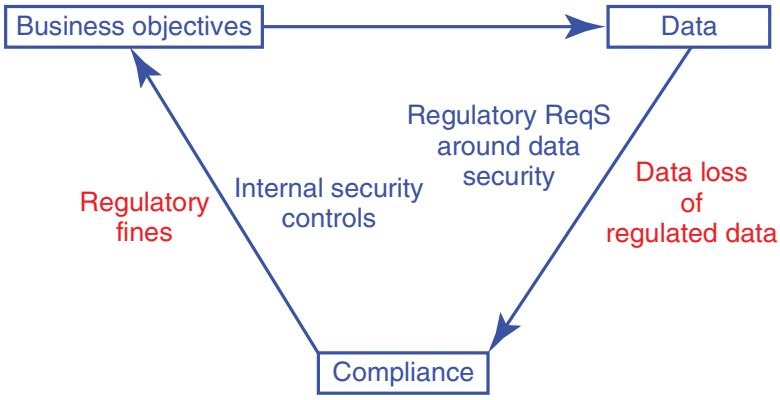


Figure 7.4 Relating Compliance to Business Impact

degree of built-in security considerations can be made at this point by factoring which security standards or guidelines should be introduced to the collective threat modeling process. Depending on the level of detail obtained about the application's business goals and objectives, certain governance material may be introduced in stage I of the PASTA methodology and ultimately considered for implementation as preemptive countermeasures in either stage I or II.

Stage I: Activity 3 (S1:A4) Define Risk Profile

The activity to perform within Stage I (Define Objectives) of PASTA is to identify or prepare a risk profile for the subject application. This exercise provides valuable insight to preexisting risk elements that have affected the application in the past or those applications with a similar technology footprint, deployment model, architecture, or overall use case. This preexisting risk is referred to as *inherent risk*. Different from the BIA, the risk profile seeks to comprehend what inherent risks affect the application from a vulnerability standpoint as well as what threats are inherent to the application being designed. Risk profiles can be derived from prior risk assessments from within a company's enterprise technology risk management group or a private outside professional service group. If an application is new, there may not be an existing or recent risk assessment. If such is the case, risk assessments that have been previously conducted and of similar type (based on technological use, data use, etc.) could be used to draw risk correlations to the application being threat modeled. This is done in order to establish an existing risk profile for the application across common areas.

Inherent Risk

If an application risk assessment has already been conducted, a threat modeling team can see what inherent risk issues carry forward into the PASTA threat model. Most technical risk assessments show open issues relating to newly discovered vulnerabilities from prior risk assessments. Accompanying the risk analysis is generally a remediation matrix that reveals relevant, top issues that represent the weaknesses for the application. Initially, it is important to first understand the technical risk issues for the application environment. Next, it is important to map these vulnerabilities to technical components across the threat model. Risk relevancy will be determined by whether technical risks can be mapped to application components such as assets, actors, data sources, use cases. If they can, they should help in forming a baseline risk level or risk profile for the application. The true risk analysis will arise from analyzing how business objectives and supporting application use cases have been adversely affected by the presence of these vulnerabilities and weaknesses.

Risk profiles provide a baseline of risk around the application and help provide a contextual understanding of some inherent threats to the design, underlying platforms, or even operating environment. As mentioned before, risk profiles can be derived from prior risk assessments. The most recent (last 12 months) risk assessments will provide the greatest level of relevancy around which inherited process,

TABLE 7.5 Possible Inherent Risk Issues by Application Type

Type	People	Process	Technology
Cloud application	Cloud service provider support receives no security training	Onboarding new tenants in multitenant environment receives no vetting	Multitenant cloud environment with only programmatic controls for logical segregation
Mobile application	Discovered that rogue threat agents are creating rogue instances of app in App Store	Nonprivate app store does not vet authenticity of applications or developers.	None-use of controls that help ensure authenticity of mobile application (e.g. hashing, digital certificates, web service authentication to unique application fingerprint)
Client server	Historical information leakage via internal threat agent	No restricted access to developers across lower environments (DEV, TEST, QA) and production environments (STAGE, PROD)	Liberal application access controls

technology, and even people components are considered relevant to the threat model. As most other threat modeling methodologies look at the technical, PASTA also considers nontechnical related process/people issues as well. In the end, building a better understanding of risk via threat modeling means identifying inherent risk issues that affect how application components are designed, developed, implemented, and administered. The following is a brief table reflecting examples of inherent risk across people, process, and technology (Table 7.5).

In many instances, application threat modeling solely focuses people to look at the application stack. Other methodologies may actually exclude the network or system stack. It is important to realize that threat agents and inherent weaknesses to process and human resources may circumvent the most fortified applications. These activities benefits quickly allow for an introspective look as to where other risk areas may warrant analysis and a mitigation strategy.

Prior risk assessments of an application that is being updated is the easiest scenario to depict under this activity. However, if there are no prior assessments that have been completed since the application is newly developed, prior risk assessments may still be used. Selecting prior risk assessments with similar attributes (use cases, deployment models, etc.), data sources, architecture, people components, and process components may allow for some level of similar risk *DNA* to be applied to the new

application. For example, if two client-server applications (one is a legacy app while the other is newly developed) use the same authentication method that is homegrown and known to have prior risk issues, those details can be ported over to a new inherent risk profile for the newly developed application.

Context should drive the relevancy of risk to the application being threat modeled. Not all risk issues are relevant and in order to keep the threat model lean and topical, relevant risks need only be applied. Context will be driven by perspective as it is difficult to determine universally what and how risk associations from one environment to the next should be made. Overall, risk context should regard whether similar application components are being used and leveraged, particularly if using a similar framework, underlying system components, and data sources. Other components that may legitimize the need for inherited risk to be applied are around similar use cases, deployment models, hosting models, outsourced development groups, third-party software components, Open Source libraries, and network paths/protocols. Additionally, architectural similarities may warrant that risk issues become inherited from one application to the other. The following list provides some exemplary guidance on what components may warrant the need to port risk-related considerations from one application to the one being threat modeled under PASTA.

- Use of similar authentication models (e.g. OAuth, LDAP, Active Directory, Stored Credentials).
- Use of same system platform actors (e.g. host accounts used for API authentication).
- Use of similar architecture model (e.g. published web services interacting with mobile clients).
- Similar data stores/warehouses.
- Similar network communication protocols/services.
- Similar use of application and human actors.
- Similarities in system/platform types used in support of application environments.
- Similarities in use of application frameworks (e.g. Struts, Spring, Java Frameworks, .NET) .
- Similarities in using third-party software (e.g. NHibernate, application proxies, Drupal, JAR files, OS extensions, Apache, memcache).

Participants of Stage I

Within stage I, the primary participants include project managers, business analysts, Information Security Officers (ISOs). These members provide primary "Responsible" and "Accountable" levels of support while other groups in IT (Developers, Architects, Engineers provide a secondary or tertiary level of support at the "Consulted" or "Informed" level. As shown in Figure 7.4, the BA (Business Analyst) role should have solid knowledge of what the goals and business objectives are for any given application. As such, they carry a *Responsible* or *Accountable* role. In this stage, they have

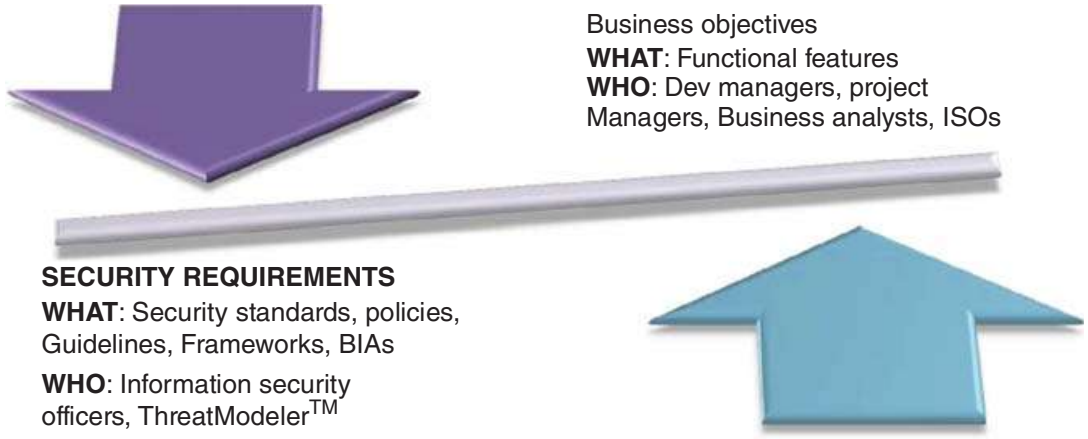


Figure 7.5 Business and InfoSec Balance in Stage I

the responsibility to share what the clear goals and requirements for the application are in order to ensure that scope creeping takes place and that proper understanding of business impact is documented and loosely associated with premature use cases or overall application functions. Given that the focus of stage I is to provide an overall objective to the application threat model in support of the SDLC process for a given application, these participants should focus on providing proper transparency around the business objectives and how the security objectives help fulfill them. This means that from the security perspective and in collaboration with the other roles in the RACI model (for this stage), the BA, PM, and ISO roles need to also receive input as to how security and/or compliance objectives could impair the fulfillment of those associated business objectives. Figure 7.5 helps to depict the type of communication and collaboration that should exist and over what topics.

Generally, the requirements phase of an SDLC process introduces the desire to pile on several functional features for a given application environment. The *WHAT* under the business objectives of the aforementioned see-saw diagram comprises varied application features that management feels too important and critical to exclude from the scope of the SDLC process. Conversely, the *WHAT* provided by the security side of the see-saw figure represents the countermeasures aimed at securing an application environment. Things such as security standards provide preemptive mitigation techniques that can be included based on the intended design pattern, platform solution, and even deployment model. The *WHO* listed under each side of the diagram includes key members that can provide appropriate business and security context in order to contribute to stage I activities under PASTA. Key roles during this stage include those who are responsible or accountable for defining business requirements for the business application as well as those who can articulate security requirements that would mitigate inherent risk issues and/or address regulatory requirements aimed at reducing compliance risk. These suggested roles, along with their associated definitions, are presented as follows.

For a full detailed breakdown of roles for this activity and all activities for this stage, please review the appendix area for the included Threat Modeling RACI diagram. While the aforementioned figure only depicts roles for which individuals will

be either Responsible, Accountable, or Consulted as part of the relevant stage activities. All roles are best exemplified by the RACI at the end of the book.

Beyond roles, artifacts are needed to be consumed by various stage activities as well as produced by each stage activity. Under the following section, we will discuss what specific artifacts should be gathered as input as well as what expected outputs should be produced for this overall stage.

Integration and Collaborative Opportunities for PASTA

Every functional feature within an application can give way to dysfunctional use cases or abuse cases. Many feel that the conversations among security professionals and those on the business and IT side of the SDLC process come from two different camps of life: one from the glass half full, while the other coming from the glass half empty camp. Balance is the answer to balancing the marriage between software development teams and security groups. The collaborative and integrated approach to PASTA lends to provide a more balanced approach and purposively begins with the understanding of business objectives around the application as a way to educate security professionals a bit more on what exactly they are really protecting. Harmonizing the two camps will also come with collaborating across various activities where artifacts, organic to one group, are shared and discussed with another as an input to the PASTA stage's activity.

The previously referenced Figure 7.6 provides an image where each respective effort is applied in the right amount and with the appropriate resistance from the opposing force to achieve a proper balance of both functional features and security countermeasures. Such security countermeasures are applied against the residual risk that has not been accepted or transferred by the business unit sponsoring the

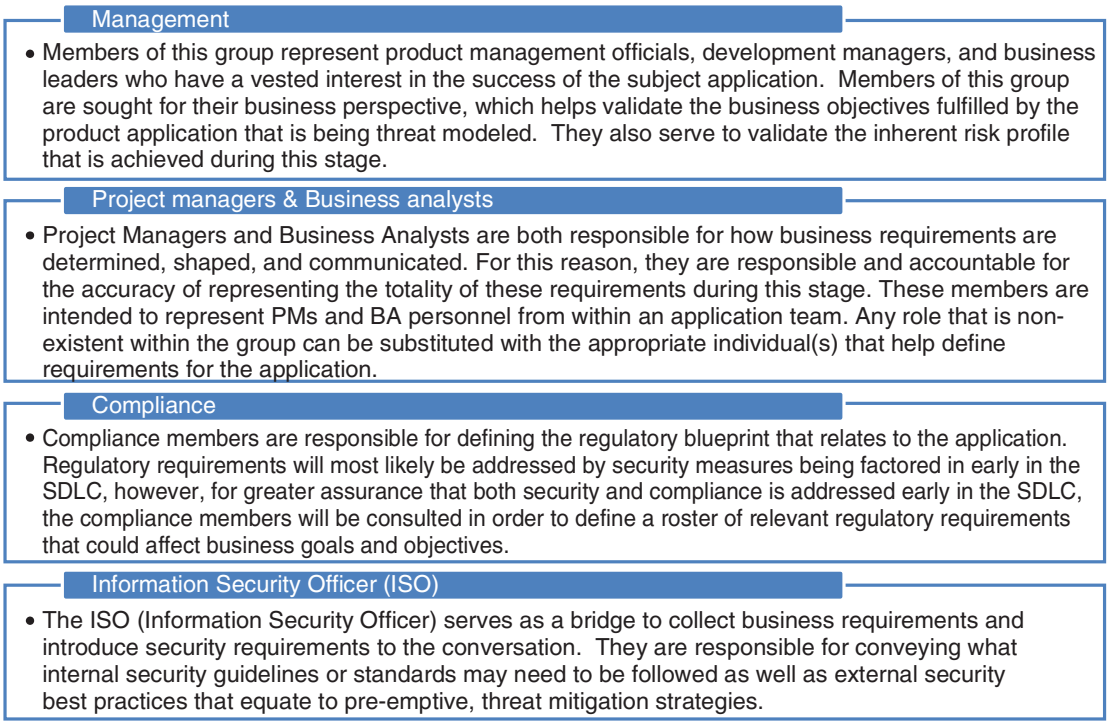


Figure 7.6 PASTA Roles for Stage I

application environment. This effort represents the yin and the yang commonly depicted between business and InfoSec. Fortunately, this adversarial view is lightened via the PASTA methodology as it is very conducive to a greater level of transparency and understanding among each group's objectives. This is due to the fact that the PASTA stages force collaboration among disparate groups – all who support and the SDLC process in various ways.

Taking a historic look at IT, we can recall how the schism between technology and business was improved through introducing new roles such as the systems analyst and the business analyst – both roles that lobby between business and information technology groups, such as network engineering, system administration, and database administration. In InfoSec, the Information Security Officer (ISO) has similarly helped to provide this mediating role. The ISO plays an important role in the PASTA threat modeling methodology as they provide the bridge among security and IT groups. Over its iterative cycle, all participants in PASTA will grow more aware of activity objectives and appreciate both the business risk aspects as well as the technical risks considerations. In the beginning, however, ISOs play a pivotal role given their background in mediating across business units on topics related to business risk and technology risk. ISOs have also traditionally represented Governance, Risk, and Compliance (GRC) groups and are involved in producing many of the commonly used deliverables within that function such as Business Impact Assessments (BIAs), Risk Assessment reports, Vendor Risk Assessments, and standards documents. Being able to understand the objectives of both business and security will provide for greater unity and collaboration during the PASTA process.

A key benefit to PASTA's stage I is that it helps clear the negative image associated with ISOs and the artifacts produced by them or their GRC colleagues – namely Risk Assessments and Business Impact Analyses (BIAs and also Business Risk Impact Analysis. Instead, prior security deliverables are leveraged in stage I of the threat model in order to portray initial risk, security, or compliance issues that should be considered against the functional requirements for the application environment. In regard to the ISOs' roles and their inherent weaker abilities in more technical security issues, their inclusion in the threat modeling team helps to offset some of their more deficient areas. As a result, ISOs can continue to work with the business in order to gauge how the applications will embellish existing security knowledge as well as interpolate business risk more effectively compared to their more technical counterparts in security operations that are leveraged later on in the PASTA process.

Stage I Summary

So now that we have a good understanding of stage I (Define Objectives) of the PASTA methodology, we turn our attention to building from these efforts in order to apply them to the technical scope of the application environment in stage II (introduced and covered in the next section). To recap, stage I of the PASTA process allows business objectives to be understood and security-related generalizations to be made in order to drive obvious governance efforts that should be followed. Answers related to the following questions are some of the tangible takeaways from this stage that should be addressed.

- What is the scope of the threat model?
- What is the most critical objective supported by the application?
- What is the most critical objective supported by the threat model? (e.g. – identifying threats related to information leakage)
- What will this application do?
- What risk has traditionally affected this application in the past?
- Are there inherited risks based on deployment model, architecture, or third-party technologies that are assumed?
- How many inherent risk issues are associated with the application today?
- What data requirements does this application have?
- Who does the application serve? (internal, external clients, external consumers, government, etc.)
- Does this application have revenue generating/impacting considerations that should be noted/defined?
- What are the Service Level Agreements (SLAs) that would be associated with the application?
- What dependencies are created by the application to customers/consumers? (e.g. Cloud data backup service that is B2C, Utility/Energy related, Financial Reporting)
- What inherent risks are implicitly assumed by the service delivery associated with the application? (e.g. PII, Safe Harbor, Top Secret)
- What is the full scope of regulatory or compliance considerations that should be extracted as regulatory controls and considered for design considerations or building other types of functional requirements?

The aforementioned are the basic questions that any business should consider when brainstorming new ideas or enumerating features for a new software application. The answers to these questions help define the technology, people, and physical infrastructure that will be needed to help sustain the desired solution in the context of fulfilling the business objectives.

In conclusion of this phase, we take a look at the key steps and goals that should be achieved by stage I. They are as follows:

(a) Defining a list of key business objectives for the application.
(b) Defining security requirements (particularly around the data).
(c) Leveraging any existing Business Impact Analysis (BIAs) deliverables or have one launched.
(d) Determining the regulatory requirements associated with the application environment.

Figure 7.7 summarizes the steps for stage I of PASTA and reviews possible inputs and outputs associated with this phase of the threat modeling methodology.

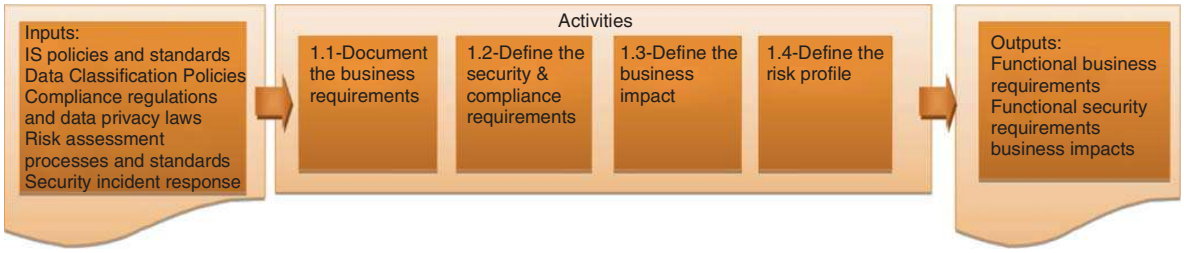


Figure 7.7 PASTA Risk-Centric Threat Modeling – Stage I – (DO) Definition of the Objectives

Stage II of PASTA – Defining the Technical Scope (DTS)

In a risk-centric approach to application threat modeling, the focus is on protecting high-risk assets. PASTA helps identify whether the risk of an asset's compromise is worth more or less than the time and effort to develop countermeasures. For this reason, the focus in stage II is to identify all of the assets in the application environment. The objective is ultimately to enumerate all types of hardware and software components that support the use cases of the application. Later in the process, the threat modeling team will help map relevant threats to the assets identified.

We recall that stage I of the PASTA methodology maps well into any *requirements definition phase* of a given SDLC process by listing any governance-related issues, along with inherent risk and business impact considerations. In this chapter, we build upon this foundation of defined business objectives, inherent risk, and associated impact to understand the application environment from a technical perspective. This collective understanding is very useful in order to define a technical scope for the application environment. Secondly, extend the use of *blind threat modeling* from stage I, we will have an opportunity to build a baseline of security controls aimed at reducing the attack surface for each asset component[2] contained within the scope of the threat model. Controls may traverse physical, logical, or network boundaries. As introduced in stage I, blind threat modeling seeks to reduce the scope of functionality for each component without reducing the overall functionality of its intended use within the application environment. Much of blind threat modeling is simply security hardening and functional deprecation in the absence of a defined threat pattern. In this stage, the technology scope can dictate what types of controls work best to divest needless functionality from software components and harden those services, accounts, ports, and overall component configuration. In summary, security hardening achieves preemptive risk mitigation, which in turn supports the preservation and fulfillment of business objectives associated with the overall application environment.

There are five key activities within Stage II of PASTA. They are as follows:

- S2:A1 Enumerate Software Components.
- S2:A2 Identify Actors & Data Sinks/Sources.
- S2:A3 Enumerate System-Level services.
- S2:A4 Enumerate third-party infrastructures.
- S2:A5 Assert completeness of secure technical design.

The activities of stage II provide a list of which the underlying technology stack will be comprised. This includes the platform/system, relevant databases, application servers, network servers, infrastructure equipment, biometric hardware, and any other

[2]Component is intended to refer to an asset or subasset within the threat model that could serve as the target to a direct or indirect attack.

IT asset that will be used to fulfill the objectives defined under stage I. Essentially, the activities across this stage defines the various components (assets or sub-assets) that will be later dissected by the application threat model. Any of these technologies could either serve as the immediate attack surface or target component within the threat model. The technology scope therefore helps form the environment in which a future attack tree will be representing, along with associated vulnerabilities, attacks, actors, and so on.

Stage II of PASTA (Technical Scope) attempts to answer the common problem or question, "I didn't know that we were running/using that technology in our app." Stage II is essentially a large enumeration project. It focuses on itemizing software and hardware assets in order to later define a lean and relevant attack surface whose components could be exploited by various possible threats. Activities in this stage should produce a roster of technological components that are inventoried. This in itself is incredibly useful and can be leveraged over time as the technology scope of the application changes. The roster of application components needs to extend beyond traditional realms (i.e. server, infrastructure appliance, end point). The list should span to include layered pieces of technologies that may be embedded. The simple rule is to identify anything that has a direct exposure to a caller or actor, whether it is from a human or a compiled library that only calls another component based on an event or interpreted result. Essentially, the asset breakdown should extend beyond hardware and include software-based technologies such as proxies, middleware, authentication servers, compiled libraries, open-source libraries, third-party APIs, browser-based plug-ins, and so on. Today's feature-rich UIs for example need to be dissected properly in order to ensure that all of the client and presentation layer technologies are properly enumerated.

Later on, these assets will serve as the main "trunk" to the attack tree to be developed in PASTA's stage VI. The tree will ultimately show underlying use cases, data components, and actors interact with each other. From each one of these nodes, further branches can be split out to show unique threats for each underlying node stemming from the initial asset component. Scoping is important for this exercise as too broad of a scope may confuse the subsequent steps. Too broad of a scope inhibits good analysis. For this reason, a tighter scope of assets is suggested in order that a simpler and more efficient threat model can be created.

Stage II: Activity I (S2:A1) Enumerate Software Components Achieving the objectives for this activity will come from the following exercises:

1. Interview Consulted or Accountable roles in the RACI as to what software and data sources reside in the application.
2. Review any existing data schemas to understand type of data managed by application.
3. Use tools to quickly enumerate software components, data repositories, file systems, and so on.

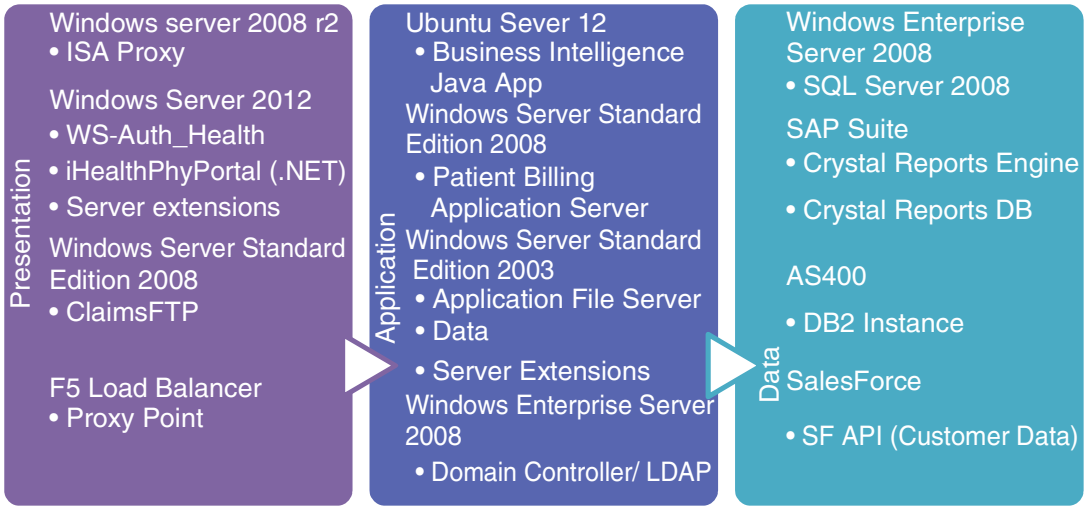


Figure 7.8 Software/Data Enumeration Containers

The easiest way to enumerate technology scope, particularly from software and data sources, is to establish "containers" for where the application software components reside across the overall architecture. For example, in a traditional three-tiered application environment consisting of Presentation, Application, and Data Layers, software technologies can be enumerated into these three containers. An example of how these three containers would look is presented in Figure 7.8.

The "containers" are ways to organize assets (both data and software) into groups. In the figure, the containers show four activities for which enumeration techniques are needed. Each "container" reflects a layer within the environment that has a clear difference in functionality across the application environment. If lists are used, a prefix to the type of application component also helps to organize the components by a type of value that reflects the server type, architectural layer, impact level, or other characteristic.

Building a technical scope begins with understanding how the data traverses the OSI model within the application scope. This analysis should be granular enough to understand how data is managed across the relevant network (within the application), encompasses file storage solutions (from file repositories to physical storage devices), as well as considerations for client software technology.

As part of this stage's activity, the following questions serve as sample questions around data use that can be used to ensure proper scoping within the threat model.

1. Where is the data originating from, what distinct actors or unique data calls are being made, and what sources are requesting the data? (Defining proper trust boundaries, API security implications.)
2. Where is the data ultimately being stored (short-term basis)? Considerations for caching, dynamically created flat files, and relational databases should be made.

3. How many types of networks/network segments will data traverse? (Network configuration and segmentation, data in transit security considerations.)
4. Are there any application servers that will provide any level of validation to the data and at what points? (Implies any distributed or middleware servers that may be in scope.)
5. Will the processes around my data provide use for nonrelational database considerations? (Data at rest considerations, file level encryption, file system-level security.)
6. Is there a long-term storage area for which the data will ultimately reside? (SAN implications, data classification, data retention implications.)
7. Will data be accessed remotely (outside of a company managed LAN)? (Implications for security access control models, authentication, authorization, network security.)
8. What software will interface with the data (web related, client related)? (Implies authorized use criteria, CRUD (create, read, update, delete exercises), data ownership/access models, whitelisted APIs, etc.)
9. In what format will my data traverse over the network (serialized data stream, file upload, xml data blob, etc.)? (Implies use of data in transit security measures, injectable data structures, overall security of data schemas.)
10. What network and application level services will help sustain my data or its user base across the application stack? (Implies dependencies to network services that affect the messaging and security of the data and use of any network standards to secure such services – DNS, DHCP, SNMP, FTP, Active Directory/LDAP, etc.)

These 10 questions provide a good starting point for understanding the context of an application via its data use. In knowing how system or application components are using data types and of inherent impact levels (per type), threat mitigation can be applied in a commensurate manner. Later in stage III (Application Decomposition), DFDs will help to produce the interaction of all system and application components to these data sources.

Providing a proper *technical scope* should begin with consideration of the data life cycle within the application environment and the technology that ultimately sustains and secures it. Purely looking at technical scope based on technology use versus data use will not make much sense via the PASTA threat modeling methodology because of its focus on risk. A large part of risk will stem from data value, information loss, and so on. During this stage of the PASTA methodology, considerations for data use will help define the technical infrastructure that will be used as part of the SDLC process. Later in stage III, a logical representation of data calls can be done via data flow diagrams (DFDs). DFDs help illustrate the interoperability of technology components across the application environment, including, but not limited to, application, database, and client software layers. Examples include how access control is used to control data access across various sources. Understanding data flow across schematics, blueprints, or architectural application designs will help to correlate the

types of countermeasures that may need to be implemented should these measures be found to be either vulnerable or fostering ripe attack vectors.

Stage II: Activity II (S2:A2) Identify Actors and Data Sinks/Sources Achieving the objectives for this activity will come from the following exercises:

1. Identify what actors (human or application based) are serving as worker processes, requestors, callers potentially leveraging authentication criteria.
2. Run CRUD (Create, Read, Update, Delete) exercises against application actors to map associated permissions across the application layers.
3. Perform static analysis of configuration files, which may store username/passwords. This may reveal application level actors that are less well known.
4. Audit LDAP/AD servers that may provide integrated authentication techniques.
5. Sniff application traffic to other key asset components that may contain a pre-shared key, cert, authentication engine in order to see if authentication calls have been missed in enumeration efforts thus far.
6. Identify data repositories by identifying protocols common for data transfer among client and data repositories (SMB, MySQL, SQL Server, Postgres, DB2, FTP, SCP, XML-RPC, etc.)
7. Perform data discovery techniques in order to identify any data extracts/exports that may relate to application data sinks/sources.

Similar to the other activities around enumerating software, systems, and third-party software, this activity is focused on identifying smaller components that actually run or operate within the previously listed exercises. Within these two exercises, we will identify all database, system, and application actors that are making/receiving requests on each asset or across the application environment. There are tools to map actors associated with running system process IDs (PIDs) that help identify some of the inherent actors on a given system or third-party product. One example of doing this at the system/platform level for a physical or virtual Windows host is using SysInternals by Microsoft.

Mapping a user's activities to active PIDs on Windows is simple using Microsoft's Process Explorer.

1. To begin, download Process Explorer from http://technet.microsoft.com/en-us/sysinternals/bb896653.aspx.
2. Unzip process explorer into a directory where you keep utilities.
3. Next, run procexp.exe.
4. Process Explorer will present you with a list of all the processes running along with other information such as CPU usage, RAM consumption, PID, a description, and the software's manufacturer.

TABLE 7.6 Simple CRUD Mapping Across a Product Application

	Presentation Layer	Application Layer	Data Layer
Create	• ftp_user01 • dc1_super1	• soap_user01 • app_int01	• batch_usr1 • hostdb_user1
Read	• w3wp.exe • dc1_super1	• soap_user01 • app_int03	• batch_usr2 • hostdb_user1
Update	• Rrpc_app01 • dc1_super1	• soap_user01 • app_int02	• batch_usr1 • hostdb_user1
Delete	• N/A • dc1_super1	• soap_user01 • app_int01	• batch_usr3 • hostdb_user1

5. Select the Users drop down and you will be able to select which user's processes you wish to view.
6. If this data is needed externally, selecting Save from the File drop down will allow you to export the current user's process information as a CSV.

Design Time Exercise – CRUD Beyond using tool-based techniques to extrapolate the actors and services of the product application environment, application architects and members of the development team should be able to create an initial list of actors and required services that sustain the application use contained within the environment. Developers, for example, can perform CRUD exercises to determine what actors will be used to Create, Read, Update, or Delete data from the data repository, whether it be a backend database or a flat file system. An example of a simple CRUD exercise can be found in Table 7.6.

When searching for relevant data sources in application environments, these are best and easily identified via application schematic designs, which may explicitly identify a nice pair of database clusters that contain all data for the application, or better yet a single mainframe. In today's highly diverse word of data stores, understanding data flow and storage is a treasure hunt. In instances where schematic data flows do not exist, certain e-Discovery tools, along with other efforts, can help to build a DFD that makes sense. E-Discovery scans for certain known application file extensions (.xml, .csv, .txt, .dat, etc.) can all be used in order to regex the nonsystem related files of a file system to see where possible data sources/sinks exist in the application environment. Beyond e-discovery efforts, source code reviews can search for authentication strings that may authentication to data sources. This can further reveal what ftp, RDBMS systems, or file shares are being used to do data pushes/pulls across the data environment.

Stage II: Activity III (S2:A3) Enumerate System-Level Services Achieving the objectives for this activity will come from the following exercises:

1. Interview Consulted or Accountable roles in the RACI as to what platforms and system services are to support the application.
2. Review any existing network design documentation.
3. Run quick tools to enumerate platforms and ports/services used.

Platform enumeration seeks to discover what system types or operating systems are being used. This may also include middleware. Service enumeration is aimed at identifying what services are running on the system assets on which much of the application software will be running. Both of these exercises will require the use of some system commands and/or simple tools that facilitate identifying both services and actors in association with various use cases of the product application. A port scanner, for example, would be able to do both the platform identification and the service enumeration for a given scope. If, for example, my test product application runs on a 192.168.52.0/28 network, I can scan that and see what the scope of platforms and services are.

Note that system platforms encompass application servers and/or databases within each container. They also reveal possible third-party products or autonomous application environments (e.g. cloud services, SaaS services) that may support the overall application solution. Third-party products in general are difficult to include in the threat model since they are largely black boxes for which an underlying understanding of technology is absent. Third-party cloud environments are even more complex since the ability to scan or interface with any facet of the cloud infrastructure is not commonly permitted by the cloud or hosted operator. Vendor products and tools, although proprietary in nature, at least present the opportunity for security practitioners to scan and interface with the host in order to ascertain some level of security assurance. Both third-party services and appliances should be listed under the correct container for activities under this step.

There are scores of package management utilities available, including YAST, apt, and dpkg for Linux, and Windows Program Manager for Windows. While these tools do their jobs well, they are limited in their capabilities and if there is any diversity of operating systems on the network, then systems administrators will end up with a diversity of report types. To avoid this unnecessary use of time and resources, it is recommended that you use a security scanner such as Tenable's Nessus or Rapid7's NeXpose. These will produce formalized descriptions of installed packages in a heterogeneous network with much less time and effort than using the aforementioned options. They also provide functionality well beyond the scope of package management, including the ability to identify services open on a given platform as well as the ability to fingerprint the platform OS and any embedded OS that may be present. All of this information helps the threat modeling process by identifying a blueprint of assets with possible inherent vulnerabilities. Such data will be further leveraged in stages IV and V (Table 7.7).

TABLE 7.7 Software Enumeration from Automated Tools

Software	Count
Microsoft Office XP Professional with FrontPage [version 10.0.2627.01]	1
Microsoft Silverlight [version 2.0.40115.0]	1
Mozilla Firefox (2.0.0.2) [version 2.0.02 (en-US)]	1
Nessus [version 4.4.1.15078]	1
Symantec pcAnywhere [version 12.0.0]	1
Tenable LCE Client 2003 (×86) [version 3.6.2]	1
Adobe Reader 7.0 [version 7.0.0]	1
Mozilla Firefox (3.0) [version 3.0 (en-US)]	1
Microsoft Silverlight [version 3.0.40624.0]	1

Referring to the aforementioned figure, we see that the containers encompass software and system assets that operate within a presentation or DMZ layer of a fictitious architecture. The subsequent layers represent an application layer where business logic or some degree of validation may be applied to requests and responses for data exchanges that are both downstream and upstream. Lastly, a data layer represents where the greatest level of protection should be applied to the assets, given the presumed data centralization of key business and/or client data.

Missing from the example is a container for client-side technologies that are developed by the product/software development team. This includes the likes of JavaScript, Flash, and Silverlight technologies. These technologies should also be included in the threat modeling activities for stage II as they could be the target of threat mitigation in subsequent steps.

The collection of the data for technological asset enumeration can be contained in the Comprehensive Threat Modeling Workbook Tab labeled "S2-Define Tech Scope for App." With respect to technology enumeration, the worksheet will also allow technology components to be divided into various categories using the drop down features of each row. The objective is to create a comprehensive list of technology assets that can be reviewed for functional use cases, interoperability, and inherent security (or lack thereof) within the proposed product application design.

Stage II: Activity IV (S2:A4) Enumerate Third-Party infrastructures Many application environments use various external networks. Some may be in the form of cloud-based services such as PaaS, SaaS, or IaaS while others will be along the lines of a more traditional third-party ASP or colocation model. Whatever the service, it is important to bring third-party infrastructures into the technical scope. Depending on the relationship and the legal terms around "right to audit," certain automated solutions could be used similar to the activities for a captive or self-hosted model. However, many third-party vendors do not like their environments to be scanned without proper permission, so it is important to ensure that any discovery scanning efforts

against a multitenant or shared hosting environment is cleared beforehand. For this reason, realistic actions to perform under this activity include the following:

1. Interview third-party technology SMEs in order to identify what software, data, platforms, and system services are to support the application.
2. Review any existing network design documentation from the third-party.
3. Run quick tools to enumerate software used, data repositories (flat files, relational DB), fingerprint platforms, and ports/services used (may depend on legal agreements with the third party).

Generally speaking, technical points of contact are generally made available to nearly any customer of any size. Most account management teams themselves will have the information or know of the internal SME who can provide requested asset/component information pertaining to your hosted solution. For some of the larger hosting service providers (such as Windows Azure, Amazon AWS, Akamai, RackSpace, etc.), the hardest thing is simply trying to find the right point of contact with whom to correspond with. Although no hosting provider enjoys the inquisition-like experience of being asked questions related to their environment, the smartest hosting providers have actually organized technical information around the most common questions and answers, including a roster of what exposed services and APIs pertain to the services that are provided to most tenants. As most hosting providers are not keen on divulging their architecture and software components, even in the wake of audits from their most affluent clients, attempting to extract as much information from them via a light assessment process can go a long way. Second to that is the ability to interact with the hosted environment and derive what network, system, and application components are present within the hosted model.

The following is an example where a cross section of an application model may be using the Microsoft Azure Cloud solution. Azure Storage may provide a cloud-based repository for your application to leverage as part of an overall functionality. Within Azure Storage, there may be namespaces that your application is using or interfacing with. An example of some of the services found within the Azure Cloud product (Azure Storage) is listed as follows:

- *Microsoft.WindowsAzure.ServiceRuntime:* Classes in this namespace enable you to interact with the Windows Azure environment from code running within your role.
- *Microsoft.WindowsAzure.Diagnostics:* Classes in this namespace enable you to collect logs and diagnostic information from code running within your role.
- *Microsoft.WindowsAzure.Diagnostics.Management:* Classes in this namespace enable you to collect logs and diagnostic information remotely.

These classes may create new instances of objects that are used in the application that should perhaps be included as part of a software component hosted by a third-party vendor. Depicting the aforementioned components in an enumeration list

will allow for subsequent application decomposition efforts to take place in stage III. Following similar exercises will help to enumerate other third-party technology components.

Stage II: Activity V (S2:A5) – Assert Completeness of Secure Technical Design
At the end of S2:A4, one will have the ability to organize the application components into groups that allow one to better map how defined use cases are supported by the underlying technologies and networks (in stage III). Classification of asset components also facilitates the strategy for inherent risk mitigation to take place, all depending on how the assets components are classified. Multiple classifications can take place to "tag" asset components in various ways such as architectural position (data layer, app layer, etc.), criticality level of server, criticality level of information, criticality level to business. All of these designations are not necessary to define immediately, but reaching a point where these characteristics are defined over time will provide for a much more streamlined form of identifying what asset components are worth performing some level of inherent risk mitigation during this activity.

This activity presents basic exercises aimed at organizing the identified application components and providing a level of security assurance that inherent risk mitigation strategies are applied to the identified components from this stage. It should be pointed out that this activity's efforts extend beyond enumeration and into applying some level of countermeasures that come from existing requirements for security. This activity provides for an analysis on how to classify the scope of technology components that will be threat modeled as well, therefore it looks to verify the thoroughness for which the scope has been created. Doing so affects so many other activities downstream, some of which we have already discussed, such as inherent threat mitigation. Others include correlating vulnerability research or threat intelligence to the components that have been identified. This is achieved by making application dissection both intuitive and comprehensive. *Containers* or dissection categories can be used to organize threat modeling *ingredients* or elements, which include actors, services, data, shared pipes, and so on across the breadth of the application solution. The criteria for a container can be anything (as previously mentioned-criticality, architectural role, etc.), but a good rule of thumb would be to use a criteria that is consistent. One of the easiest forms of criteria can be the architectural positioning of the application component. For example, the following seven containers reflect an architectural location of physical asset components in the overall technical scope of an application.

1. Client Software.
2. Client Platform.
3. Client Hardware.
4. Server Software.
5. Server Platform.
6. Server Hardware.
7. Network.

These seven areas are collectively broad enough to address most threat modeling targets and narrow enough to individually provide adequate threat modeling ingredient organization for the threat modeler. Items 4–7 (inclusive) can also be virtualized. You may have both traditionally hosted and cloud-based components that relate to software, platform, and hardware and this would not be redundant. The reason would be that the administrative service and process supporting such technology would be vastly different from a self-hosted physical asset versus a virtual hosted asset in a multitenant PaaS environment.

As the technology changes, the relevancy of these container groups may change or even disappear over a long period of time; however, even with the proliferation of virtual computing platforms, the hardware containers, for example, are still very much relevant given virtual host resource consumption levels of physical hardware components such as power, physical, and network interfaces – all of which can receive unique configurations that undermine secure implementation of the whole application. Overall, the platform security stack is often overlooked by other types of threat models; therefore it is important that stage III builds upon the successful enumeration of technology assets in stage II in order to define the interrelationships among all assets (both physical and virtual). Although we are engaging in application threat modeling, applications often times leverage the environment variables and configurations of its underlying platform or system host.

Containers formed under this activity help organize asset types and initiate how interrelationships are formed. A great example on how this activity can make for better application decomposition later in stage III is the area of embedded system security. Malware-laced hardware products, particularly in POS devices for retailers, health-care handheld devices with client software, and proprietary military equipment with an application stack embodied in a small ecology of asset types that have embedded kernels, users, roles, APIs, and more could easily be missed in other threat models. Depending on the Threat Analysis conducted in stage IV, these variables may actually become material to the overall threat model but should be considered and identified earlier. In other threat modeling methodologies, a threat analysis may legitimize this threat; however, there may not have been the necessary component enumeration and classification in order to identify which component to apply countermeasures to. PASTA provides this opportunity to prepare for such an organization to application components in this stage.

A representation of each of these seven containers for maintaining dissected application components is presented in the series of graphics as follows (see Figure 7.9).

Within this PASTA methodology walk-through, security analysis and countermeasures can be introduced across each stage in order to address and support business objectives related to the environment being threat modeled as well as mitigate risks identified by each phase of the methodology. Governance artifacts, however, are best consumed within stage II of the PASTA methodology. Within Stage II of the methodology, correlations are drawn to appropriate governance efforts in order to build security into preexisting SDLC efforts that are part of an established workflow. This allows traditional security shelf-ware, such as requirements, checklists, standards, and (in some cases) policies to become “actionable” and baked into the

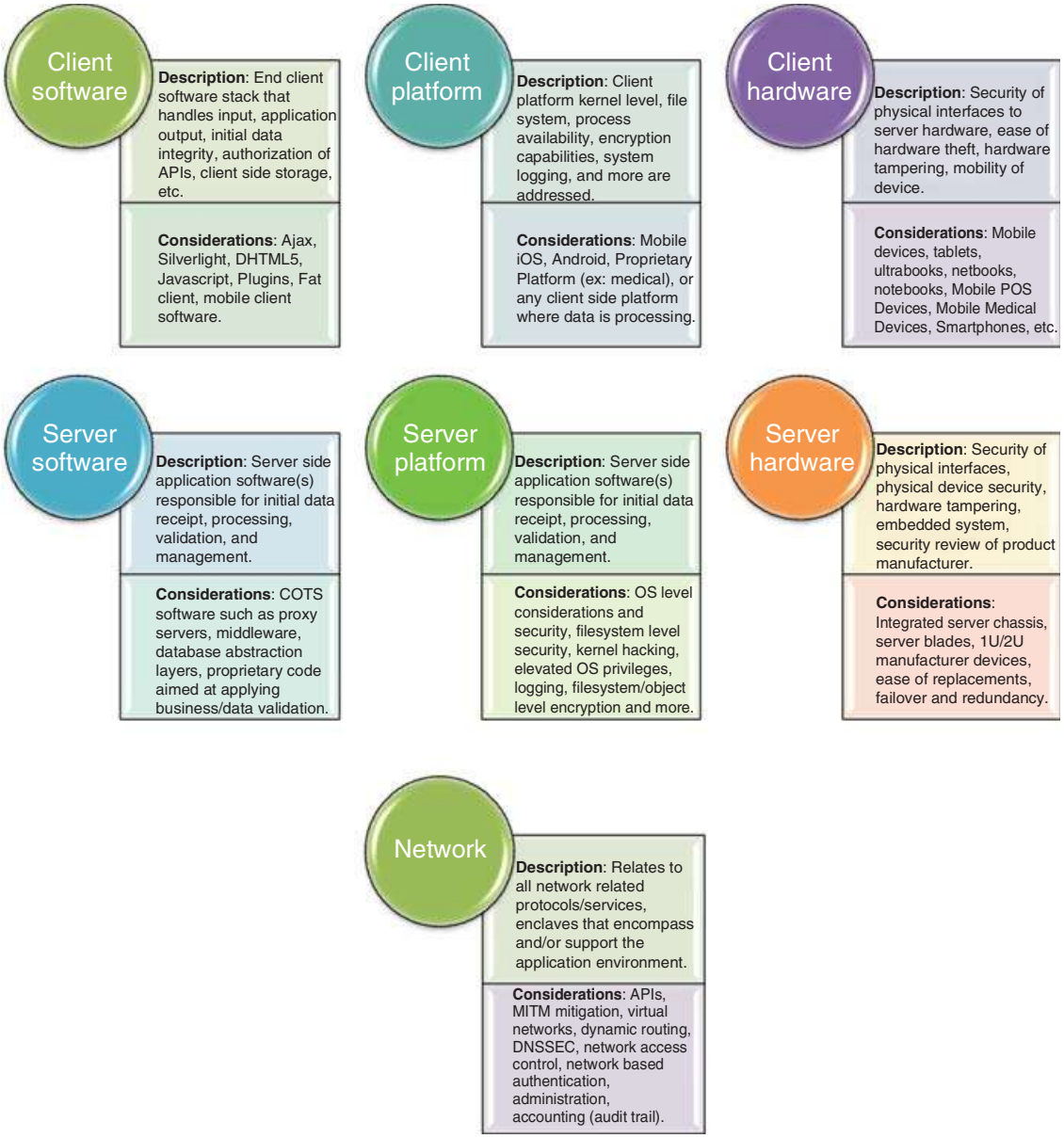


Figure 7.9 Stage III Application Containers

development process or change control process for a given application environment. The translation of business to security objectives will ultimately create the opportunity for traditional security governance efforts to be injected into the application design – reiterating the idea that application threat modeling can finally integrate security efforts that have traditionally been excluded early on in the SDLC process.

Ultimately, upon reaching a clear understanding of the types of technology to be used under stage II (Technical Scope), a baseline of security considerations can be formed in order to create a baseline level of security configuration aimed at sustaining the technological, operational, and business objectives for a given application. Moreover, we see the potential to leverage technology and security governance in order to provide some degree of tangible evidence or examples of these governance

by-products. The following table lists examples of governance documents and/or tools that may primarily be leveraged from the manufacturer of given technologies across the application environment. Most of the references given subsequently are denoted as external forms of standards for securing or hardening various technologies commonly found in application environments. A company's own internal technology standards would be the preferred starting point, presuming that they support the business objective via their given function. The following table is not meant to represent a comprehensive set of hardened security standards/tools, but are mostly used as examples to convey the point of applying various recommendations for hardened technology platforms. Most importantly, these references are completely free and are specific to a type of platform and are generally provided by the technology manufacturer, such as Microsoft, Oracle, Cisco, and so on (Table 7.8).

The point of the aforementioned table is not to provide a listing of security governance tools and documentations, which are bound to expire, but instead to demonstrate that a lot of resources for preemptive security hardening can take place during stage II as a technical scope of assets is defined for the application. Many of these resources can be found on the manufacturer's site of the OS or platform used. The key point to be made is that this layer of preemptive hardening provides a species of *blind threat modeling*, which simply acknowledges that a specific threat has not been defined, but as threats are inevitable, implementing an easy baseline of platform or system controls could add security to various layers, thereby supporting the age-old adage of defense-in-depth, which is still a reasonable and encouraged approach in general.

The previous table introduces countermeasures that address possible uses of technology within the application environment, all this without having entered into the formal threat analysis. This translates to something that can be termed *blind threat modeling* – considering and implementing base security controls prior to formally revealing defined threats and threat actors. The blind threat model is formed in recognition of data use cases for the application, protecting the fulfillment of defined business objectives, and preemptively addressing regulatory requirements that are naturally in scope. No other threat modeling methodology factors in this aspect of blind threat modeling and although this inclusion of controls and countermeasures may be criticized as nontopical to a defined threat, this process provides a primer of security controls supporting business objectives and fosters greater reliance to build security requirements early on in the SLDC. Many of the security standards that would be applied would naturally mitigate a large amount of system/network configuration and implementation flaws that would normally be present across system and software implementations (SSL Implementation, PKI roll outs, Authentication etc.). As a result, baking designated security controls into the application architecture preemptively elevates the level of risk mitigation and accomplishes a huge feat of truly building security-in.

In the event that limited technology governance material exists internally, external references to security checklists and/or hardening guidelines could be used as security governance for various assets (targets) in the PASTA threat model. For large company enterprises where technology and security governance is more prevalent, internal

TABLE 7.8 Free Hardening Guidelines/Tools for Inherent Risk Mitigation or Blind Threat Modeling (Stage II – PASTA)

Asset	Governance Reference/Tool	Reference
Windows asset (Win7, W2K8, and beyond)	Enhanced Mitigation Experience Toolkit (EMET) v3.0	http://www.microsoft.com/en-us/download/details.aspx?id=29851
ISA Proxy Server	Windows Hardening Guide for ISA Proxy 2006	http://technet.microsoft.com/en-us/library/bb794718.aspx
Windows IIS (Win7, W2K8, and beyond)	URLScan 3.0	http://technet.microsoft.com/en-us/library/dd450367(v=WS.10).aspx
Windows Servers or Computing Endpoints	Microsoft Baseline Security Analyzer (MBSA)	http://www.microsoft.com/en-us/download/details.aspx?id=7558
IIS Server (v6 & up)	IIS Lockdown Tool	http:/support.microsoft.com/kb/235864
IIS 7 Web Server	Windows Server Hardening Guide (2008)	http://technet.microsoft.com/en-us/library/bb794718.aspx
Windows 2008 Enterprise	Podcast and Hardening Guide for W2K8	http://www.microsoft.com/events/podcasts/default.aspx?pageId=x344
Apache Web Server	Whitepaper on Hardening Apache Web Server	http://people.apache.org/~sctemme/ApconEU2008/Hardening%20 Enterprise%20Apache.pptx
Oracle Database	Oracle Database Vault	http://www.oracle.com/us/products/database/options/database-vault/ index.html
MySQL Database	Making MySQL Secure Against Attackers, Setting Up SSL Certificates for MySQL	http://dev.mysql.com/doc/refman/5.0/en/security-against-attack.html http://dev.mysql.com/doc/refman/5.6/en/secure-create-certs.html
Cisco iOS	Fortifying SNMP, General Management Plane Hardening	http://www.cisco.com/en/US/tech/tk648/tk361/technologies_tech_ note09186a0080120f48.shtml#fortify http://www.cisco.com/en/US/tech/tk648/tk361/technologies_ tech_note09186a0080120f48.shtml#configman

(*continued*)

TABLE 7.8 (*Continued*)

Asset	Governance Reference/Tool	Reference
W2K3 Server (Standard)	Windows Server 2003 Security Baseline (solution accelerator)	http://technet.microsoft.com/en-us/library/cc163140.aspx
MS SQL 2005 Server	Database Mirroring Best Practices and Performance Considerations (Availability)	http://technet.microsoft.com/en-us/library/cc917681.aspx
Sun Solaris 10 Application Server	Solaris Security Toolkit 4.2	http://www.sun.com/software/security/jass/
SuSe Enterprise Server Edition	SuSe Enterprise Security Tools and Documentation	The security tools that Novell has available ranging across an array of different topics that include everything from secure OS configuration to certificate implementation. http://www.novell.com/communities/coolsolutions/tools?filter0=securityandfilter1=All
Juniper Networks Firewall	Juniper Security Configuration for Switches	Provides reference to secure configuration guidance on the JUNOS Software. http://www.juniper.net/techpubs/software/junos-security/junos-security10.1/junos-security-swconfig-security/junos-security-swconfig-security.pdf
Windows 2010 SharePoint Server	Framework for deploying shared services across SharePoint farms	Reveals a best practice security model around the implementation of share service farms and hosts http://office.microsoft.com/en-us/windows-sharepoint-services-it/windows-sharepoint-services-security-model-HA001160771.aspx
Windows SharePoint Security Features	Security Features present within SharePoint servers	Windows SharePoint Services 3.0 around inherent security features http://office.microsoft.com/en-us/windows-sharepoint-services-help/about-security-features-of-windows-sharepoint-services-3-0-HA010021578.aspx
ESX Security	VMWare Infrastructure Hardening (v. 3.5)	http://www.vmware.com/files/pdf/vi35_security_hardening_wp.pdf

Ubuntu Server	Tools and guides around how to secure Ubuntu server instances	https://help.ubuntu.com/community/Security
Any client-side web technology	OWASP Anti-Samy	A collection of programmatic code snippets that allow for countermeasures to be factored into most web applications. https://www.owasp.org/index.php/Category:OWASP_AntiSamy_Project
Any web-related application code base	OWASP Cheat Sheet Series	A collection of cheat sheets that show examples of code snippets on how to best mitigate insecure coding errors in web applications and associated components. https://www.owasp.org/index.php/Cheat_Sheets
IIS Webserver	IIS 7.0 Benchmark	Prescriptive guidance for establishing a secure configuration posture for MS IIS 7.0 (source: Center for Internet Security (CIS))
Apache Web Server 2.2.x (v3)	Guidance for securing Apache web servers on primarily RHEL platforms	Center for Internet Security
Microsoft Windows Server 2003 Security Standard	Hardening Guide release by Microsoft to secure the system level services and built-in applications	Microsoft's Pattern and Practices site; Also referenced by NSA.gov
Database Security Checklist and Guidance	Database security considerations from the US Department of Defense	Defense Information Systems Agency (www.disa.mil)
Microsoft ISA 2006 Proxy STIG Version 1, Release 2	Security technical implementation guideline for Microsoft ISA proxy	Defense Information Systems Agency (www.disa.mil)
z/OS RACF STIG, Version 6, Release 5	Guidelines for security mainframe environments' RBAC model via RACF	Defense Information Systems Agency (www.disa.mil)

Note that these URLs may change over time.

standards, guidelines, checklists, or other governance by-products can be leveraged. Additionally, there are several external references for which a secure baseline configuration at the application and system level can be introduced – all in support of the technical and security objectives, which are both areas that should ultimately support business objectives.

It should be mentioned that these external references should really only be used to inspire or sustain those organically developed by a governance group for an organization. If these and/or similar governance items are leveraged and applied against the application environment, the threat model must account for the controls or baseline countermeasures that are implemented as a result of adhering to these governing requirements or recommendations on security at the network, system, or application level. Given that we are attempting to lower the inherent risk profile of an application to an acceptable residual risk, we have to encompass the countermeasures that relate to the network or system environment, particularly if they have been hardened via the conformance to a body of governing security standards.

In the end, governance can stem from internal efforts as well as external sources driven by wider security-related groups. The benefit of these latter groups is a slightly more vast perspective, which may also be their detriment, particularly when in conjunction with Application Threat Modeling efforts. Nonetheless, the content is recyclable, usable, and provides a good baseline. The following is a varied list of external governance material that could be followed during this stage of the application threat modeling process in order to be assured that a solid security foundation is created at the foot of the application environment.

Participants Across Stage II Activities – Technical Scope

As will be reflected for each of the stages of the PASTA threat modeling methodology, the inception of the process hinges upon a clear understanding of the business purpose or focus for the threat modeled application. As we will see, the focus will shift slightly more to the security and technical aspect of the application environment analysis in order to deconstruct the functional into dysfunctional areas that can be analyzed for probable vulnerable and attack vectors. The nature of those involved will also slowly begin to exclude more of the business folk and encompass the likes of developers, security testers, pen testers, and web application specialists. For stage II (*Technical Scope),* the key people to be involved are application architects, system engineers, and developers. These three groups of individuals do not represent the entire participant base for this phase for every type of organization; however, they are central to addressing the security of the aforementioned components related to the network, application, system, and data handling within the developing threat model. Therefore, they carry a *Responsible* or *Accountable* role in the previously shown RACI diagram (Figure 6.8). The architects will conceivably have a strong domain over the interrelationships of application calls across the application/network environment. Therefore, their holistic view will allow them to perform application enumeration activities for this stage as well as consider blind threat modeling techniques to evaluate and potentially use tools and reasonable hardening measures. The network architect is also valuable here

in order to understand the network infrastructure that is supportive of the environment and what underlying Layer 2/3 protocols are needed to support the application. For reasons similar to those of the system administrator/engineer, the threat modeler needs to understand the default network configuration and hardening that may exist as the underlying communication backbone of the application environment being threat modeled via PASTA – this includes the security configuration of network switches, routers, IPS/IDS devices, and more. The network professional may also contribute in the areas on how high availability network technology is leveraged for given application environments as well as other network-related services that may be used by the targeted application, such as DNS, LDAP, AD, PKI-related servers (Certificate Authorities), application/network proxies, IPSec services, and more. All of these considerations will affect the threat model by revealing countermeasures against attacks at the transport layer. As a joint effort, the architects and system engineers can provide an accurate assessment as to the total range of IT assets (both hardware and software) that would be needed and in scope for the application environment. Knowing what is in the application environment is currently a challenge for most security assessment efforts as several IT outliers find themselves outside the reach of traditional assessment efforts. Using a combination of technical discovery techniques by the threat modeler (or requested by the threat modeler to SecOps) along with Q&A efforts with architects and system administrators, a defined list of assets across an *n-tiered* environment is possible. The scope of effort for these individuals obviously encompasses the logistical boundaries of the application environment, but may extend to other areas where connectivity and access may exist, such as partner sites, cloud-based APIs, or lower computing environments.

Stage II also requires the role of the lead system administrator for the subject environment. As mentioned before, larger environments may have standardized their server side technology, virtualized on a given platform solution, or gone to their cloud for a hosted solution. The lead system administrator or system engineer is someone that above all else may have knowledge of the standard base image on a given server in any one of these environments. At this point, the threat modeler should work with the various individuals in order to derive what base security controls were configured into the disk image. This may range from a vanilla image from the manufacturer of the OS (Solaris, Windows Server 20xx, SuSe Enterprise Server, etc.) or application server software (Apache, Citrix XenServer, etc.) to a well hardened system based on technical and security requirements, as mandated by a technology or security governance entity within the organization. If no security requirements in the form of hardening guides or technological standards exist, then this is an opportune time to consider what security requirements should be in place. Such security standards derived from this process should be done agnostically of the specific use of any given server technology or platform within the threat model. The intent here is to not progress without some effort of security governance taking place during stage II of PASTA. At the very least, if no governance documentation exists, parallel governance efforts can begin here in order to understand at least what security controls could have been leveraged based on the high-level business and IT objectives understood for the application in stage I of PASTA.

Developers (as the last Responsible or Accountable group in this stage) know what types of software or hardware requirements will adequately support their application environments. In addition, they are quite particular about what types of application servers would function best for their data storage and API needs. Application servers such as databases, proxy servers, memcache servers, e-mail, and cloud-based applications such as Salesforce (CRM) or even Cloud-based services such as OAuth web services, Azure/Amazon AWS APIs, are all technology areas that provide additional information reference points (in terms of asset sources and overall expansion of the possible attack surface). These become relevant components or family of components (referring to Cloud technologies and services) that feed into asset enumeration activities performed within stage II.

Although the *Responsible* and *Accountable* groups are Architects, Developers, and System Administrators, *Consulted* roles are vast in this stage. This is to ensure that all technical assets, networks, and software are accounted for as part of relevant application components. Those consulted (refer to RACI diagram in Figure 6.8) are BAs, QA members, Security Operations (internal or external to organization), security risk management professionals, and those carrying the role of information security officers for various lines of business.

The involvement and activities completed by the aforementioned group of individuals introduces control maturity to PASTA, particularly realizing the case of technology security governance. Beyond nonexistent, a next level governance attempt that could be started from the PASTA process is to begin with general, server/technology agnostic governance documentations that point to base controls that could be applied regardless of asset use. For a supposed CMMI (Capability Maturity Model Integration) value of 1, this newly developed governance documentation may suggest more generic security controls at the system or application configuration level that can be leveraged, regardless of technology use. A common example is applying such generic guidance to web server assets of different use cases: Internet versus intranet presence. Specifically, protecting connection string information in both instances of a web server (assuming a relational backend database) is equally important. More lax configurations on certain server side controls will depend on the nature of the intranet application and its use cases; however, there may be greater leniency on the amount of network protocols that are open on a given host for an intranet server as compared to an internet server found in an external DMZ environment. This is simply one example where a given server type may need to have a security standard developed as a spinoff effort to the threat modeling process. The threat modeler will not be involved in this governance process, however. As it is outside of his/her scope, he/she may liaise the effort between IT and security governance.

Regardless of whether a new artifact of security governance is derived from this process, the key point to take away is involving the lead system engineer or system administrator who may contribute technical guidance as to the base level of controls that may be present across the proposed application architecture. The designated role of the threat modeler is to interface with chief architects, developers, or system administrators, who serve as custodians to the subject application environment, will help the

threat modeler determine what topical, governance items can be leveraged. Similar to stage I of PASTA, this exercise under stage II allows the following to be achieved:

- An enumeration of baseline level security controls to mitigate attacks across the attack surface to be defined. The word "baseline" in this case is used in the context of what existing suite of technical controls are present at the OS level or application server level in order to enumerate a "default" set of countermeasures that may assist in attack mitigation or reduction.
- An understanding of technical dependencies among the application environment to platform, software, and network-related technologies.
- Cross reference to any existing technology standards/guidelines allows technology/security governance to become actionable as part of the threat modeling process.

Thus far, we have mentioned three groups of individuals that may be involved in defining the technology scope of PASTA's stage II: the developer, the architect, and system engineers/administrators. These three groups of individuals do not represent an exhaustive list of possible participants that are to be *Responsible/Accountable* within the Technical Scope stage of PASTA, but their inclusion is essential. Other members may be used who have a strong sense of technical understanding to the network, platform, or software components to be used within this stage of the application environment. Related to the SDLC process, it should be noted that this stage of PASTA can be tied to the Requirements Definition stage of a given SDLC process in order to see what underlying technology security requirements can be applied to the overall application environment, in support of the functional business requirements. In the following Stage, Application Decomposition, we will explore the interoperability of the system/platform, application, data, and technical components in order to map out things such as actors, assets (or targets), calls, requests/responses, and more. This stage of PASTA directly correlates with the design process of a given SDLC life cycle.

Contrasting Large and Small Enterprise use of PASTA's Stage II

Although large and small organizations may have similar business objectives for application environments of the same type (point of sale, inventory management, content distribution, etc.), the underlying technology governance implications that support these objectives will differ among the two during stage II. From a technology governance standpoint, larger corporations have an easier time applying a standardized list of inputs to a proposed solution. Large corporations tend to heavily standardize their use of technology. This includes standardization efforts across web servers, hardware, network infrastructure devices, and even third-party services. Due to their depth of financial resources, comparative to smaller companies, larger companies have the strength of both technology consolidation and standardization – which in turn provides a great value for achieving preventative administrative security controls in the form of standards, guidelines, and hardening techniques. This is not to imply that all large corporations standardize technology and services; however, they

do possess more of an inherent capability to facilitate security governance. Greater access to employee resources within respective governance groups, financial resources, and overall purchasing power all lend to characteristics where larger organizations are better positioned when it comes to technology standardization, and overall technology governance. Conversely, smaller companies face inherent challenges that are nearly opposite to these same business traits. With limited access to financial resources, a smaller employee base to lead governance efforts, and limited funds for procuring technology, smaller firms indeed face an uphill battle to achieve a high level of threat modeling maturity in stage II of PASTA.

Procurement's Impact to PASTA's Stage II Activities

A word on procurement of hardware/software and how shopping for the most cost effective technologies may affect cost of administration as well as cost of securing these assets, particularly for SMBs. The greatest inhibitor for smaller firms in stage II of PASTA is money. Here is a great example of how other business functions can actually affect the adoption and growth of threat modeling in an organization, particularly among smaller firms. Smaller companies have less sensitivity to value calculations and philosophies such as total cost of ownership (TCO). As a result, their technology decision making is cost driven versus strategy driven. The focus of small or even mid-sized enterprises is more geared toward finding the most appropriate solution at the best possible price. This negates the ability to conform and develop technology standards, let alone security standards for technology governance. The reason is that as a greater level of technological diversity proves challenging to harden and securely configure. Note that the word *challenging* is used and not impossible. Securing heterogeneous pools of technology is very possible, but also very time-consuming. Faced with optimizing purchases versus strategizing how new technologies can be streamlined and supported more uniformly, smaller organizations allow a more heterogeneous pool of technology to take root within their organizations. Ubuntu servers running Apache, Windows desktop user environments, and a mix of infrastructure brands at the network level are common for both types of environments; however, when it comes to technology standardization, larger companies may defer to exclusively use one brand of technology over another in order to facilitate ease of administration.

Stage II provides opportunities for both small and large companies to consider a bare bones baseline of security across their procured and employed technology. Organizations that can streamline technology assets and third-party services to a select group of technology can allow security standards to be developed and maintained more easily. Maintaining standards is greatly undervalued from a security perspective. What has not been made apparent to most is that a linear relationship of implementation ease exists between IT (technology) governance and security governance. As a brief example, a yearly governance effort to revise policies or standards (typical for nearly all governance groups) is made increasingly difficult by technology variances across an organization over time.

Smaller organizations, such as start-ups, will lack in this area and will generally have neither the breadth of capital resources nor a standardized process for procuring

and implementing technology. This thereby forces them to have a wider array of employed technology platforms that will extend their attack surface because of the more varied technology that will require unique security mitigations.

A standardized technology or service environment allows security governance to be more easily managed versus one where several different forms of technology are applied. Applying governance around disparate technologies, for the purpose of ensuring proper configuration, hardening, access control, logging, and so on, requires more time and effort than those environments that have been standardized.

This is why stage I of PASTA is to understand the objectives for a solution that is or has been developed in order to lead into stage II – defining the technical landscape of the solution. As objectives to a new or existing software application are defined/redefined, the threat modeler can at this stage begin to work out the inputs that would be needed in order to support such a solution and derive what inherent threats/countermeasures may be at stake. The opportunity for the threat modeler to illustrate how security governance becomes tied to these same inputs is even more important as it makes the notion of building security in more of a reality, as such base security controls are associated with the application environment based on revelations of data types, employed technology, and software solutions to be used within the overall ecology of the application environment.

Stages I and II activities brought forth opportunities for security governance to be instituted as a primer, particularly when applied in parallel to the SDLC process. Generally speaking, larger organizations are in a better position to sustain business objectives with well-defined governance standards simply due to their size and ability to have dedicated groups leading in those areas. Smaller organizations do not typically have the luxury of unique governance personnel who build a master library of strategic policies, standards, and guidelines for consumption. The harsh reality is that smaller organizations have more pressing matters to attend to, namely running and growing the company and putting out operational or business development fires that ignite along the way. This unfortunate disposition does not mean that smaller organizations are precluded from engaging in application threat modeling, but rather that their efforts in this area should be more creative, targeted, and staggered over time. For example, smaller firms can look to industry frameworks for quick-wins on existing governance standards from organizations such as OWASP in order to apply security principles to business objectives for a web-based application being proposed under stages I/II of PASTA. Larger firms can obviously apply the same approach, but they typically have enough resources to develop custom governance documents that can dictate the types of technology standards and processes to abide by. In their circumstances, material from places such as OWASP, PTES, CIS, ISC2, ISACA, Microsoft, Oracle, SANS, and more will help support or shape their tailored governance efforts applied during Stages I and II of PASTA.

It is possible for smaller enterprises apply specific governance material in stage II of the threat modeling process, since it is then that the business will understand the types of technology that will be utilized for the application environment being altered (via change control) or via new development efforts during the SDLC process. The smaller scope of security governance efforts is directly related to the amount

of hardware and software supported by the application environment. With a smaller scope of technology to standardize (for potentially both IT and Security perspectives) specific hardening guides for the system environment can be applied with greater ease. This typically lends to an advantage for the smaller organization, although the lack of consistently applying technology governance often reveals inconsistencies over time, thereby reducing how effectively such companies mature under stage II of PASTA. Given their more complex and mixed technology, applying security governance during threat modeling is even more necessary for larger organizations, particularly in stage II. For larger enterprises, standardizing not only aligns well with applying security standardization and overall IT governance, it also ports other intrinsic benefits, such as cost reduction, improved TCO, better internal/external support from choice vendors, and others. This is because larger companies quickly realize that procuring the same technology lends to cost savings in administration and support, particularly when buying from a single vendor and potentially in bulk. Acquiring and using the same technology facilitate applying security standards and guide against these more uniform and dedicated forms of technology. Committing to a certain flavor of web server, network router, firewall, middleware, and code-off-the-shelf (COTS) solution provides the opportunity for governance and security to be enforced with reduced risk of changing the underlying technology. Over time, companies can quickly become "HP," "Windows," "Oracle," or "Unix" shops based on their standardized asset procurement and asset management practices. As an example, a large company that has a strategic partnership with Dell will build out their Web, Application, and Data environments based on the globally accepted technology standards for company web servers, application servers, or database servers. From there, associated security standards can be applied and "baked in" to the baseline configuration of the asset image.

Although there are obvious differences in terms of how small to large organizations address stage II of PASTA, the diverse extremes portrayed in both instances exemplify how IT governance and security governance can evolve into a synergistic effort where standardization is taking place. This shows how simplified IT and IS governance can coincide and be uniformly applied in a unified process. Standard builds of varying security levels can be built in conformance to industry inspired standards from NIST, ISO, CIS, MITRE, SANS, OWASP, DISA (military), and beyond. While the smaller firm may appear to have less of the advantage, they can actually grow from a smaller governance practice and develop a greater quality of governance through a reduced scope. Separately, larger firms have the ability to develop security assurance if their asset acquisition strategies are led by IT governance goals for standardizing technology types. Regardless, this phase allows PASTA's stage II to incorporate security governance as an inherent countermeasure to enumerated threats in subsequent stages of the PASTA model.

Stage II Summary

In conclusion of this phase, we take a look at the key steps and goals achieved by stage I: (1) understand underlying technology (network, software, system) and related

dependencies associated with the application environment, (2) round up any applicable technology/security standards to be applied to technology assets used within the scope of the application environment, and (3) properly define an application scope for the threat model to focus on. Figure 7.10 summarizes this process for us and summarizes possible inputs and outputs associated with this phase of the PASTA threat modeling methodology.

Now that a business understanding of the application has been defined via stages I and II has built upon that information with the practical understanding of technological components, we look to analyze the application via exercises aimed at dissecting or decomposing the application. This phase builds upon Stages I and II by evaluating the dissection of each application component by how it supports business objectives defined in stage I, while leveraging basic controls via security governance in stage II. Stage III of PASTA, Application Decomposition, is truly a unique process that provides a collaborative setting where the threat modeler can facilitate discussion among development teams, security teams, IT/Security architects, and more on how the application is designed and ultimately built. This collaboration ties very well into design phases of any SDLC methodology and openly includes multiple SMEs over a tabletop exercise to understand the depth and breadth of how an application works. This unique component to PASTA fortifies this threat modeling methodology above others for the purposes of deriving risk. The analysis, however, must first begin with a thorough understanding of the application's intended design and use cases so that subsequent phases of attack enumeration, vulnerability analysis, and threat assessment are effective. The following section also explores data flow diagramming to a greater extent, as well as introduce the necessity of defining trust boundaries across a given application stack within the application environment. Additionally, the section will also focus on the heightened importance of security architecture and design to the PASTA application threat modeling methodology.

Stage III of PASTA – Application Decomposition & Analysis (ADA)

In relation to SDLC efforts, application decomposition can take place during the design phase. This is timely since by this point, requirements or use cases are generally more defined and can therefore be correlated to other areas, including but not limited to network routing, hardware, configuration (system/software), database tuning, and much more. This correlation and analysis provide an opportunity for security threats to be better understood and properly mitigated in the subsequent PASTA stages that follow. Since in the design phase, use cases become more clearly defined and connected to underlying technology, application decomposition helps to ensure greater familiarity with how use cases are (1) designed to behave and (2) related to other components of the threat model (use cases, actors, assets running stored procedures, services, etc.). With this organization of usage, the threat modeler can facilitate understanding how threats affect various parts of the subject application. Via the application decomposition stage, complex or confusing application architectures can be unraveled in a manner that allows all involved SMEs and threat modeling team members to truly understand how the application works and where it can be subjugated.

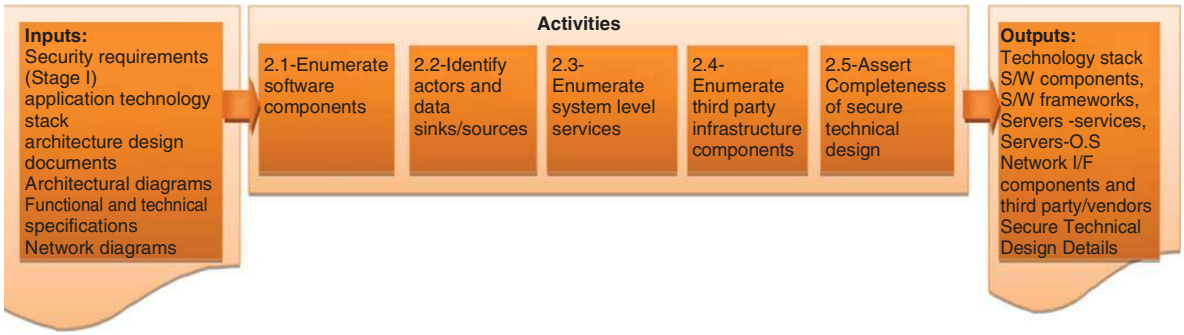


Figure 7.10 PASTA Risk-Centric Threat Modeling – Stage II – (DTS) Definition of the Technical Scope

Stage III (Application Decomposition) of the PASTA methodology includes four key activities. They are as follows:

- S3:A1: Enumerate all application use cases (ex: login, account update, delete users, etc.).
- S3:A2: Perform DFD of Identified Components.
- S3:A3: Security Functional Analysis & the Use of Trust Boundaries.

In the following sections, we will review the details associated with each activity; however, first we will begin with a background on stage III threat modeling goals.

Stage III: Activity 1 (S3:A1) – Enumerate Application Use Cases Use case enumeration is aimed at ensuring that all planned functionality is captured and revealed in the DFD. This benefits everyone involved (Architects, Developers, System Engineers, Business Analysts, etc.) by visualizing how the functional requirements are diagramed in a visual. The use cases should reveal the actors or processes that are part of the call/request. This provides the context for what use cases are most critical to protect and later, allows for vulnerabilities and attack patterns to be appropriately mapped to them. For this activity, begin to list the use cases for the application in the following manner (Figure 7.11).

As shown, use cases encapsulate other threat modeling components such as services, trust boundaries, actors, data sources, named pipes, and so on – all things that support application use cases. Use cases are developed from functional requirements, which largely reflect the understood objectives by the business for the application. As such, a use case enumeration exercise will help formulate a hierarchy of what actors, requests, responses, or data boundaries are used as part of each use case. This may have already been accomplished via the activities typically performed during the DEFINE phase of the SDLC process.

Use cases will have a one-to-many relationship with various components such as actors, data sources, named pipes, and so on. Since we have defined much of these under stage II, we simply now have to play matchmaker with the use cases identified in stage I with and application components identified under stage II. Application

Use Cases	Stored Procedures/Batch Jobs	Actors
• Register new patient • Update patient data • Add insurance information • New insurance claim • Update insurance claim • Delete insurance claim • Assign image to patient record	• [XML Data Blob Parser.py] • [ReportGen.sql] • [NormalizedEMR.sql] • [UpdateImageRecords.sql] • [PurgeRecordedFile.py] • [EncryptPHI.py]	• (L) [platform account1] • (L) [domain account] • (L) [DB Svr account] • (L) [DB Admin account] • (L) [DB User account] • (H) [Client User] • (L) [App WebSvc User account]

Figure 7.11 Enumeration of Use Cases, Services, Stored Procedures, Batch Scripts, and Actors

TABLE 7.9 Sample Identification of Use Cases for Health-Care Application

Use Case	Actor (Presentation Boundary)	Actor (Data Boundary)
Check-in patient	IIS worker process	(CREATE) MED001 domain user
Register patient	IIS worker process	(CREATE) MED001 domain user
Enter patient symptoms	IIS worker process	(CREATE) MED001 domain user
Validate patient symptom validation	IIS worker process	(READ) HealthUsr001 (SQL login)
Update patient record	IIS worker process	(CREATE) MED001 domain user
Pull patient medical history	IIS worker process	(READ) HealthUsr001 (SQL login)
Make imaging request	IIS worker process	(READ) HealthUsr001 (SQL login)

components should be identified within the architectural layer to which the use case is found. In some cases, the application component may be within a different architectural layer than that of the use case (e.g. – client request component sends coupon code to application web service). A simpler representation of the exercise that would accomplish this activity task is shown in Table 7.9.

White-boarding exercises can assist to "talk out loud" about how components support use cases across the overall architecture. Thereafter, it will be easier to map out their interoperability with one another within the DFD. Remember, that if the application is large, threat modeling the most attack prone layers may make the most sense, especially if time constraints are present and threat modeling is still in its infancy as a program within an organization.

The following is a brief example of how these associations can be tied to a root use case, as part of an application decomposition effort, which in this case applies to a fictitious stock trading application (Figure 7.12).

In order to better exemplify exercises associated with application decomposition, we will focus on building on the threat modeling stages and associated activities we have collected thus far. Using a hypothetical example of an online web application, the following example will extract and link the business objectives, associated technology, and use cases for a sample marketing application.

In this example, we will take an online software application that takes credit card purchases for a particular cardholder and performs a mash-up against current offers that retailers have for consumers. From a stage I perspective, an applicable business objective for this application may be:

Business Objective of Marketing Application *To leverage credit card transaction data with future advertisements of retailers in order to sell a mix of consumer spending information to interested retailers wishing to better penetrate geographic markets, cities, etc. Key supporting objectives are to ensure that the two types of data are current, well-managed, properly correlated, and protected.*

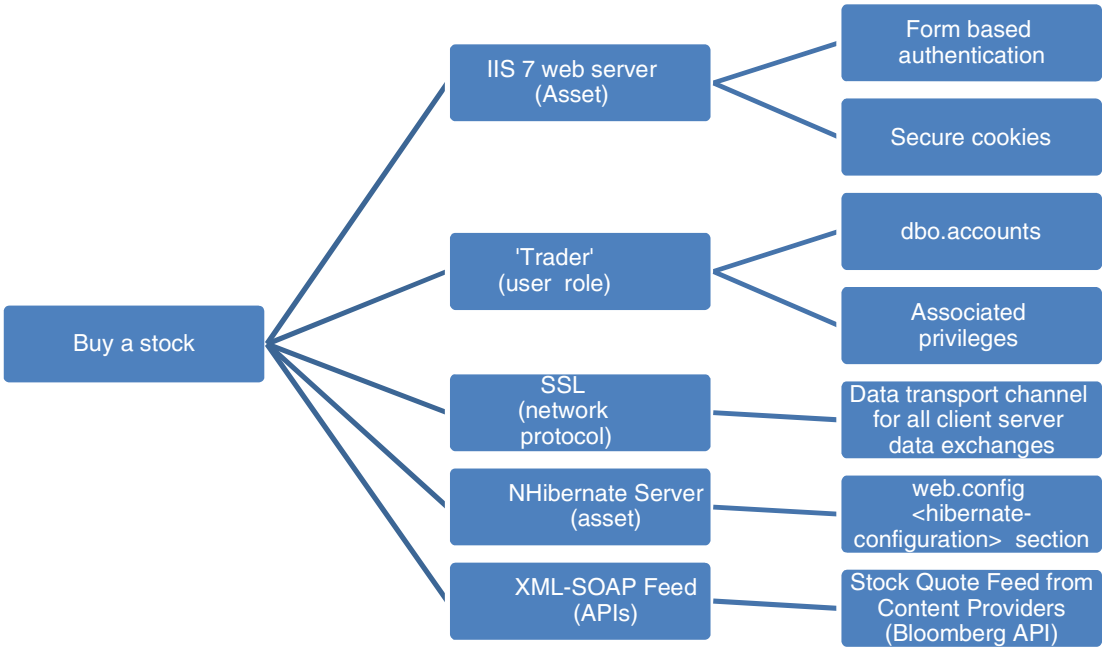


Figure 7.12 Use Case to Application Component Mapping

Additional discovery efforts in this stage may also reveal that the subject application of the threat model will include a few published web services that support retailers' ability to interact with to backend data repositories. Some of these APIs will seek to obtain consumer buying trends for a particular geographic region (e.g. zip code, state) for a subset of cardholders. Other interfaces to the subject application may include content feeds from various coupon providers who offer small quantities of coupons or offer them in bulk. These could be correlated to cardholder accounts so that they may receive targeted offers for a particular retailer, product, or service.

In support of a subject application's intended focus, the threat model can continuously identify a myriad of application use cases, which we can dissect within stage III of PASTA. At this Stage, we presume that Stages I and II have been adequately been completed and documented for this fictitious marketing application. Stage III (Application Decomposition) will leverage the information from prior stages, so it is important not to lose sight of the business objectives defined in stage I or the technical scope defined in stage II when applying PASTA. With the understood business objectives from stage I, functional requirements that help ensure such an objective can be developed during the Planning and Design phases of a generic SLDC process. A sample of functional objectives from stage I may include:

1. Maintain a *customer profile* based on purchases made and other correlated demographic information (age, sex, location, etc.).
2. Securely *store and leverage customer profiles* in a normalized state that facilitates searches of customer profiles and transactions.

3. Maintain accurate customer contact information in order to *solicit new offers* based on prior buying history.
4. Provide a *marketing module* that allows marketing professionals to devise new campaigns or offers.
5. Provide a *financial module* aimed at defining offers, coupons, or promotional thresholds for given products in order to ensure some degree of margin.
6. Receive *batched data feeds* related to transactions made at stores, by various referential IDs, such as Store ID, zip code, and so on.

A tabular representation of the sample application is found on the following page and demonstrates how PASTA's first three stages build on one another in terms of dissecting the subject application. Objectives in the top header stem from stage I and give way to use cases in stage III, which support functional requirements that aim to fulfill those business objectives. Stage II provides that technical scope of what technology platforms the application will use. This will be increasingly relevant and important for other areas of stage III and looking ahead to stage IV – Threat Analysis.

This very concise table is not aimed to be a comprehensive list of all objectives, assets (both software/hardware), and use cases pertaining to the hypothetical application. However, it does provide a good example on how broken down components of the application can be organized.

In previous PASTA stages, outputs related to aforementioned business objectives, functional requirements, and technical scope help provide the proper context for application decomposition. It also provides the ability to gradually see what existing, inherent controls (physical, technical, and logical) are present. Each of the activities in the previously mentioned PASTA stages will help show how the application handles things such as secure design, data flow, inherited trust, inherent security controls, and user privileges around use cases.

Assuming a three-tiered, internal facing web application, the following can be depicted as a possible set of use cases for the application to be dissected against. The terms requirements and objectives have been used interchangeably later in order to denote some aspect for which application use cases can be built upon. Lending from the functional requirements or business objectives in the hypothetical application used thus far, a myriad of use cases could be identified and isolated within the application. A subset of possible use cases are supplied in Table 7.11. This sample set of application use cases, identified as part of define/design stages of the SDLC process (primarily) or the application threat model (secondarily). For the sake of brevity, use cases pertaining to only functional requirements "Customer Profiling" and "Customer Data Storage" were used from those depicted in Table 7.10.

Stage III: Activity II (S3:A2) – Perform Data Flow Diagram Exercise Stage III of PASTA (Application Decomposition) has the opportunity to ensure that each dissected area of the application supports the objectives defined in stage I and adequately run on the assets defined under stage II. In stage II, we inherit component names as well as descriptions around each component (e.g. type of client-side plug-in, versions around software components for web, application, and database servers).

TABLE 7.10 Hypothetical Functional Requirements/Objectives for Marketing Application

Functional Requirements/Objectives						
Stage I related (objectives defined)	Customer profiling	Data analytics	Data harvesting and brokerage	Offer creation and management	Financial offer management	Batch processing
Stage II related (technical scope of software/hardware assets)	AppServer Profiling Engine (Apache/Tomcat/Java, Ubuntu Server 12;HP Integrity rx2800 i4 Servers (HA Pair))	DBServer (MySQL Cluster CGE, Ubuntu Server 12; HP Integrity rx2800 i4 Servers (HA Pair))	DBServer (MySQL Cluster CGE, Ubuntu Server 12)	CouponServer (Windows Server 2008, IIS, Enterprise SQL Server, .NET web services, FTP service; HP ProLiant DL320 G2)	Third-party service provider managing current offers via published web services; hardware supplied by third party	JobServer (Ubuntu Server 12 with Perl/Python scripts and cron jobs enabled; HP Intensity Superdome Server)
Stage III related (identified use cases from functional requirements)	Automatically capture store sales data related to purchases (name brand, price, time of day, store number, cc type, etc.)	Provide "canned" analytics around customer aggregate customer data	Allow data entry of Name, Address, Age, Sex, Zip, Marital Status, and Ethnicity (optional)	Allowance for entering new offers from retailers	API for COGS (cost of goods sold or costing data)	Nightly store uploads of store purchases

(*continued*)

TABLE 7.10 (*Continued*)

Functional Requirements/Objectives					
Allow self-fulfillment for new online profiles to be created	Provide interface for user-driven dynamic queries around aggregate data	Allow interfacing with other marketing related apps to provide Name, Address, Age, Sex, Zip, Marital Status, and Ethnicity (optional)	Manage expiration of offers made automatically; allow revocation from offer administrators	Allow financial users to enter in unit costs for products sold	Batch processing of new financial thresholds
Cross reference in-store purchases with other marketing data servers containing "bulk" marketing information	Cross reference and validate with other marketing tools that maintain client data	Receive, process, integrate, and package demographic information from other marketing sources for authorized sale	Manage renewal capabilities of various campaigns	APIs to other financial tools and software in order to perform predictive financial analysis	Batch processing of new weekly offers

TABLE 7.11 Deriving Use Cases from Functional Requirements

+		Use Cases	Description of Use Cases
Functional Requirement(s)	Customer profiling	Capture store sales data	API stores local transaction servers with the following metadata: retailer, price, time of day, store number, cc type, and so on. and feeds this application's data repository w/the captured info.
		Self-fulfillment of new users	Form online-based enrollment, a web interface requesting e-mail info (such as username and password), mobile number, name and address, and credit card.
		X-marketing data lookup	Cross DB queries using "LIKE" statements in order to find counts of possible consumer matches to other marketing data sources.
	Customer data storage	Canned reporting	Precanned operational reports. E.g.: <purchased item> by <zip code> and <gender>, <purchased item name> by <item type> with matching <store buyer profile data>, and so on.
		Dynamic reporting	Allows users to construct business queries to data repository in order to create their own types of marketing report.
		Cross reference foreign marketing sources	Sends SOAP requests to external marketing data web services in order to query if matches exist for existing customer profiles.

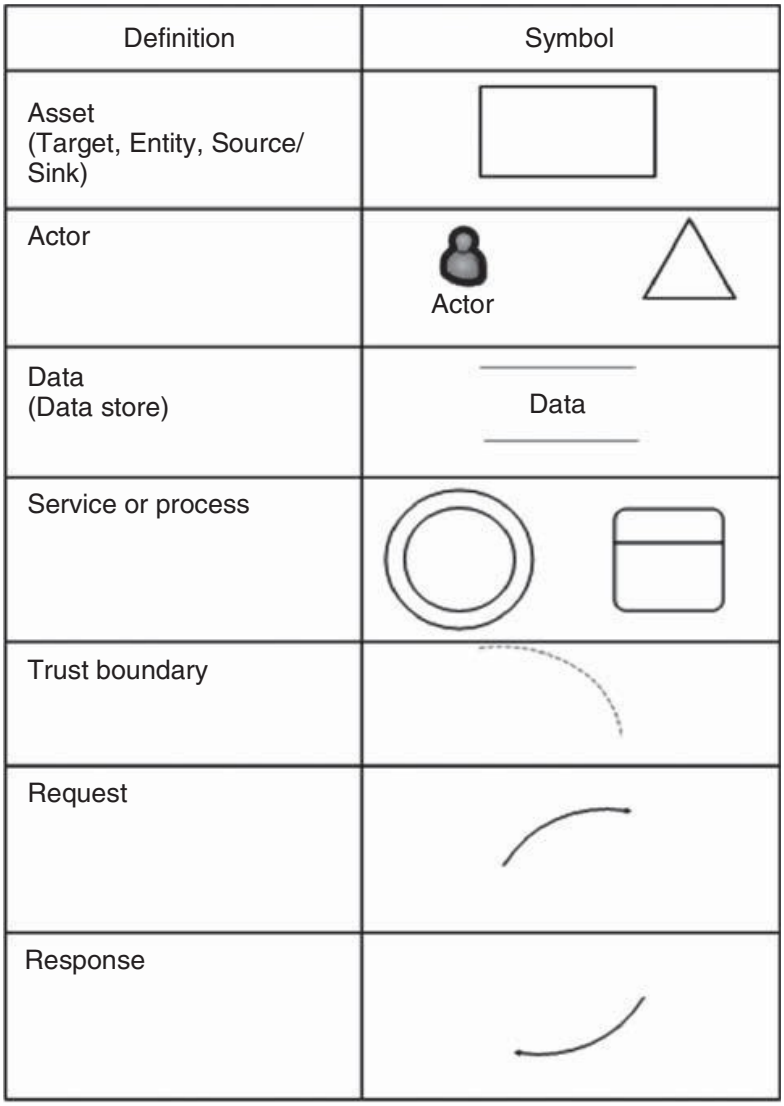


Figure 7.13 Common Syntax of Symbols for DFDS

Before we take to the whiteboard to pontificate on the application components within your application stack, two considerations should be made at this point. One is that a common set of DFD protocols and taxonomy of terms should be defined and understood for your organization. Although there is a common set of DFD icons and symbolic representations, it should be noted that DFDs should reflect the best use of terms and iconic figures that fit your organizational groups that are involved within stage III of PASTA. If a more generic and standardized set of representations is desired, then the following legend (Figure 7.13) should be considered, as they represent the common symbolic representations for components, processes, and requests/response within a DFD.

Labels in DFDs should contain descriptive information on what the call/request being made contains. It should provide sufficient information to denote the type of information being passed. Too much information can clutter the label and affect the legibility of the DFD. Add information that is material to understand the nature of

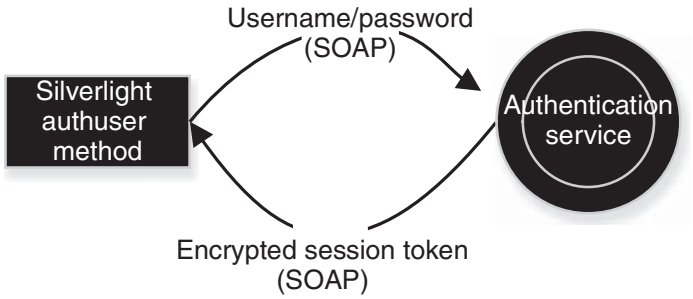


Figure 7.14 Data Flow Authentication Example

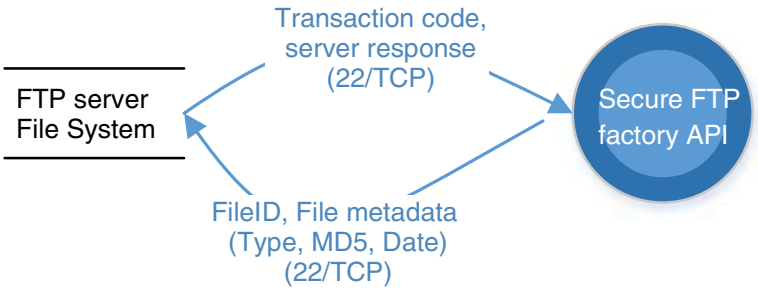


Figure 7.15 Data Flow for Data Exchange Across Two Entities

the call/request. The example (Figure 7.14) makes clear that the data flow revolves around web-based authentication. Developers can see what data is being sent/received as part of this request.

We see another example that includes the port/services used by application calls/requests. We see the target components (data sink of an ftp store) and the application server (source) that establishes a secure connection to send over files. Connection protocol and data descriptions of what is sent back is revealed (Figure 7.15).

In the following figure, the components from previously used health-care use cases have been mapped along with their application calls and responses. Although we will get into trust boundaries in detail shortly, note how trust boundaries are rendered as dotted rectangles around the components that are interfacing with a defined data store server. The trust boundaries represent distinct logical or physical boundaries between two distinct system applications, thereby using the physical characteristic as a basis for trust validation, previously covered in stage II of PASTA.

Note also that unique arrows were not drawn to the components under each system platform as it would have cluttered the DFD. However, the goals of reviewing the relevant actors as well as the use cases that trigger the flows among components are successfully depicted.

The point to be made in this section is that no two DFDs will look the same, particularly among organizations or even development groups. DFDs will vary in their appearance. The goal is to build a DFD that is understandable by *your* team and one that can be leveraged to denote where abuse cases may take place due to excessive trust awarded to application entities or insufficient amount of countermeasures.

As we look to yet another example of a DFD, it should be mentioned that the scope of components to DFD will be largely driven by time, topicality, and not creating an obnoxiously, complicated diagram. In this example, we will borrow from the previously mentioned Customer Profiling application mentioned earlier in Table 7.11 (S3:A1). As shown in Figure 7.16, the DFD is a targeted one; focused only on the self-enrollment feature of the application. Once again using the universal syntax of threat modeling icons, we see a whiteboard representation of how this DFD would look like (Figure 7.17).

All of the application components pertaining to this *use case* (not application) have been illustrated in the DFD along with the associated requests, responses, data, and main protocols utilized. Via the DFD see the application's use of Silverlight plug-ins, associated *.xap* files sent to the browser, it's interaction with a*n* MS Silverlight Web-Services instance and ultimately a data MS SQL store to where much of the user enrollment data would be kept. Although the aforementioned is a raw whiteboard drawing, the illustration is a key artifact for PASTA since it shows an activity that was largely worked upon by both security and development team members.

As one can see, all use cases *were not* depicted in the DFD and this is intentional. DFD exercises are best illustrated in manageable sizes. If all of the components for the application were to be illustrated, it may get too overwhelming to convey over one DFD. This in turn may discourage people to use DFDs in the first place or even application threat modeling. For this and other reasons related to simplicity of use, knowing what aspect of the application you need to DFD is important. Over time, the comprehensiveness of the exercises will be the sum of its parts (or other DFDs).

The online enrollment use case diagrammed also employs the most common syntax of DFD symbols. These symbols provide the most frequently used representation of application components and allow for some level of universality to be achieved for those who may have experienced DFDs before. It is important to point out that if the standard symbols prove to be nonintuitive for a collective group, it's perfectly fine to change what icons or symbols are used. Ultimately, since these diagrams are intended to be internal artifacts for those engaged in a secure SDLC implementation, determining what symbols to use should be based solely on what makes sense for your group. This is an example where best or common practices may not "best" for you and your team.

Using Containers to Organize DFDs Reverting back to the manner in which components are grouped via containers in stage II, we can use a technology list to consider components at a physical layer in the DFD (MS SQL Server, IIS Server, Human Actor) as well as things from a logical sense in terms of what they do (e.g. handle web requests, process serialized xml streams). Architecturally, the DFD can also be divided by where each component exists in the scale of the overall application flow. More of this architectural-based criteria will be depicted when looking at the use of trust boundaries in DFDs. The previous example was simply meant to take a manageable portion of the application and portray how it interacts with other components. It is worth mentioning again that one of the main benefits of DFDs is that they allow for multiple groups (both in IT and in InfoSec) to understand application flow among

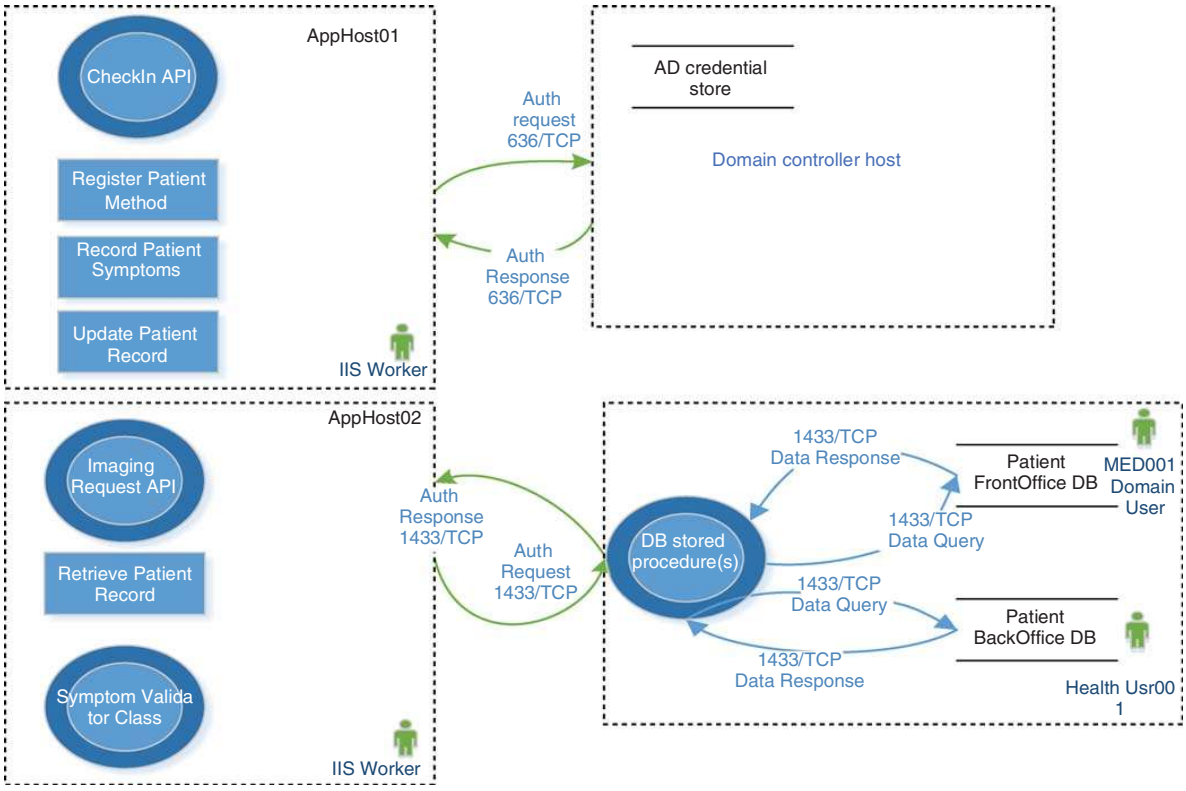


Figure 7.16 DFD Example Using Physical Boundaries for Organizing Components

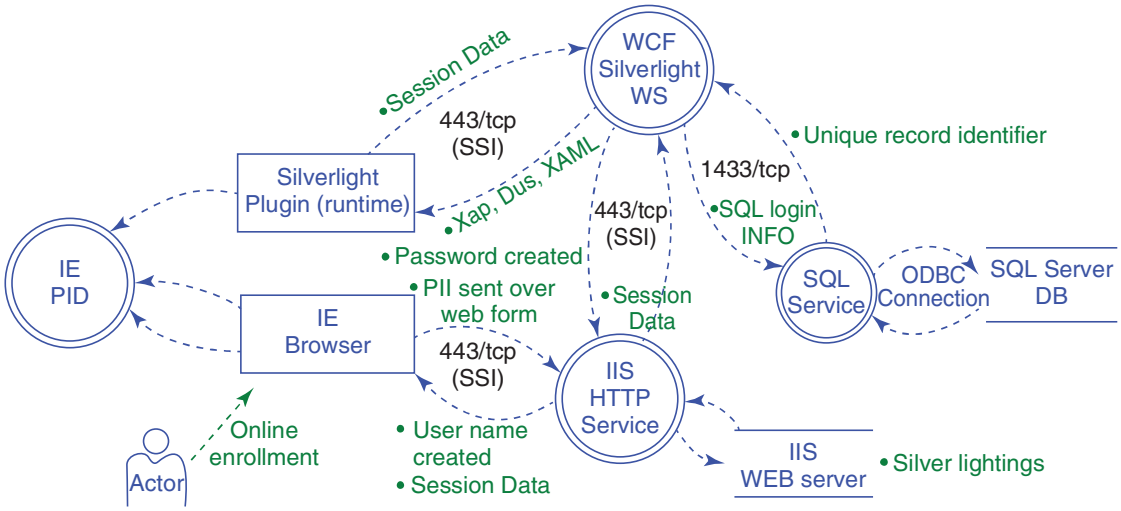


Figure 7.17 Whiteboard DFD of User Self-Enrollment

components that were not previously considered. It is in the awareness of how a subject application behaves that fosters security to naturally develop from earlier in the SDLC process.

Recalling our use of containers from S2:A5 (Classify & Map Application Components), we can begin to map out interactions of application components within a DFD that traverse logical, physical, or architectural boundaries. Leveraging the fictitious, health-care use cases from S3:A1 (Enumerate all Application Use Cases) (Table 7.11), we can map out the interactions among components via application methods, functions, calls, and APIs. The following is a separate type of DFD that uses slightly different symbolic representations but considers the relationship of logical application components to their physical hosts (Figure 7.18).

In application decomposition, the dissection process should address three areas for it to be thorough: architectural, physical, and logical. Although time may limit the depth of the decomposition process, future iterations of DFDs produced do have the opportunity to revisit how the application functions. Moreover, based on the criticality level of the components derived in S2:A4 (Assert Completeness of Secure Technical Design), the depth of dissection can vary for any iteration of a threat modeling activity for S3:A2 (Perform DFD of Identified Components). Over time, the criteria for building DFDs can vary, but as a basis it should consider components from the physical, architectural, and logical realm in order to be considered comprehensive.

The first criterion, architectural, focuses on what activities are taking place across an application environment and how threat modeling components are interacting with one another. These threat modeling components include actors (system/db/application), services, data sources, compiled binaries, batched/schedule programs, and hardware interfaces. Reverting back to our fictitious marketing application as an example, establishing trust boundaries across the application layers requires starting around the application architecture. This means that application calls and data flows from callers are mapped from data sources (databases), domain controllers, file servers, and so on to receiver sources. Actors associated with both caller and receiver

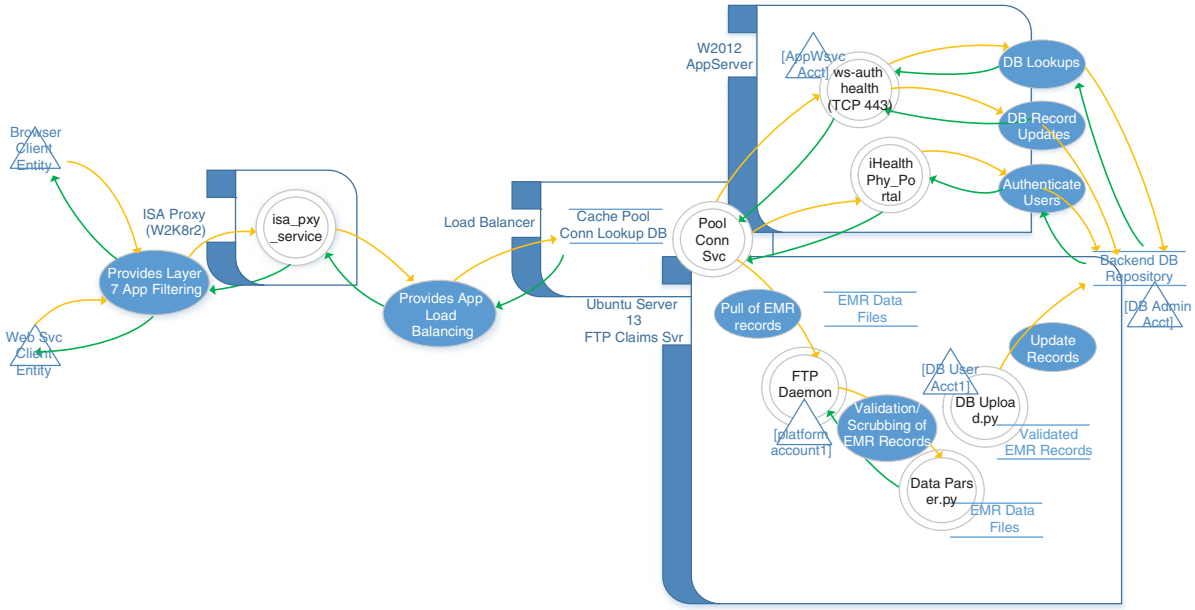


Figure 7.18 DFD Health-Care Example Using Container Approach

identities are also cataloged per call/data flow. Distinct callers, data sources, system environments (virtual/physical), software source, compiled binary source, and network segment (VLAN) would all contribute to the need for a trust boundary to be defined. In certain cases, trust boundaries should be even defined for when none of the aforementioned variables change, but this would be dependent on the risk profile for the application. A military-based application with top secret clearance may, for example, define more inner trust boundaries to increase security validation. Using security containers can greatly assist in this exercise as they define where many of these components reside within the threat model.

Grouping web servers within the DMZ, application servers in the application tier of the overall architecture, and lastly the database servers within the data layer, provides easy architectural wins for where trust boundaries can easily become defined. This is a basic depiction of using solely an architectural approach to define trust boundaries and does not include some of the other parameters that were recently discussed (such as data sources, batch/scheduled jobs, actors, etc.). This architectural approach also assumes threat modeling an environment that is traditionally located and managed under a single data center and not an application that relies on a distributed architecture model (e.g. multiple ASPs or Cloud service providers). We can touch upon this momentarily, but for now, we focus on a more traditional model in order to better understand the application S3:A2 (Data Flow Diagramming) efforts (Figure 7.19).

Stage III: Activity III (S3:A3) – Security Functional Analysis and the Use of Trust Boundaries Trust boundaries within DFDs introduce where new security countermeasures need to be developed. Both DFDs and embedded trust boundaries can be

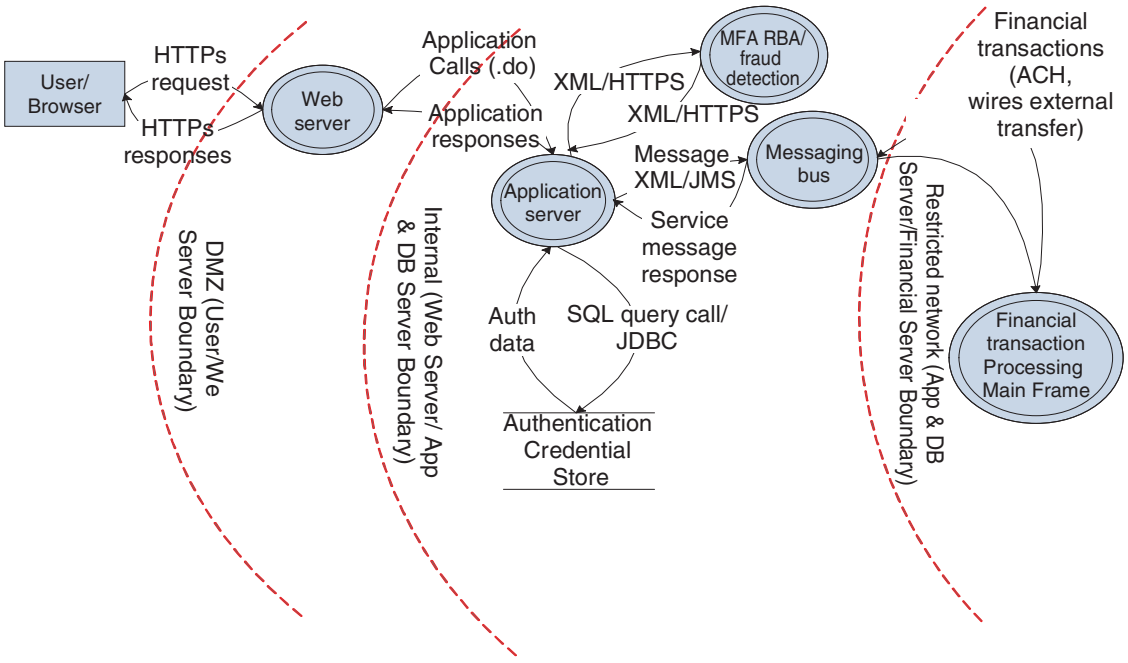


Figure 7.19 DFD Using Architectural Considerations for Component Grouping

best fulfilled as part of a Stand-Up SCRUM activity or in the Design phase of a Waterfall SDLC methodology. Note that trust boundaries are always represented using dotted lines between segregated application components. For this activity, use the architectural location of the assets identified in stage II to define unique trust boundaries between architectural layers and possibly among application components or assets. Although trust layers can be defined in multiple locations, the easiest will be using architectural layers. In doing so, you will be able to segregate the architecture into layers of trust where countermeasures can later be applied. Using the architectural approach, your DFD should have 2-3 trust boundaries defined among the presentation, application, and data layers. More can be used if needed. Generic rules of thumb for adding more trust boundaries include the following guidance:

- When and if data flows from one controlled network environment to another.
- Between sink/sources (external entities) and requesting process components.
- Among distinctly coded process components.
- Among data stores and process components that have a separate code base.
- When platform or system boundaries are crossed (Ubuntu Memcache server to IIS Web Server).
- Codebase changing between two systems where data flows exist.
- Groups responsible for software development/system engineering differ among assets in DFD.
- Actors among components/assets in DFD are different/have different privileges.
- Data request going to an external sink/source in DFD.
- Source (data source) to sink (data destination) of data flow is outside boundary of application.
- Data changes characteristics between two data flow endpoints.

Building off of prior activities conducted thus far, we can now effectively build a "trust model" into our DFDs given what we know of application components', such as use, physical location, architectural positioning, inherent risk, criticality to the overall application environment, and presumed range of actors using each component. From this information, members of the threat modeling team can collectively appreciate an illustration that defines trust boundaries among areas of the DFD. The goal of defining application trust boundaries is very important to PASTA's threat modeling methodology because it is the first time all involved RACI participants can see where and why distrust should be present within the application and from which container or container object threats may originate.

Examining the implied trust relationship among application components needs to be done methodically, particularly when the impact rating of the asset is determined to be HIGH. An example is a handheld device's biometric authentication control that interfaces with the client application software stack on the device. If the mobile device's security was solely dependent on the device's hardware security control measures, then any hardware-based circumvention or hardware-based exploit could

undermine all of the mobile software security features. Applying this same example to Supervisory Control and Data Acquisition Systems (SCADA) in the Energy sector, one may have a more complex and impact-intensive threat model. An example here is the overlay of modern day graphical user interfaces (GUIs) to traditional administrative access and functions on an industrial control system (ICS) that traditionally may have only had console or serial cable access from a dedicated host and may now support TCP/HTTP protocols, which give life to a web UI for ease of access.

Leveraging how application components are organized under S2:A5 (Classify & Organize Components), containers can help facilitate where possible trust boundaries need to be established among actors, assets, technology assets, services, data, web services, APIs, and compiled binaries that interface with the SCADA environment. Defining trust will extend beyond how these components have been classified and will also consider how these components' authenticity, integrity, and authorization is validated or verified within the application model, based on their physical, logical, and/or architectural proximity and similarity to one another.

Architectural considerations to application decomposition presume that similar asset types have been grouped appropriately based on similar functionality. This essentially creates layers for a given application environment. These definable boundaries are aimed to separate distinct actions among processing entities for the application environment. A traditional three-tiered application environment may have two trust boundaries that separate the web layer from the app layer and the app layer from the data layer. As an example, web servers are not architecturally placed within the same trust boundary as client machines because these two distinct computing environments are managed by separate user groups and there should not be an assumption that they are being managed equally well and with equally judicious responsibility on security configuration. As a result, a trust boundary would naturally be placed between these two distinct architectural areas. A few application characteristics to help identify the location of where a trust boundary should be placed include the following:

- Unique entities are receiving unique data requests for processing.
- The Actors (or Callers) are unique among the sending and receiving processing agents.
- Technology or platforms vary greatly among the architectural layers of the threat model.
- Types of protocols across the application layers changes (e.g. web traffic only in one layer vs. XML-SOAP, SAML requests).

Architecturally speaking, it is a common approach that external facing assets should not receive any level of trust from inner layers hosting other application/data components. The reason is quite simply playing the role of devil's advocate or not hedging your bets against Murphy's Law.[3] Later under stage IV of PASTA, this level

[3] Anything that can go bad, often does.



Figure 7.20 Spectrum of Trust for Defining Trust Boundaries Across Architecture

of "paranoia" may be substantiated by internal logs and alerts or via external threat intelligence. More rationally, however, it is quite easy to understand that a semipublic network environment, such as a DMZ, may have already been compromised. As such, downstream communication to inner network layers should be questioned and validated. Architecturally focused approaches to placing trust boundaries within a DFD traverse a trust spectrum. A simplistic representation is reflected in Figure 7.20.

These architectural access levels help the threat modeler know that some form of countermeasures or controls will be needed in order to safely pass along requests from other assets/actors to target assets across other trust boundaries.

In general, trust boundaries are placed where boundaries of trust should be formed. These boundaries provide the foundation for considering various types of countermeasures around the calls made across the trusty boundary, both upstream and downstream to the application. The benefit of grouping assets based on trust levels is that it helps to develop or implement preventative, detective, and reactive countermeasures. From a preventative sense, trust boundaries help enforce proper access control measures among boundaries via the network, platform, and beyond. Trust boundaries also aid in detective means, providing audit-logging capabilities that can be fed to a Security Incident and Event Monitoring solution or central logging repository. Last, reactive measures can also be considered if the product application is okay in having automatic security responses be applied and potentially blocking certain access or network request automatically. This would equate to more active defense measures for example.

Beyond architectural-focused efforts in defining trust boundaries under stage III, additional trust boundaries may be warranted based on the other two criteria previously mentioned – physical and logical. Architectural considerations for trust boundaries provide a low maturity level for establishing trust boundaries since assets are grouped by a network VLAN or generic categorization of use (network services, web servers, application servers, database servers).

The easiest example of where physical considerations should denote that unique trust boundaries be defined are physical assets located in distinct networks, potentially that of a third-party service provider. It should be easier to understand the applicability of physical considerations in establishing unique trust boundaries since distinct networks, asset custodians, physical infrastructure may be noncongruent to those processes/control measures at a primary data center. It is likely, however, that a trust boundary had been previously defined with architectural considerations alone since a VPN connectivity, P2P connection, or API over SSL would reveal a separate network that would be architecturally apart from a presentation layer or application layer of an overall application solution.

More advanced threat models may extend beyond the architectural approach and leverage physical considerations for defining inner trust boundaries. In more sophisticated threat models where the impact or risk rating of the application scope is higher, physically separate servers will most likely warrant a trust boundary defined among other physical assets since the uniqueness of such an asset will encompass threat modeling variables that may be extraneous to those already enumerated (new actors, new services, new APIs, etc.). This requirement for a unique trust boundary will be affected by a few factors such as those listed in the following checklist:

- Is the asset currently found within a trust boundary defined by architectural criteria?
- Is the asset unique in its code base as compared to other assets within its same architectural realm?
- Is the asset host to a third-party application service/server?
- Does the asset physically host many virtual hosts, thereby building a unique application ecology within the physical realm of the asset?

The aforementioned list is good in considering additional "inner" trust boundaries that may need to be defined within a previously defined trust boundary, based on architectural considerations. Inner trust boundaries in general are found in more advanced threat models, since the interrelationship of applications and servers are more intricate and the data potentially more sensitive or critical. In most cases, it is highly unlikely that a trust boundary becomes defined solely based on physical criteria and not inclusive of architectural considerations. Even if a web server is found on the same physical host as a Java application server, virtual network/architectural segregation is possible, so architecturally, the segregation and trust boundary could be defined based on the need to achieve *architectural* separation. The greatest exception to this, of course, is if a physical host server encompasses several virtual guest machines. In relation to virtual assets within the threat model, they should be regarded as separate "physical" hosts and thereby potentially require trust boundary to be defined among them and other unrelated assets. The cloud-based scenario further complicates this and requires using the third criteria, logical considerations to further refine where new trust boundaries should be applied. Even within the ecosystem of virtual hosts, review of how virtual networking and virtual hosts are used and segregated should be evaluated for providing trust boundaries within such a virtualized environment.

Thus far, physical considerations to trust boundaries extend the basic architectural methods to decompose an application environment. Logical considerations increase the capabilities of application decomposition even further into more advanced techniques based on the logical flow of the data (APIs/feeds/scheduled jobs), the use cases for the application, data repositories, and identity/origins of the actors. This extra factor for possibly defining new trust boundaries should be applied similarly to other criteria, but primarily in considering the following:

- Are there new actors involved at the host or application level (upstream/downstream) that will be handling the data being sent/received?

- Is the code base for the receiving end point of an application programming interface different from that from which the data originated?
- Is the actor's identity and related security context as a user different between the source of a data exchange and its destination?
- Is exchange of commands or data extending across two distinct interpreter environments?
- Are the utilized use cases elevating or reducing the criticality/sensitivity of the information being exchanged?
- Is the application supportive of polymorphic data or one with an abstraction layer?

These aforementioned checkpoints help determine whether additional trust boundaries need to be depicted in a DFD exercise and extend the architectural and physical conditions for using trust boundaries. Logical considerations to defining trust boundaries are truly reserved for more complicated or advanced threat models where the importance of the application data or criticality of the services provided by the system is high. Similar to physical criteria for defining trust boundaries, logical considerations can be applied using a layered approach that incorporates all of the approaches mentioned. Overall, each approach should be considered and applied based on the context of both the application use cases as well as the inherent risk profile that pertains to the application.

Beyond introducing trust boundaries within an application, this stage III activity aims to create opportunities for other activities such as enumerating actors, unique data sources, services, kernels, interpreters, client technologies, or simply any place where alteration to data, schema objects, application commands, or source code could be altered. The dissection is typically achieved as part of a collaborative work effort among developers, system administration, network engineers, and architects. In some cases, one individual with in-depth knowledge of all these areas may possess the needed expertise to properly dissect the application; however, they will commonly rely on the collaboration and expertise of multiple SMEs for this type of stage II activity. The components dissected in this phase tie in the previous logical considerations around defining trust boundaries. Identified services, actors, and data sources all help to better define trust boundaries for the application in scope.

The DFDs presented thus far adhere to the common syntax of DFD symbols; however, if more high-level depictions are desired, DFDs such the following in Figure 7.21 can provide architects, business analysts, and project managers with a high level and very easy representation of where trust boundaries should be defined. The following figure represents an Android-based mobile application. It is not surprise that key components include a mobile client front-end communicating with a two-tiered, web application environment. In the diagram that follows, we see how the physical, architectural, and logical considerations that help define trust boundaries within the DFD.

Figure 7.21 shows various trust boundaries in shades of red with different labels; all depending on the types of trust boundary formed using the architectural, physical,

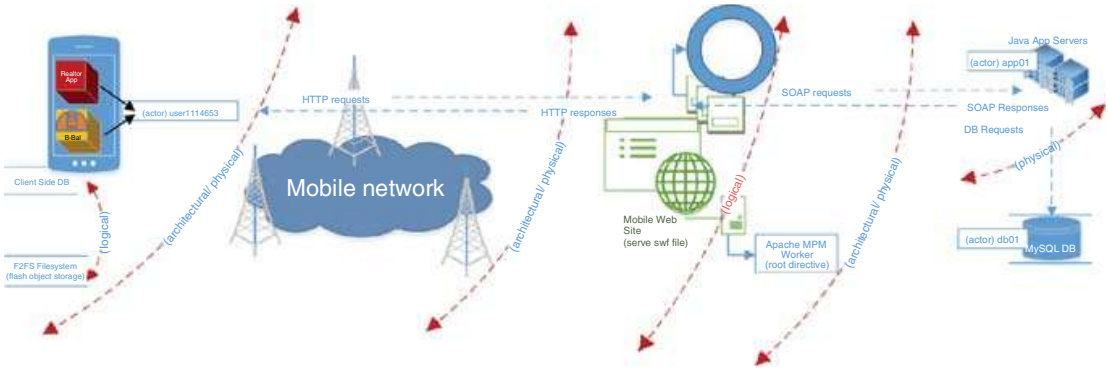


Figure 7.21 Decomposing Mobile Web App Example

and logical designations. This simple decomposition does not comprise all the possible actors, data sources, data flows, ad use cases that would be included in stage III of PASTA. However, the figure does reveal how application decomposition should begin at a high level. Depending on the inherent risk profile for the application, where greater governance of the software development actually takes place, greater focus may take place at the client side or the server side of the environment. Focusing on each of the embedded trust boundaries, further decomposition of application components can take place at the server level, application layer, network, and even client hardware device – in this scenario the mobile handset. The point to be made is that embedded flows are never too small or insignificant or too large not to be considered something to decompose the application into data sources, actors, requests/responses, processes, compiled objects, and more.

Do not Forget the Physical Interfaces You may determine that including nontechnical components in a threat model is nontopical, but we can guarantee you that the attackers will not. While decomposing the application in stage III of PASTA, it is also important to keep an open mind as a threat modeler on attack scenarios. Many security professionals and technologists will simply focus on areas they are most familiar with or ones that have been most discussed in the media. This is a disservice to any type of application decomposition exercise as it becomes biased to components that are "marketed" in security media and not based on relevancy to your product application. As an example, technology-focused mindsets may simply center on OSI layers two and up, while missing various physical elements that could directly or indirectly affect the security posture of the overall environment. For this reason, a balanced regard when dissecting the application environment is critical. If a physical exploit is possible and viable in a postimplemented scenario to a product or software application, countermeasures would need to be considered at the other layers of the OSI model as well.

A great example of this is in Figure 7.22. At first glance, the figure reveals an application's architecture with fairly secure transportation measures, employing both SSL and IPSec tunnels between satellite store locations and the corporate sales server where centralized daily receipts are processed and stored. A secure perception may even be sustained solely by an application risk assessment or even a software-centric threat modeling methodology. The PASTA methodology, however, will also

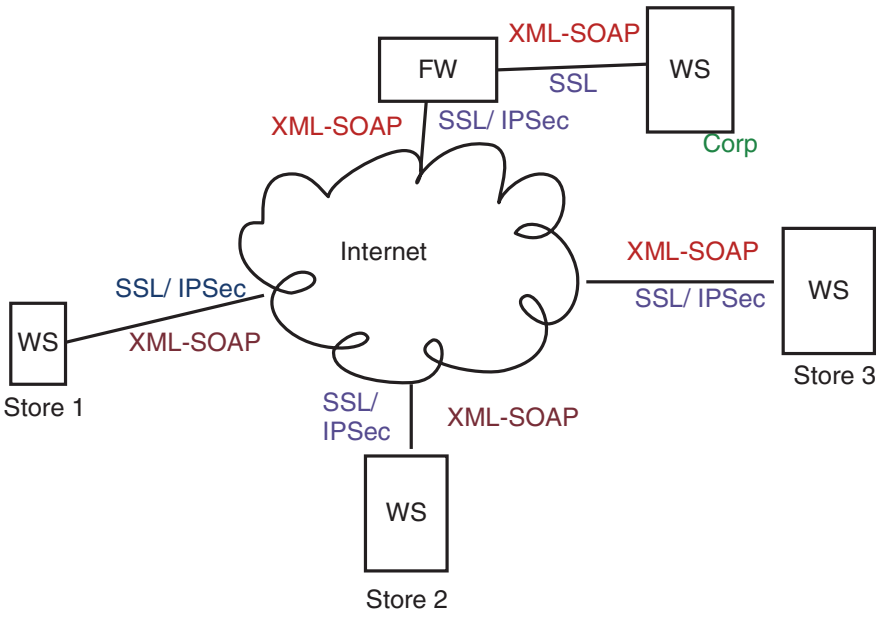


Figure 7.22 API from Stores Local Transaction Server with the Following Metadata

consider physical or environmental factors that may affect the security of each individual store's web server or simply the introduction of rogue web service clients wishing to make unauthorized data interfaces with the corporate web service namespace. The physical scenarios for such authorized requests can range from a stolen or tampered-with web server at a store location to physically introducing a fake Certificate Authority (CA) or DNS server in the local environment. In either case, an altered host, or one that is successfully inserted into the local architecture of a POS environment, can introduce a perpetrating host containing unsigned or tainted compiled library files.

In Figure 7.22, we see the use case for batched or time driven XML-SOAP feeds initiated by store web servers to their corporate operations web service end points using SSL communications run over a VPN tunnel. The data format is XML and the serialized data sent adheres to XML-SOAP calls under Microsoft's .NET Web Service Framework.

Under the depicted architecture, if such a physically compromised server had no authentication/integrity checks to application requests/responses, this can easily affect use cases related to authorizing or processing unauthorized credit card charge requests that could not be processed at the time of a transaction (which is not uncommon in a merchant retail environment). Even if authentication criteria are factored into the compiled libraries or binary files of each store's web server, reverse engineering such code in the scenario of a physically compromised host is not only possible, but also plausible. This is especially true if unsigned libraries or SAML requests are not employed per transaction where additional authentication checks related to identity of requestor are not validated.

Once again focusing on one of the use cases in a DFD allows us to more deeply explore areas where possible countermeasures may need to be developed. The aforementioned figure provides a backdrop to the subsequent security analysis that helps the threat modeler compartmentalize use cases reflected by the application architectural diagram.

Participants of Stage III – Application Decomposition

Understanding data flow is of utmost importance in stage III. As a result, who that work with data and know the data flow that it is designed to take across/within an application environment will be best positioned to speak to how trust boundaries should be defined and where appropriate countermeasures should be developed. The knowledge of the data environment and its role within the application are key in helping further define possible attack vectors and, most importantly, what types of countermeasures should be developed at various levels, whether it is at the data level or at the network/client level. Beyond the threat modeler facilitating the activities for this stage, those who play a *Responsible* or *Accountable* role in this stage are Developers, Architects, or System Engineers. Depending on the level of knowledge that BAs may have on proposed data flows, their roles may evolve from an *Informed* role to a *Consulted* role (based on Figure 6.8). All of the other defined roles will be working with BAs and PMs already in order to gather the high-level specifications around data usage so they will remain instrumental in executing many of the activities for this stage.

Stage III Deliverables

Practical inputs of this stage of the PASTA methodology could benefit from an array of different artifacts that the threat modeler could leverage as part of this stage. This list of artifacts as possible inputs is detailed in the following table. The lack of any of the following does not require these items to be produced, as much of them can be produced during the vulnerability analysis phase of the PASTA process.

Input	Description and Use
Configuration baselines	Provides a roster of configuration settings at the platform level as well as for any major software installation currently present on an asset within the threat model.
Software enumeration	Provides a roster of possible security countermeasures or pivot points of attack from a software perspective.
Network service listing	Provides a roster of possible network entry points that may be vulnerable for exploitation.

Input	Description and Use
Hardware (asset) enumeration	Encompasses a list of assets (e.g.: web, database, fileserver, software, network devices, VoIP equipment) for the application environment based on the proposed architecture to be used.
Identifying actors (role listing)	Considers the role of users (within an RBAC model) at various phases of the application and how they could either sustain or unfold security measures across the scope of the application environment within the threat model.
Manual processing	Considers human roles or physical controls that may be involved with manual processes the application environment depends on. This may include biometric measures, SmartCard technologies, call back authentication techniques, transcription services, data entry, courier services (tapes), physical records archiving, physical tape handling/processing, and so on.
Third-party roles	Role of third parties and similar listings of the aforementioned related to any infrastructure that will help either support or fulfill services on behalf of the application environment within the threat model.
Personnel considerations	Operations folk across technical, support operations that may be targets of social engineering ploys or those who have carte blanch, unaudited, or unmonitored access to key assets in the application environment.
Captured network flows	Beyond common network services such as DNS (53), SNMP (161), NTP (123), many architects do not have a strong handle on the amount of network services leveraged by the scope of application servers under a threat model. As a result, captured net flows for a brief period of time (month) offer invaluable data to get a glimpse of common network traffic that support valid use cases.

Stage III Summary

It is very easy to lead into the next stage of PASTA (Threat Analysis) without reviewing all the possible inherent flaws or strengths that a DFD has to uncover under the Application Decomposition stage. It is very tempting to follow the security rabbit hole by leaping into tangents where countermeasures or risk analysis can naturally take place. Instead, the threat modeler should ensure proper breadth of coverage in understanding the technology leveraged within the boundaries of the application

environment and what likely threats should take precedence in their analysis. This is where PASTA diverges from other application threat models, as it attempts to focus on the most viable threats both inherent to the technology footprint of the environment as well as the industry segment and the data managed by the application environment.

The key takeaway for this stage's activities is that Application Decomposition is a stage in PASTA where comprehensive coverage should be applied to enumerate underlying platforms, technologies, software (and related versions), COTS (Code Off The Shelf), network services/equipment, physical security, and the overall process that governs all of the aforementioned for the scope of the application environment. Under application decomposition and the use of trust boundaries, PASTA provides considerations on where abuse cases may give way to data-focused attacks, platform continuity, authentication bypass, data integrity violations, and so much more. All of these types of considerations become more precisely defined, particularly during the threat analysis phase (stage IV) of the PASTA methodology.

Many of the activities in this stage leverage the use of a produced DFD. DFDs provide an illustration of how application components (from stage II) relate to one another. This is achieved through both analysis and some simple white-boarding efforts. The dissection or decomposition of the application truly is simply taking the enumerated components and mapping out their requests/responses with one another. In computer science, application decomposition or *factoring* provides a good form of dissecting a complex problem into more digestible components. Revealing application components and their calls to other components within the application boundary provides a visual depiction of use cases in the DFD.

Traditional assessments do not have activities that improve the level of understanding around how application components interact with one another. For most security assessments, knowledge of use cases, actors, or trust boundaries is part of a black box. For this reason, PASTA's application decomposition (or stage III) is a valuable stage to this risk-based threat modeling methodology. The stage III security analysis scrutinizes and focuses on network components, application services, on-demand events or triggers, RBAC models, batch processes, and more.

Already in stage III, as we decompose our applications, threat modelers must be cognizant of what inherent threats could be a factor. However, the benefit of how stage III is carried out is that its primary focus is enumeration exercises across physical and logical entities in order to simply determine what actors and use cases are supported by the application. A summary of key activities that should be performed in stage III is recapped as follows:

- S3:A1: Enumerate All Application Use Cases (e.g. login, account update, delete users).
- S3:A2: Perform DFD of Identified Components.
- S3:A3: Security Functional Analysis & the Use of Trust Boundaries.

In the next section, we will take a look at stage IV – Threat Analysis – which will now bridge the knowledge of what we know of the application to that of what we perceive as a threat agent to its understood business objectives and identified use cases. The following is a visual summary of stage III (ADA) (Figure 7.23).

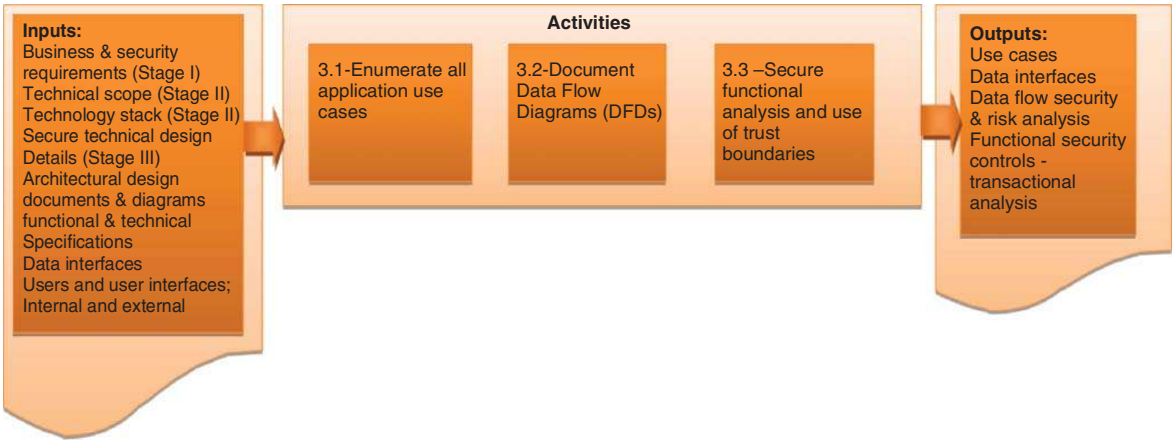


Figure 7.23 PASTA Risk-Centric Threat Modeling – Stage III – (ADA) Application Decomposition and Analysis

Stage IV – Threat Analysis (TA)

Reexamining Threats Before we dive into threats, let us understand just how relative that word ("threat") really can be. As one may expect, there is no universality of threat to all industries, companies, or applications. Just like criminal groups and attackers do not have the same level of appreciation for information or technology assets across industries, companies, or applications. This being said, let us focus on the three major threat targets to which threat modeling can help protect: Data, Infrastructure, and Human Life. Directly or indirectly, these are the ultimate targets worth defending. For level-setting purposes, we examine all three below:

Data: Private or sensitive information pertaining to end users; also includes system information for which access or elevated assets to various parts of the application infrastructure could be obtained. In essence, data can be summarized into PHI or PII. If the product app does work with sensitive data, then there may be a clear threat. Therefore, a value should describe the level of threat to the data based on its attractiveness for both internal and external attack agents. We use a rating of high, moderate, or low for the assumed threat that the data naturally invokes given its usefulness in the black market or in other illicit venues where PII can be sold for pennies on the dollar.

Infrastructure: Beyond the attractiveness of data is the attractiveness of a robust infrastructure. Attackers who have knowledge of the power and sophistication of an infrastructure would truly enjoy being able to leverage the collective computing power in order to launch other types of attacks. This is a well-developed threat that in certain instances may be heightened if the computing environment gets worldwide attention. Likewise, less robust computing environments have traditionally received less intrigue from attackers, although recently in 2014, mobile-based Android attacks have surged seeking to leverage a small amount of computing power but in a collective network of compromised mobile devices. In the end, as the distributed nature of attacks continues to evolve, we are going to see a possible target-value shift from powerful distributed servers that are plentiful in the physical and virtual space to smaller computing assets, bound by a common malware agent able to perform essential malicious computing functions, consistently well over time.

Human: Wearable medical devices have been around for longer than most people think. Coupled with embedded software, wireless technologies, and a thin software footprint, human life is now well within the realm of exploitation and has been for nearly a decade. Before it may have been a lobby-con joke to think that JavaScript can hack humans, but now they really can. A recent example came in 2013/2014 when researchers announced that apps compiled against the Android 4.1 Jelly Bean API can exploit a bug in JavaScript. (Small tangent: both the Android 4.1 Jelly Bean API and the Google Glass itself would be listed as components to the threat model in stage II). The resulting exploit would allow hackers to obtain a live view from users of the Google Glass. All depending on

the human – their role, position in society, wealth, status, and relevancy to a large plan can make human targets quite prevalent, particularly in the military when more integrated and wearable technology becomes introduced.

The point of identifying these threat areas is to have these considerations visible when fulfilling activities in stage IV. What could be intended threats around application *X?* Is it really the data? Is it something greater? Could other aspects of the application environment be really the focus (e.g. – human life, introducing service downtime, etc.)? These are key questions to consider as we fulfill the activities in this stage. In general, *threat*, much like *risk*, is going to be relative; relative to your data and your infrastructure. A practical guidance will be to continue monitoring emerging threats in a highly evolving Information Technology landscape in order to correlate what architectural, programmatic, asset, industry, or information related threats are relevant to the product application being threat modeled.

Stage Objectives The objective of this stage is to identify the relevancy of threats to the application being threat modeled and/or its underlying components. Thus far, in this risk-based approach to threat modeling, we have only really looked at a few variables in the risk formula, such as inherent risk and business impact. The viability of threats will be regarded in this chapter, thereby beginning our analysis of the right side of the traditional risk equation.[4] As all possible threat examples cannot be covered, the key takeaway from this chapter will be on how to identify good threat intelligence and apply them in estimating probabilistic threat instances that are credible for defensive measures to take place.

Threat feeds. Threat intelligence. Threat Data. Threat Landscape. Let us not forget Cyberthreat. There is a lot of threat talk out there and there is no security conference with a vendor hall in the world that will allow you to forget it. Beyond simply using it in security speak, it is important not to have the word lose value, particularly as it relates to threat modeling. In this portion of the threat modeling methodology, we focus on simplifying the term's use to two key areas: threat data and threat intelligence. Threat data is the data that is used to make effective threat analysis. Threat intelligence is the data that is collected and already analyzed, and largely served by service providers as a service feed for your operations center. In fulfillment of a risk-based threat modeling process, we are keen on having accurate and well-founded threat data and threat intelligence. In doing so, we can drastically improve the threat modeling and overall risk analysis for threat mitigation.

Stage IV of PASTA begins to augment the level of risk analysis initiated in stage I, where risk profiling and business impact considerations were established. As part of the Threat Analysis, we will begin to substantiate various possible threat scenarios that are based on sound threat intelligence. This section will define how to interpret threat sources and apply them to existing or new software applications. Analyzing

[4]Risk = Threat × Vulnerability × Impact (Basic Risk Formula).

threat related data and tying it to application threat modeling is not done today – this is unique to PASTA. Even beyond threat modeling, threat aggregation is only taking place across major enterprises that have the time and resources to consume this data. Even in these circumstances, the received threat intelligence is generally consumed by a security operations team and if security measures are taken, they are localized to being primarily network driven in nature. Isolated from any level of threat knowledge are software development groups as well as system engineers who may be responsible for system/database administration. This is yet another demonstration that even with good threat intelligence, it means nothing unless properly consumed and applied within meaningful areas of the greater network architecture.

Stage IV's key objectives include the following:

- Review credible, diverse sources of threat data.
- Leverage internal sources of data, originating from security incidents, log/alert data.
- Enumerate likely threat agents who may be able to carry out supporting attack patterns for given threat.
- Identify the most likely threats to the application (should be less than 5).
- Determine a threat likelihood value for each threat that is developed.

The aforementioned objectives are intended to be fulfilled using the following stage IV activities listed. Each of these activities will be covered in the sections that follow:

- S4:A1 Analyze the Overall Threat Scenario
- S4:A2 Gather Threat Intelligence from Internal Sources
- S4:A3 Gather Threat Intelligence from External Sources.
- S4:A4 Update the Threat Libraries.
- S5:A5 Threat Agents to Asset Mapping.
- S4:A6 Assign Probabilistic Values around Identified Threats.

Stage IV:Activity I (S4:A1) – Analyze the Overall Threat Scenario The goal in this activity is to identify threat patterns that are most likely targeting similar application types with similar architecture, data use, deployment models, and other technology-related characteristics. Simply said, the goal for this activity is to list threats against the application. Some of the threats will be inherent to any type of deployment model. For example, adversarial goals for botnet propagation are indiscriminate to architecture, platform, hosted model, and other application environment characteristics. Some threat patterns will not apply based on the application type or even industry that it is in. Take for example inventory applications of automotive car parts. There is very little mass use for such information to be compromised or even be affected from an integrity or continuity point of view. As such, considerations for threat patterns may be lessened. Of course, there are always exceptions, especially

if the information around a car part can be used as leverage as part of an overall threat. However, such a targeted attack would be addressed under this activity of stage IV – Threat Analysis.

Considerations for viable threat patterns are a lot simpler than it sounds. The world of InfoSec will have you believe that if you do not start by consuming their threat intelligence, you will not know how to protect your own environment. I think that may make more sense in highly distributed, noncentralized, and outsourced environments, but if you have a good hold of your product/application deployments, your team within your company will be best suited to say what threats are most likely to happen.

For this activity, internally assessing the perceived threats from members of the product team, application team, and senior management will undoubtedly provide the most realistic threat patterns that could possibly affect a product application. Ideals team members for this activity would be middle managers, product managers, IT managers, and senior business executives who are familiar with the application and its intended use and benefit. Conversations around threats to the application and company as a whole should be discussed. The reason for such a broad scope is that threat agents may not necessarily be looking at an application as a sole target but simply a means to a bigger target, particularly if the application itself is publicly accessible. For this reason, it is important to begin with threats that are germane to the application environment followed by threat assessments on what threat motives could include neighboring data or application access from the subject application that is being threat modeling. The following is a simple representation of the order of the threat analysis.

As shown in Figure 7.24, the selected team members will provide quick ideas and evidence as to what data, components, human/physical assets, and/or third-party infrastructures are most likely to be prime targets. Collecting the thoughts and feedback of team members will yield a summary of threat possibilities that reflect possible motives and targets at different levels. Looking at these variables in a cross section of different applications for different industries is presented as follows.

Table 7.12 provides a quick way to list threat scenarios for any type of application. From such a simple breakdown, we can further build upon what we have built in terms of a threat model for our application. In this activity, we can begin to focus on the data, human, physical, and third-party targets within our application, along with their associated threat motives.

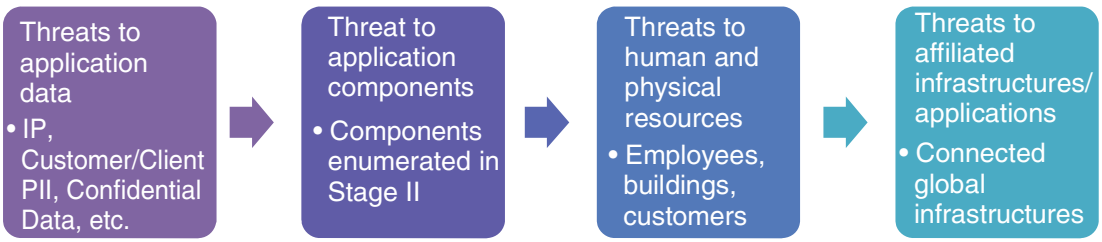


Figure 7.24 Areas to Consider around Threat Evaluation

TABLE 7.12 Sample Threat Considerations for Various Applications

Threat Considerations	Online Banking App	Transcription Software (Health care)	Multitenant Cloud Service Offering (CRM)
Data component target	Customer PII, Credentials, Bank account info, Source code, admin credentials, certificates	Patient Health Info	PII of online records, Business Intelligence/ Recon, Credential data,
App component target	Published web service APIs, mobile client computing apps, Source code repositories, domain controllers, DNS servers, CA server, token server	Backend data server, fileserver components, Stored library components, source code repository	Virtual Host Controllers, Virtual Network Management Interfaces, Management Interfaces, Published web service interfaces, FTP services, Administrative interfaces to virtual infrastructure
Human component target	Not likely	Not likely	Not likely
Physical component target	Not likely	Patient	Main Data Center Facility
Affiliate target	Trusted third-party development or IT shop	Outsourced development shop	Redundant site operations
Possible threat agent	Hacker syndicate groups, individual hackers/hackers for hire	Hacker Syndicate Groups, Individual hackers/hackers for hire	Hacker Syndicate Groups, Individual hackers/hackers for hire
Possible threat motive	Identity theft, financial fraud	PII theft, Affect data integrity of PHI (misdiagnosis)	Mass compromise of multitenant PII information and/or systems & infrastructure

Without making any correlative analysis to global trends, it is possible to make simple associations to threats per industry. Certain threats are obvious, such as the threat of financial fraud through credit card data compromises. Others are less obvious, such as the compromise of an obscure third-party parts manufacturer. From information leakages to network breaches, knowing what the threats may be pursuing for the application that is being threat modeled is a question that most product/software team members should be able to answer. Companies that can go through similar exercises, as shown earlier, will quickly discover that they can narrow the focus to a few threats that can be factored into the threat model. Simple inherent threat targets that companies of various industries can factor into their threat analysis are listed in Figure 7.25.



Figure 7.25 Sample Threat Possibilities per Industry

The last figure was simply meant to be a quick representation of top threats that affect various industries. During the threat analysis, this information is useful in order to determine if the application being threat modeled is subject to the threat possibilities depicted at a macro level. From there, it will be important to narrow down a generic threat against the components identified in stage II of the PASTA methodology. Some questions to pose members of the threat modeling technical group would be the following:

- What current threats exist that are specific to the application model and employed infrastructure model?
  - Sought-after responses will help determine what specific threats are taking place against foundational elements of the product application. Example: Threats to Ruby on Rails framework, exploited in 2012–2013.
- What current threats exist related to key vendors/business associates that may undermine the security posture of the product application?
  - Sought-after response should look to identify process-based risks from operations or third-party components that are to be leveraged by the application.
- What current threats are affecting the industry or underlying target users of the developing product application?
  - Sought-after answers should address threats that have been more prevalent over time, especially in recent history.
- Has the "attractiveness" of the data changed recently? Does it contain more PII than before or information that can be leveraged by the criminally inclined?
  - Sought-after responses would be the ones that identify whether the application data has changed to possibly further entice malicious actors to have an increased appetite to target the application data. Example: LinkedIn's "Intro Service" introduced in 2013–2014.

Threat Classification Some threat modeling methodologies will suggest classification of threats in order to better organize threats into compartmentalized components. If this is beneficial to you and your team, you may consider placing your depicted threats into threat classification models such as STRIDE or DREAD – both from Microsoft. For PASTA, this is not needed; however, it could be easily incorporated into this activity within stage IV if desired. Simply after defining possible threats from the last activity, assign to a category from one of the predefined threat categories in STRIDE (Spoofing, Tampering of Data, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privileges). Some find it easier to see if threats fall into one of these categories, especially if STRIDE and/or DREAD have been introduced along with their respective taxonomies of terms. The only problem is that attack patterns and vulnerabilities do not also leverage the same classification model, thereby making the use of STRIDE or DREAD to be useful within a threat analysis phase of a threat modeling methodology. PASTA does not need these categories to be used in this activity, but instead a conscientious list on viable threat patterns from which attack trees will be later built from.

As stated several times thus far, the activities and stages in PASTA build off of one another. This makes it linear and easy to collaborate on. As we look at the following activities that support stage IV of the PASTA process, it is important to consider what unique attributes and conditions of the application may also influence the types of threats that could materialize. Everything related to deployment location, personnel operating and managing the application, and especially architectural considerations should be always considered when considering what types of threats are most likely to be aimed at the organization and specifically the application being threat modeled.

In this activity, we focused internally on what key members of the organization considered as the most likely threats to the application or product being developed. In the following activity, we will look inwardly again to the organization, but this time not to its people, but rather its process and technological controls for possible guidance on what threats to ultimately consider as part of the overall threat model.

Stage IV: Activity II (S4:A2) – Gather Threat Intelligence from Internal Sources
This activity can be summarized in one sentence:

- *Leverage historical incident reports and security alert data.*

This may be easier than it appears since legal may have different thoughts about people outside of Forensic or Incident Response groups knowing about breaches that have taken place within an organization. This has and will continue to be a roadblock for the industry as a whole since disclosing for the sake of researching and even building a threat model but not disclosing to affected customers is a bit underhanded, although the reasons for each are vastly different. On paper, however, the perception would require some talented finessing. Until then, companies engaged in threat modeling will have to make do with what they have in terms of internal threat data.

Focusing on two key data points, *Reported Incidents* and *Central Log Data* will provide a good basis of internal threat data to later develop into threat intelligence. This threat intelligence, spawn from internal data sources, will then help to substantiate the viability of threats taking place against the application. Externally harvested threat intelligence will be gathered and correlated from outside sources as well, but for this activity, internal log data sources should be the key focus. Some of the best threat intelligence can be formulated by the internal threat data that is collected by an organization. Such data can originate from internal log information, such as central log repositories or an Incident Response database. Products such as Security Incident & Event Monitoring (SIEM) solutions facilitate this greatly. The possibility to channel all internal alerts, incidents, and log data into one vehicle streamlines the process for this activity. Diverse threat data sources, such as platform/system logs, as well as any network-related logs, will provide the type of threat data from which internally harvested threat intelligence can be developed.

In this activity, some degree of Security Operations and/or IT Operations process maturity must preexist. For example, if you do not have a Security Operations Center (SOC) or do not enable logging on your systems or centralize logs to a log aggregation point, then this may be a point in the chapter that you bookmark or simply

contemplate its theoretical capabilities in your own company's environment. Additionally, if your company does not keep track of security incidents of varying types, or worse, even know what a security is, it may be time to bookmark this activity for later use. This activity aims to build off of the first activity by refining what types of passive and active recon or attack efforts have taken place over the last twelve (12) months. Much of that effort will depend on good threat data managed by groups such as Security Operations, Incident Response, Network Operations, and other IT areas. Twelve months is usually a good test bed of threat intelligence to look at since it would account for "seasonal" traffic and provides an ample scope of coverage for analysis. It will be a matter of preferences and largely capability and resources if that time span is increased. It should be pointed out that threat modeling members would not be asking for twelve months of logs to be compressed and zipped up for review. That would negate the ability for PASTA to be a streamlined process to accompany an SDLC workflow. Instead, the assumption is that an SIEM or Log Management solution could provide a huge catalyst to review the level of data needed for a lengthy period of time.

The key to any threat data harvesting is centralization. Unfortunately, most organizations, even large enterprises, have trouble fulfilling the control of centralized logging or incident data management. For most companies in the SMB market (and a large bit of mid-cap companies), the maximum scope of their centralization revolves around only network logs. From there, little to no monitoring is typically done. Larger companies have evolved beyond localized logging and have been leveraging the power of centralized logging solutions (Splunk, Sumo Logic, Nagios, etc.) over the past 10 years, thereby making the ability to harvest threat data a reality. More mature processes have sought SIEM solutions from places such as IBM, HP, and beyond in order to aggregate and normalize the terabytes worth of log data from various sources (network, system, and on rare instances, application servers). Although many places are still slow to make the time and financial investment in SIEMs, this has changed greatly in the past five years or so. Slower adoption has been experienced around storing and centralizing application logs; however, regulatory pressures may affect this adoption over the next few years.

SOCs and Network Operation Centers (NOCs) typically collect security/network-related alert data across multiple enterprise assets. Leveraging the data that pertains to assets within the product application technology scope should be made as part of this activity. Alerts against similar product applications within the enterprise or overall IT environment would also be useful to the threat modeling team. Information being sought as part of this activity includes things such as failed login attempts, failed application calls, authentication errors, arbitrary input received, invalid object references, and so on. Moreover, log records that revolve around identity violations, particularly around identity management solutions or services such as Active Directory or LDAP where authentication integration may leverage are worthwhile to review. In general, patterns that reflect anomalies in application usage are good pieces to review as threat intelligence.

Ideally, SIEMs would be the best route for harvesting threat intelligence from internal sources given SIEM's ability to correlate various types of logs beyond just

network logs. The SIEM's ability to ingest, normalize, and search for key terms, at a relatively quick rate can allow for a threat modeling team to perform targeted searches around threats that are perceived to be viable against the application. Again, if the application being threat modeled is new, alerts and events of comparable applications could be used as well. This exercise will strengthen initial threat perceptions around the application, thereby substantiating possible threats previously considered for the application.

If logs are not currently centralized and manifested via a central logging solution or SIEM, a basic step for gathering threat intelligence is to review access violations across platform, database, application, and network logs. This should be done for the scope of assets across the application architecture. Typically this may include logs from infrastructure assets, DNS servers, web servers (if applicable), data servers (file systems or relational databases), as well as application server log. If workstation logs are relevant to the application solution, they should also be included in the scope of the review.

Whether audit reports are being collected or not, it is important to identify the following characteristics that should be collected as part of audit logs. Any audit logs that are material to application components (stage II) in the threat model are the key focus. Key activities to identify across log, audit, and/or incident reports are the following:

- Access Control Logs (Identity Management solution (IdM), Network Access Control (NAC), LDAP/ AD Servers, Application Access Logs, System/ Service Logs (RDP, SSH, Platform (*nix, Windows, RADIUS, TACACS+, RACF, TopSecret (Mainframe), etc.)
  - Look for violations in access. This may translate Authorized access during odd hours may also indicate compromised internal actors. Their credentials may have been used to access or conduct unauthorized actions against some facet of the application environment.
    - Abnormal access times by any user
    - Failed login attempts
    - Account lockouts
    - Sudden activity on inactive accounts
    - Sudden activity on active accounts
    - Business logic violations for a given user role (using a report that does not correspond to role)
- Host-based Firewall alert logs
  - Illicit file/ system access requests
  - Access to blocked sites (Ex: known blacklists, etc.)
  - Abnormal disk/ CPU usage levels.
  - Alerts from application whitelisting end point solutions.
- Application Logs
  - Business logic exploitation attempts

- Injection-based attempts (CSRF, XSS, SQLi, etc.)
- Fuzzing attempts
- HTTP Error Codes
- Database level error messages
- Any level of application logs relate to the product application's use cases.
- Privilege escalation attempt

- Infrastructure Logs
  - Firewalls, router logs (egress traffic),
  - Web proxy/ Load balancer logs (if applicable)
  - Netflow traffic for interfaces associated with application environment. Intrusion Detection Systems/ Software (agent based)
  - Observed ARP spoofing attempts
  - IP conflicts
  - Spike in use of a particular network protocol
  - Firewall alerts that show spoofed IP sources or loop-back address
  - IPS/IDS alerts that show MEDIUM to HIGH or even very frequent LOW tripped alerts
  - Unified Threat Management (UTM) or Network Threat Management (NTM) logs.
- Server side logs
  - File system file access violations, SSH logs, Windows Event Logs, sudo logs, and so on.
  - Agent-based logs (Ex: from host-based security on end points – Firewalls, Host Intrusion Prevention)
- Host-based A/V or HIPS agent logs or Application Whitelisting Agent Logs
  - Administrator machine terminals with elevated access to product application features requested or tripped
- Proxy server logs
  - Logs from both network and application proxies.
  - Illicit URI requests or manipulated web requests
  - Excess or spike in HTTP 404, 302, 500 alerts
- Database logs
  - Ex: Business logic logs show anomalies in batch job requests (outside expected timeframes)
- Web server transaction logs
  - Ex: Directory enumeration attempts for folders/ file objects that do not exist.
- Human Resource Reports
  - Personnel write-ups provide non-traditional intelligence on troubled personnel that may pose as threat agents in the event of an insider threat.

- Facilities Management Reports
  - Violations to access from physical areas within a company provide additional threat intelligence as to what internal threats may be present by internal employees or rogue human actors who may have obtained illicit credentials to protected areas. Reviewing timestamps and reconciling user roles to access times may paint a picture of a developing threat. Logs from proxy cards, biometric readers, and other physical access control points that enable logging can be centralized to logging aggregators for easier access and analysis.

The aforementioned list provides easy ways to narrow down viable threat data criteria. This would help to narrow the focus on what to search for in these massive incident event log repositories. For example, related to access control logs, if an application has expected or predictable usage times for certain roles, then reviewing alerts outside of these time windows may represent anomalies and possible indications of illicit access to application data. Outliers or anomalies in normal application use cases are good indicators of possible threat indicators. This is simply one for which there are many per each type of log that can be consumed.

One of the benefits from harvesting log data is that it builds for good threat context. Such information aids in determining if presumed threats have any validity. Threat intelligence may even suggest that active or recent attack patterns are actively present and even have some level of persistence. If the subject application is a new application, but leveraging preexisting infrastructure or assets, then log data around those other application components should be leveraged as part of the threat analysis. Additionally, log data from comparable applications components, having similar technology, use cases, and deployment models, may also be of use.

In terms of roles, working with enterprise system and network administrators will prove to be key allies in fulfilling internal threat intelligence gathering. Being able to gain the help of administrators in network and system operations will determine the success of this activity. In most large enterprises, as long as approved access is granted, many of these SIEM products have a reporting web interface that they can simply log in and run their queries or even set up their batched report requests. Filters are provided in many of the SIEM products today in order to filter queries to down to header level information, IP, action type, TCP protocol, timestamp, and more. For non-web-related applications, establishing criteria for protocol use, authentication types, IP-based header values will be important versus those HTTP headers used in web-related applications.

The next activity in stage IV now builds on our initial threat analysis and review of internal threat data to collaborate with external threat sources. The continued objective is to conclude with a substantiated list of threats for which attack trees will be built in the stages that follow.

Stage IV: Activity III (S4:A3) – Gather Threat Intelligence from External Sources

This stage is about threat intelligence gathering versus amassing threat data. The former is driven by identifying who attackers are, what they are planning and what they are after. The latter is about data used to evaluate whether or not something could be

a threat (e.g. – elevated counts to protocol use on a switch or inside network device). We have focused on threat data earlier in this Stage and now we look outwardly for guidance on current threats that could affect our application environment being threat modeled. This activity aims to refine the list of possible threats against the subject application that is being threat modeled.

External threat intelligence gathering has a wider breadth of data to consume and is generally done so by third-party managed security service providers (MSSP). Very few companies have internal resources that harvest external threat data. Even fewer companies have internal resources able to correlate threat data to their own industry trends, company technology footprint, let alone application components. As a result, much of the external threat intelligence data is consumed by SMBs and enterprises alike in the form of subscribed threat feeds. Various types of services offer slightly modified version of this service. Some MSSPs offer a hybrid service where internal alert data from client networks would be mashed with external threat data consisting of reported security incidents, security bulletins, and other security advisories. Other MSSPs simply offer external threat feeds. Many additional service options are also present as well as much of these types of services imply some degree of customization. The relevancy to overall threat analysis is that a broader range of threat data can be analyzed and factored into the threat model.

Regardless if an MSSP is used or if an internal SOC is correlating breach data, reported security incidents, and security advisories, the following criteria can serve as valuable points of guidance for qualifying good threat intelligence to bake-in.

1. Correlate threat intelligence to the perceived threats identified in Activity S4:S1. This external threat intelligence will legitimize the threat perceptions that were internally identified under the first stage.
2. Focus on threats that will adversely affect the use of defined use cases from stage I.
3. Look for threats that are ideally related to your industry segment. Secondly would be that they are related to the employed technology or service model that the application is following (Cloud-SaaS, client server, etc.).
4. Identify threats that, if realized, would present the biggest impact to the application's use, adoption, promotion, or even the company's reputation.
5. Focus on threats that pass a basic supposition of viability. If it sounds too far-fetched, you can rest assured that adversaries or hacker groups are also trying to fathom an easier way to conduct their layers of attack.
6. Don't lose site of the big threat picture: data loss, infrastructure compromise, IP Theft, and so on. Keeping site of this will provide the right context to evaluate the threats that are researched in this phase.
7. Do not forget about the threat agent or malicious actor. The threats may seem viable, but it is important to consider who or what would perform the layers of attack that represent a given threat.
8. Make sure that the threat is still relevant. Threat information does have a shelf life since as attackers launch attacks and defenders understand these attacks,

attackers will have to find new ways to attack or different places to attack who may not be informed of latest attack methods.

9. Understand the attack vector relative to the threat identified by the threat intelligence. Examples of attack vectors can be (LAN based, NFC Protocol, "sneaker net," browser based are some examples. Considering the channel of attack will further substantiate the possibility of a threat that is supported by a mature attack vector.
10. Abuse/Misuse Case – Although this is an activity that is really fledged out in Stage VI of the PASTA methodology, it is worth considering what possible abuse cases can be derived from threat patterns. If there are very little to none, then it would be best to focus efforts on more credible threat patterns.

Now that we have come guiding criteria on selecting possible threat sources, here are some actual threat intelligence sources to consider.

- [PLATFORM] Microsoft Security Intelligence Report (http://www.microsoft.com/security/sir/default.aspx)
- [HEALTHCARE] HITRUST Cyber Threat Intelligence and Incident Coordination Center (http://www.hitrustalliance.net/ctas/) – Health care-focused threat intelligence reporting.
  - Health-care Threat Briefing – summary of top threats in the health-care industry.
  - Health-care Industry Threat Report – overall synopsis on threats to health-care industry.
  - Health-care Incident and Malware Reports – summary of malware reports affecting today's health-care entities; provides indication of targeted malware attacks aimed at either affecting patient lives or compromising patient information.
- [GENERAL] NTT Group has unveiled a new threat intelligence report in 2013 that looks at global threats, cross industries. Regional threat outlooks can be obtained here: http://www.nttgroupsecurity.com/.
- [GENERAL] SANS Internet Storm Center (ISC) – Useful information on regarding spikes in threat data around the world, mostly network focused. (https://isc.sans.edu/reports.html)
- [GENERAL] US-CERT (http://www.us-cert.gov/security-publications) – Summary of threat intelligence data that is industry/platform agnostic.
- [GENERAL] Security Partner Threat Data (McAfee, Symantec, Dell-SecureWorks, Cyveillance, etc.) – Ongoing briefings of mostly malware or Internet-based threats that have been recorded/logged by some of the world's leading security software firms and their network of security software agents/appliances.

Threat intelligence, generally provided by security and technology firms with a research department, is able to provide either a free or pay for service on active threats.

MSSPs with a research department may be able to provide companies with a periodic feed of active threats as observed across client environments or studied honeypots where some of their vendor technology may be deployed. These security points of presence often have the ability to integrate threat data to a central repository where macro-level analytics can be performed. The dominant focus of threat intelligence today revolves around web infrastructures and network-based appliances/equipment, however, given the rise of use and popularity of mobile platforms in both personal and corporate settings, threat feeds around mobile and other platforms will undoubtedly increase, therefore, those types of feeds should be aggregated whenever possible and if applicable to a company's application development roadmap for products.

Apart from the previously mentioned security companies, the Verizon Business group has recently created the Verizon Enterprise Risk Incident Sharing (VERIS) Framework as well as an accompanying web application that allows for anonymous sharing of security incidents. The result is an online warehouse for identifying incidents that relate to those experienced by a given company or set of companies. The VERIS framework embellishes four key measurable areas that encompass multiple characteristics related to an incident. They are shown as follows in Table 7.13, under the column titled "Metric Area."

Over time, more and more organizations will look to share incident-related information. This sharing has already begun to become aggregated by places like the Verizon VERIS network and the HITRUST Cyber Threat Intelligence and Incident Coordination efforts. Today, MITRE has established TAXII[5] (Trusted Automated eXchange of Indicator Information), which defines a suite of services aimed at exchanging cyber threat information across organizations, industries, and product groups. The service is built around the STIX[6] (Structured Threat Information eXpression) language, which aims to standardize the protocol used when sharing incident or threat information. This structure language is similar to the CCE and CVE languages, which aim to standardize common configuration and common vulnerability lists. Over time, it is expected that the amount of threat intelligence from across multiple industries and organizations will lend to a highly useful list of threats to consider when threat modeling.

Threat intelligence is best leveraged when the information from the threat feed is properly correlated to the application environment that is being studied by the application threat model, in addition to the threats defined to be most relevant from S4:A1. Threat intelligence that conforms to the TAXII structure will also assist in the consumption and sharing of threat intelligence to other sources when needed. Using information from threat feeds would normalize attack, vulnerability, weakness, and other application component data that may be relevant to the threat. These threat feeds serve as counter-intelligence to attacks in cyberspace and provide an opportunity for multiple parallels to be drawn during the threat analysis phase (stage IV) of the proposed threat modeling methodology.

[5]http://taxii.mitre.org/.
[6]http://stix.mitre.org/.

TABLE 7.13 VERIS Framework of IR Metrics

Metric Area	Characteristics	Purpose/Additional Metrics
Demographics	Date of incident	Facilitates trending over time
	Primary industry	Facilitates industry-specific analysis and comparisons
	Region of operation	Facilitates regional analysis and comparisons
	Number of employees	Facilitates analysis and comparisons based on organizational size
	Number of dedicated IT security staff	Provides an indicator of the size and budget of the information security program
Incident classification	Agent	Defines internal attack agent, internal, or partner
	Action	Defines whether action is malware, hacking, social, misuse, physical, error, and environmental
	Asset	Asset type, # of assets affected, additional asset details, business function, mgt/administration decisions over time on asset, location/hosting details of asset
	Attribute	Confidentiality, Integrity, Availability, Authenticity, Utility, Possession of Control of Data
Discovery and mitigation	Incident timeline	Initial action to Incident, Incident to Discovery, Discovery to Containment/Restoration
	Incident Discovery	Evidence Sources, Discovery Method, Other Discovery metrics
	Capabilities and remediation	Program Maturity, Control Assessment, 20/20 Hindsight Solution, Cost of 20/20 Hindsight Solution, Corrective action and recommendations
Impact classification	Impact categorization	Direct Impact Categories, Indirect Impact Categories
	Impact estimation	To obtain an estimation of impact in currency (dollars)
	Impact qualification	To understand the severity of the impact relative to the organization's tolerance for loss

Leveraging information from prior incidents is invaluable to many companies wishing to build further context around attacks that they may have witnessed. It also provides a frame of reference to understanding attacks against varying types of application environments. The challenge in analyzing incidents for the benefit of improved threat modeling is the diversity of incident types reported, which makes it prone to

have a lot of fractured groups. If reported differences contain more unique properties than similar ones, it will affect the integrity of the threat intelligence that would be formed from such reported incidents.

This forces some degree of synchronization among threat intelligence reports and incident reports. Although the intent by the industry is to have reported incidents provide threat intelligence, the secretive nature of breaches and companies reluctance to anonymously disclose has made the process slow. Threat intelligence service providers are therefore having to introduce other forms of "intelligence" into the threat equation, which is risky and somewhat deceitful practice by many standards. In the future, it is hoped that an increased collaboration of anonymous incident reporting can provide for improved threat intelligence. This would greatly elevate the quality of threat intelligence from which this activity in PASTA would directly benefit from.

Nonetheless, the threat intelligence provided by companies such as Symantec, Intel Security (formerly McAfee), Dell-SecureWorks, or Juniper Networks have their benefits and use during stage IV of the application threat modeling methodology. Wherever the source, it should be mentioned that sources for threat feeds become less reliable over time and in general should be qualified every year to two years in order to make sure that the data sourcing is up to date and comprehensive.

Stage IV:Activity IV (S4:A4) – Update the Threat Libraries One of the most comprehensive attack libraries is managed by the MITRE organization. The CAPEC (Common Attack Pattern Enumeration & Classification) is an attack library based on prior attack patterns. Attack libraries are helpful to apply to applications and determine if any of the attack patterns are viable given various characteristics (known vulnerabilities, weaknesses, and threats). Pen testing tools have recently begun to incorporate attack libraries into their tool base. Knowing that attack models and attack probes will be presented in Stage VI of the PASTA methodology, it is important to achieve two important tasks for this present activity. These two actions are as follows:

1. Update the threat library from source. Beyond MITRE Corporation's (www.mitre.org) CAPEC library, you may check other sources of application threats/attacks that could provide for the inception of an attack tree. Both OWASP and WASC (Web Application Security Consortium) have top threat listings to web applications, as well as for mobile applications (mobile). These listings can all help to provide a database of threats that can be applied to the threat model. Most pen testing tools and DAST (Dynamic Application Security Testing) already have regularly scheduled updates that incorporate these libraries into the product itself. Some tools that currently update using the CAPEC libraries include the following: MetaSploit Pro (Rapid7), Cenzic Hailstorm (Trustwave), and HP WebInspect. Managing updates to attack libraries is possible outside of these tools; however, it is nice to have them automatically updating during this activity.
2. Consider Threat to Attack Relationship. Building off of the attack library updates and review, an attack tree at this stage will begin to form and capture the threat to attack relationship. Threats cannot be realized without a successful

attack; therefore, a preliminary analysis into the types of attacks that could achieve the identified threats from this stage should be performed.

Stage IV:Activity V (S4:A5) – Threat Agents to Asset Mapping A threat agent is a single individual or group intending to launch a threat into action. Today, nearly all threat agents will be human, but in the future, threats will be orchestrated autonomously by actors in applications. Encompassing the threat agent into the threat model is important because mitigation may apply to preventing or limiting the threat agent's actions via a range of countermeasures. Upon mapping the threat agents to possible targets, preventative, detective, or reactive countermeasures can be considered.

In this activity, we map threat agents to the beginnings of an attack tree. As shown in the following attack tree model, a threat to an embedded medical device, with a subset of use cases is depicted. Threats to each use case are depicted along with a possible threat agent. In this example, there are three possible threats and associated threat agents. Left to right, we have an internal threat agent who may have legitimate or illicit access to affect the monitoring use cases for the application. This may be an insider who may have colluded with an external threat agent in order to affect the target application, and ultimately, the target patient.

The next threat agent is intended to represent a criminal agent who has technical expertise in the software and can introduce an attack payload against the software product. This may have life or death consequences for the patient. For prime global figures or key government officials, this may be a viable threat scenario. The criminal agent represents a hacker for hire who has researched exploit scenarios that succeed against the use case of regulating heartbeats from a Bluetooth-enabled pacemaker.

The last threat agent depicts one that works within a trusted vendor organization. The vendor may produce the target asset and/or software related to the Bluetooth enabled. In this case, the threat agent may have been contracted to alter the data reporting functionality of the use case in order to allow for an unauthorized data request to be sent to the application components executing "Report Medical Data" use cases.

A mapping of all three threat agents is represented in the following partial tree (see Figure 7.26).

Stage IV:Activity VI (S4:A6) – Assign Probabilistic Values around Identified Threats This activity helps to assign a weighted percentage to each identified threat in S4:A1. The probabilistic analysis is based on considerations for access, window of opportunity, ability to repudiate, risk reward (for threat agent), and threat simplicity. Although extremely simplistic, these five pieces of information truly represent whether or not some threats become actionable against a target application or application component. The rationale behind some of these characteristics is also logical since these five threat characteristics would need to be present for a threat to be formed. Evaluating these five attributes around threat should be based on the activities fulfilled up until this point in stage IV, which included industry statistics, threat reports, or security incident reports.

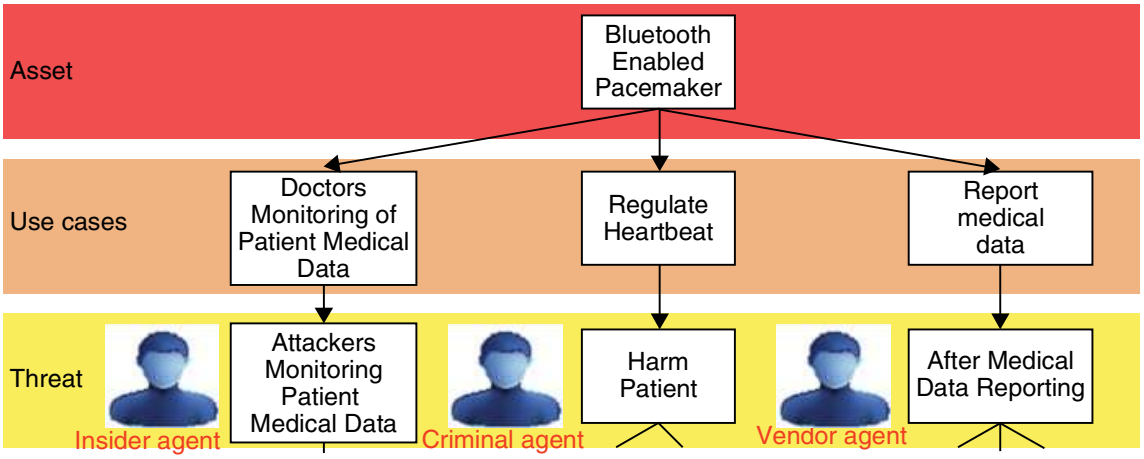


Figure 7.26 Mapping of Threat Agents to Asset Targets

Each of the five criterion is divided into three possible percentage outcomes. They are as follows:

90% – Reflects nearly all aspects of the threat criteria are possible.
50% – Reflects a moderate to large aspect of the threat criteria is possible.
10% – Reflects a limited amount of the threat criteria is possible.

- *Accessibility:* Threat landscape related to target(s) is easily accessible.
  - *90%:* Completely open to all human and/or logical application, network traffic
  - *50%:* Restrictions are in place for access against logical or human interaction.
  - *10%:* Assumed to be impermeable. Recognizing that noting
- *Window of Opportunity:* Nothing impedes threat agents from exercising threat plans and underlying attack patterns.
  - *90%:* Opportunity is always there.
  - *50%:* Opportunity is only present during certain points of time.
  - *10%:* Limited time window for threat to be exercised.
- *Ability to Repudiate:* Is there an opportunity to repudiate threat actions against a target or threat landscape that includes aspects of the application environment.
  - *90%:* Repudiation is nearly guaranteed.
  - *50%:* Opportunity repudiate is only partially present or threat components are only partially able to be repudiated.
  - *10%:* Ability to repudiate is extremely difficult.
- *Risk Reward:* Value of target over risk level to threat agent.
  - *90%:* Value received from successful execution of threat is nearly double level of risk.
  - *50%:* Reward potential is at least 50% of the value of the risk.
  - *10%:* Very little reward percentage in relation to risk level.

- *Threat Simplicity:* Simplicity level of threat. Easiest threats will have reduced layers of attack patterns as well as number of threat agents.
  - *90%:* Nearly all of threat plan is simple to execute.
  - *50%:* Large part of threat plan is simple to execute.
  - *10%:* Very little of the plan is simple to execute.

The numeric probability assigned to identified threats (from S4:A1) and threat-related criteria (mentioned earlier) is not intended to be an advanced form of probabilistic analysis. The noncomplexity is actually a good trait to maintain since much of what people find discouraging about threat modeling is that it is large and complex. Adding complexity to threat modeling is not the best of ideas, especially when trying to foster adoption. Nonetheless this probabilistic activity is intended to create a value percentage around Threat. As a result, thus far, we have a slightly altered *Risk* formulate that resembles the following:

$$(R)\text{isk} = \frac{(T)\text{hreat}_{(P)\text{robability}} \times (V)\text{ulnerability}_{(P)\text{robability}} \times (I)\text{mpact}}{(C)\text{ountermeaures}}$$

*P*robability in the aforementioned formula is obtained by taking the average percentages of all threat criteria values. In doing so, it provides a probabilistic percentage average that can be used to estimate the likely of a threat to take place, in the absence of knowing what vulnerabilities exist or what countermeasures to mitigate the risks could be developed. Later, we will also begin to explore the other probability coefficient around vulnerabilities.

Exemplifying the use of this form of quantifying threat probability, we look to a quick example in the higher education industry in Table 7.14.

Reverting back to the health-care threat scenario from the prior activity, we can revisit the attack tree to consider probabilities around one of the threat agents. Taking the second threat agent scenario where a hired hacker may be contracted to develop a custom payload for unleashing against a head of nation, we review the following threat criterion for probability (Table 7.15).

The predefined percentage benchmarks may be altered if desired as well. A 58% probability that a targeted threat to a national leader with a pacemaker may be a bit high. Both of the areas around accessibility and the simplicity of the threat could in fact be much lower than 10% considering that state leaders may have considered this risk before and thereby implemented some NFC emission containment safeguards. Additionally, gaining physical access to a national leader may prove difficulty, given their security along with unpublished itineraries. Nonetheless, this exercise provides a good basis for which to consider key characteristics of how probabilistic analysis can be assigned to threats.

Key Roles in Stage IV

Another key member or group to consider from Figure 7.26 is the incident responder or security analyst charged with event or incident monitoring. If incidents are

TABLE 7.14 Threat Analysis of a Mobile Based Loan Application Serving Higher Ed

Application Name:	iLoan4Learning v1.1
Short Description:	Mobile and web-based student loan application for undergraduate students at a major college/university.
Data:	Student PII, approximately 1.3M records.

	Threat Criterion	
Threat Criteria	*Probability (%)*	*Reason*
Accessibility:	90	Threat agent's access to app.
Window of opportunity:	50	Opportunity is present often.
Repudiation:	50	Repudiation can be controlled to some degree by the network and application logs.
Risk reward:	50	PII volume equates to large potential payout in black market.
Simplicity:	10	There are many needed layers needing to overcome in order to successfully execute this threat.
Average:	50	Probability of threat to take place.

TABLE 7.15 Threat Analysis for Bluetooth Enabled Medical Device

Application Name:	HelloHeart 1.3
Short Description:	Bluetooth-enabled device monitors and controls heartbeat regularity.
Data:	Patient Health Information (PHI), patient safety – loss of life

	Threat Criterion	
Threat Criteria	*Probability (%)*	*Reason*
Accessibility:	90	Open NFC protocol; similar device can be purchased for testing.
Window of opportunity:	10	Physical proximity needed to release payload.
Repudiation:	90	Repudiation is almost guaranteed since application logging is known to not take place.
Risk Reward:	90	Bounty compensation is extremely high.
Simplicity:	10	Exploit payload would require some R&D time. Physical countermeasures may have already taken place.
Average:	58	Probability of threat to take place.

tracked and managed over time and centralized, threat analysis can leverage historical passive or active attack patterns to support the threat analysis. Incident response analysts, network operation engineers, or security operation center analysts are ideal to work with in Stages 4 of the proposed threat modeling methodology. Depending upon how events, incidents are tracked and managed (by source, by type, by frequency, etc.), some degree of trending analysis can be done and relayed to the threat modeler in order to consider how historical reconnaissance, injection attempts, brute force attacks, phishing campaigns can indicate historical attack trends from which some degree of regression analysis can be performed. If such data is not available for the target application environment, it may be possible to leverage statistics or metrics around similar incidents or events on comparable application environments within the company. Such information provides some degree of contextual basis for understanding motives, target assets, and attack vectors for the subject application environment addressed by the threat model. This provides the improved ability to profile attack patterns based on *actual* events that have taken place against similar environments. It is important to note that this effort is not aimed to *conclusively* profile attackers and their targets based on these events alone, but it obviously lends a great amount of credible and relevant information to the threat model for an application as the motives for historical attacks will be similar if not the same to those of prior attacks, assuming no major change in functionality and/or data use.

Stage IV – Summary on Threat Intelligence and Analysis

Overall, threat intelligence will be a stage filled with research aimed at sustaining a threat model. The threat model will ultimately reflect a tree-like structure that has branches of assets, use cases, abuse cases, vulnerabilities, and attack patterns. These attack trees take full form in Stage VI. The key to this stage, however, is analyzing and collecting good intelligence data from both internal and external sources. The following is an excerpt that reflects the type of content that should be reviewed.

A summary of the activities for this stage is revealed in Figure 7.27.

Stage V – Weakness & Vulnerability Analysis (WVA)

Stage V of PASTA is about identifying vulnerable or weak areas across the application. Our knowledge of inherent risks associated with the application, a refined technology scope, set of use cases, relevant threats will now be the background to discover what vulnerabilities or weaknesses may be present across the application.

The key objective is mapping this information back to the attack tree that was introduced in the threat analysis phase under stage IV. As most IT and Security professionals know, outputs from vulnerability assessment programs are riddled with false positives. Additionally, for the purposes of a threat model, the scope of vulnerabilities may extend outside the scope of the application on which an application threat model is focusing. For this reason, vulnerability trimming will be applied in order to port over relevant and confirmed vulnerabilities to the attack tree. This is the primary

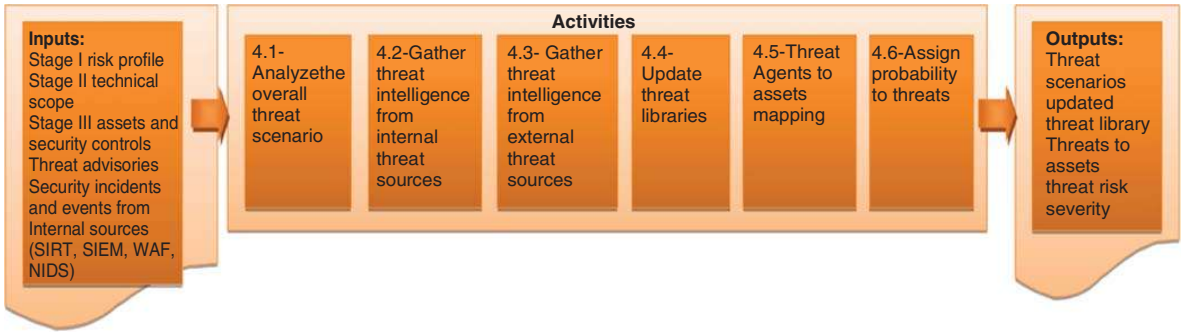


Figure 7.27 PASTA Risk-Centric Threat Modeling – Stage – IV (TA) Threat Analysis

objective. In support of this stage's primary objective, the following tasks are also important to fulfill.

1. Review/correlate existing vulnerability data.
2. Identify Weak Design Patterns in the Architecture.
3. Map threats to vulnerabilities.
4. Provide Contextual Risk Analysis based on Threat-Vulnerability.
5. Conduct Targeted Vulnerability Testing.

Stage V:Activity I (S5:A1) Review/Correlate Existing Vulnerability Data Establishing a historical context for what has been vulnerable or weak across application components is a key place to start Stage V. Doing so provides a glimpse of what areas have historically been more susceptible to exploitation. Using this information, we can align existing vulnerabilities or flaws in network/application design to the threat intelligence and data that has been collected thus far. Additionally, application use cases and objectives can be considered in light of the known vulnerabilities and weak design attributes. Overall, the collective knowledge of prior vulnerabilities and weak or nonexistent control implementations translates to a different level of concern given the background knowledge obtained from the PASTA stages thus far. Knowledge of business objectives, technical scope, and interoperability of application components, threat intelligence, and now, confirmed vulnerabilities and/or weaknesses, provides for a developing picture around contextual risk.

In terms of the age or *freshness* of vulnerability information leveraged as part of this activity, it is important not to consume information that is not too old. This would otherwise minimize the effectiveness of the threat model. The time stamp of old vulnerability data should not exceed a twelve month period, unless the application being threat modeled is a legacy application or one where upgrades or applied patches are scarce due to software incompatibilities. Aside from those exceptions, reviewing vulnerabilities beyond twelve months may be futile since patches, updates, or upgrades may have negated the initial findings.

Key elements that should be extracted from prior vulnerability data should include the following:

- Assets employed by the application (DNS, Proxies, Bastion hosts, etc.)
- Actors (Human, System, or Application-based actors)
- Client software (Browser, Mobile, Traditional Fat Client)
- Server software (IIS, Active Directory, ISA, Exchange)
- Running services (privileged, ephemeral ports)
- Third-party software (Citrix, MS Hypervisor, Oracle IdM)
- Application frameworks (Struts, .NET MVC, Spring MVC)
- Architecture
- Data sinks/sources (e.g. file servers, databases, etc.).

Using the aforementioned as criteria for containers for vulnerability data, scan results, audits, and architectural reviews can be mapped to each of these containers that may be present within the application. Since we know the footprint of application components (thanks to stage II), associating these components can be easily achieved by hostname, IP, namespace, named pipe, DNS name, method call, and so on. An example of this association is provided in the following section (Table 7.16).

The aforementioned artifact shown is simply a representation of how historical vulnerability information can be aggregated into a consolidated artifact. Consolidation of this information can take place in any format; from a spreadsheet to an internal wiki or ticketing/but tracking system. If more enterprise tracking systems are used, these application defects should be labeled appropriately in order to designate them as being part of a threat modeling effort. In doing so, they can be easily retrieved and re-classified once additional design and software flaws are found within Stage V of PASTA. Inherent vulnerabilities across all application components of the target application is simple the starting ground for this stage. The next activity will build on top of these recent preexisting vulnerabilities and look at the current architecture in order to see what design flaws may be present. This is covered in the next activity for this stage.

Stage V: Activity II (S5:A2) Identify Weak Design Patterns in the Architecture An often neglected component to application security is its architecture. In this activity, we will revisit the DFDs produced under stage III (Application Decomposition). As part of this activity, a review of key architectural concerns around data security will be examined in order to ensure that security is applied for data at rest, in transit, and while being processed. These considerations use cases depicted under stage III and identify if the request/responses among application components are not part of a well-designed security architecture.

Secure software design activities center a number of different security themes. These themes need to be addressed on the trust boundaries that have been identified in stage III. A simple list of insecure architecture gaps to identify in this activity is presented as follows:

- Unencrypted authentication channels.
- Noncentralized logging activities.
- Noncentralized or distributed authentication models.
- Unprotected or exposed administrative interfaces.
- Nonsegregated network segments among trust boundaries.
- Liberal application/network ACLs (extended permit ip any, extended permit ip 10.10.10.0 255.255.255.0 any, X Domain Policy wildcard use “*”).
- Placement of sensitive network services, such as credential stores, in public/semipublic network areas.
- Use of application services that do not require authentication.
- Nonfederated user models where a single elevated credential serves as the actor to many worker processes across an application environment.

TABLE 7.16 Threat Analysis Artifact against a Single Asset/ Use Case

Targeted Asset	Affected Service	Identified Vulnerability	Description	Affected Actor	Affected Use Case	Impact	Countermeasure
10.51.25.35	SSHD (port 22)/Secure Shell Server	Default SSH password: root with blank password (ssh-default-account-root -no-password)	The root account uses a blank password. This would allow anyone to log into the machine via SSH and take complete control.	SSH User (imager01)	Secure remote copies to physical imaging servers over network.	High	Use the "passwd" command to set a more secure login password. A good password should consist of a mix of lower- and upper-case characters, numbers, and punctuation and should be at least 8 characters long.

(*continued*)

TABLE 7.16 (*Continued*)

Targeted Asset	Affected Service	Identified Vulnerability	Description	Affected Actor	Affected Use Case	Impact	Countermeasure
imaging1.hc.org	NTPD (port 123)/Network Time Daemon	NTP 'ntpd' Autokey Stack Buffer Overflow Vulnerability (ntpd-crypto-recv-buffer-overflow)	There exists a stack-based buffer overflow in the crypto_recv function found in ntpd before 4.2.4p7 and 4.2.5 before 4.2.5p74. When OpenSSL and autokey are enabled, the flaw allows remote attackers to execute arbitrary code via a specially crafted packet containing an extension field. Additionally, the NTPD was found to be running as the "root" user.	root	Accurate time stamping of medical report data.	High	Update ntpd 4.2.4x to ntpd 4.2.4p7 Download and apply the upgrade from: http://www.eecis.udel.edu/~ntp/ntp_spool/ntp4/ntp-4.2/ntp-4.2.4p7.tar.gz Also associate another actor to run daemon.

Ddc1. careentity.org	DNS Service (port 53)/Doman Name Service server	DNS server allows cache snooping (dns-allows -cache-snooping)	This DNS server is susceptible to DNS cache snooping, whereby an attacker can make nonrecursive queries to a DNS server, looking for records potentially already resolved by this DNS server for other clients. Depending on the response, an attacker can use this info to potentially launch other attacks.	dns-svc	Ensuring the security of PHI and PHI-related systems at Health-care organization.	High	1. Leave recursion enabled if the DNS Server resides on a corporate network that cannot be reached by untrusted clients OR 2. Do not allow public access to DNS Servers performing recursion OR 3. Disable recursion

(*continued*)

TABLE 7.16 (*Continued*)

Targeted Asset	Affected Service	Identified Vulnerability	Description	Affected Actor	Affected Use Case	Impact	Countermeasure
Db1.health-careentity.com	Multiple Ports: TCP Port 1099 TCP Port 8080 TCP Port 1521	Arbitrary Code Injection (CVE-2013-0422)	Multiple vulnerabilities in Oracle Java 7 before Update 11 allow remote attackers to execute arbitrary code by (1) using the public getMBeanInstantiator method in the JmxMBeanServer class to obtain a reference to a private MBeanInstantiator object, then retrieving arbitrary Class references using the findClass method, and (2) using the Reflection API with recursion in a way that bypasses a security check by the java.lang.invoke.MethodHandles.Lookup.checkSecurityManager method due to the inability of the sun.reflect.Reflection.getCallerClass method to skip frames related to the new reflection API, as exploited in the wild in January 2013, as demonstrated by Blackhole and Nuclear Pack, and a different vulnerability than CVE-2012-4681 and CVE-2012-3174.	oracle	Ensuring the security of PHI and PHI-related systems at Health-care organization.	High	Update Java Oracle Security Alert CVE-2013-0422 states that Java 7 Update 11 (7u11) addresses this (CVE-2013-0422) and a different but equally severe vulnerability (CVE-2012-3174). Java 7 Update 11 sets the default Java security settings to "High" so that users will be prompted before running unsigned or self-signed Java applets.

Room IMG501	MySQL (port 3306)/MySQL Server	Database Open Access (database-open-access)	The database allows any remote system the ability to connect to it. It is recommended to limit direct access to trusted systems because databases may contain sensitive data, and new vulnerabilities and exploits are discovered routinely for them. For this reason, it is a violation of PCI DSS section 1.3.7 to have databases listening on ports accessible from the Internet, even when protected with secure authentication mechanisms.	Root	Ensuring the security of PHI and PHI related systems at Health-care orga-nization.	Medium	Configure the database server to only allow access to trusted systems. In general, segregation based on internal trust boundaries along with network or application-based access control filters will work adequately. Also associate another actor to run MySQL PID.

- Poor proxy implementation.
- Insecure transport of logging data.
- Excess privileges across system platforms.
- Anonymous actor calls to application service.
- Unencrypted communication channels.
- Unencrypted data storage (online).
- Unencrypted data storage (offline).
- No digital certificates used for Internet facing services.
- No digital certificates used for high-impact processing servers.
- Insecure key management storage.
- No identity validation on actor calls.
- Insecure/unreliable IP protocol used.
- Authentication data send in clear text.
- PII or sensitive client information sent in clear text.
- Weak encryption ciphers used.
- Proximity of high target asset to network perimeter.
- Superfluous use of network services among application components.
- Poor segregation of multitenant data sources.
- Poor network segregation among environments.
- Poor segregation of administrative interfaces.
- Insecure transport layer (TCP).
- No input validation across data interfaces.
- No segmented logging channel.
- No considerations for High Availability (HA) in design.
- Only 1 security zone defined.
- Insecure messaging (Layer 7) protocol used.
- Outdated or insecure service version (WEP, SNMPv2, SSHv1, etc.).
- Insufficient network defenses/lack of stateful packet inspection.
- Little to no business logic validation.
- No standard security API used for authentication, encryption.
- No integrated security domain model.
- Insecure challenge response model.
- No integrated authentication model used.
- Superfluous use of two-way trust exchanges.
- Poor implementation of multifactor authentication.
- Not currently leveraging a security framework.
- Storage of connection strings or application credentials.
- Client-side storage of sensitive data (credentials, etc.).
- Insufficient security in relational/flat file databases.

- Insecure password storage and retrieval.
- Lack of security attributes/assertions across programmatic interfaces.
- No integrity checks for data processing/transmittal.

Architects should review their network, application, and data designs to ensure that the aforementioned mentioned flaws are not included across various layers of the application. Design flaws or weaknesses need to address application design and not just network design. DFDs will prove instrumental at this point as they will legitimize the need for greater security control to be implemented at those points, along with the previously developed DFDs, in order to ensure that the aforementioned architectural gaps have not been left unmitigated. The key goal from this activity is that security considerations become applied where trust boundaries have been formed (stage III). Trust boundaries mean that two distinct areas of application components should have some level of restricted access control in order to ensure the security, integrity, and authenticity of a communication exchange.

Revisiting one of the DFDs from before, we see some trust boundaries defined in the following figure. Notice some simple inclusions of requirements that were presumed nonexistent in the DFD. By simply analyzing the interaction between a login procedure and a backend college database server, we may note that insufficient controls exist across this trust boundary. As a result, we can denote what additional architectural gaps are present. In this case, countermeasures listed in green for the data flow among the login process and the authentication engine (of the college database) are architectural gaps that can be added as vulnerabilities for this stage (Figure 7.28).

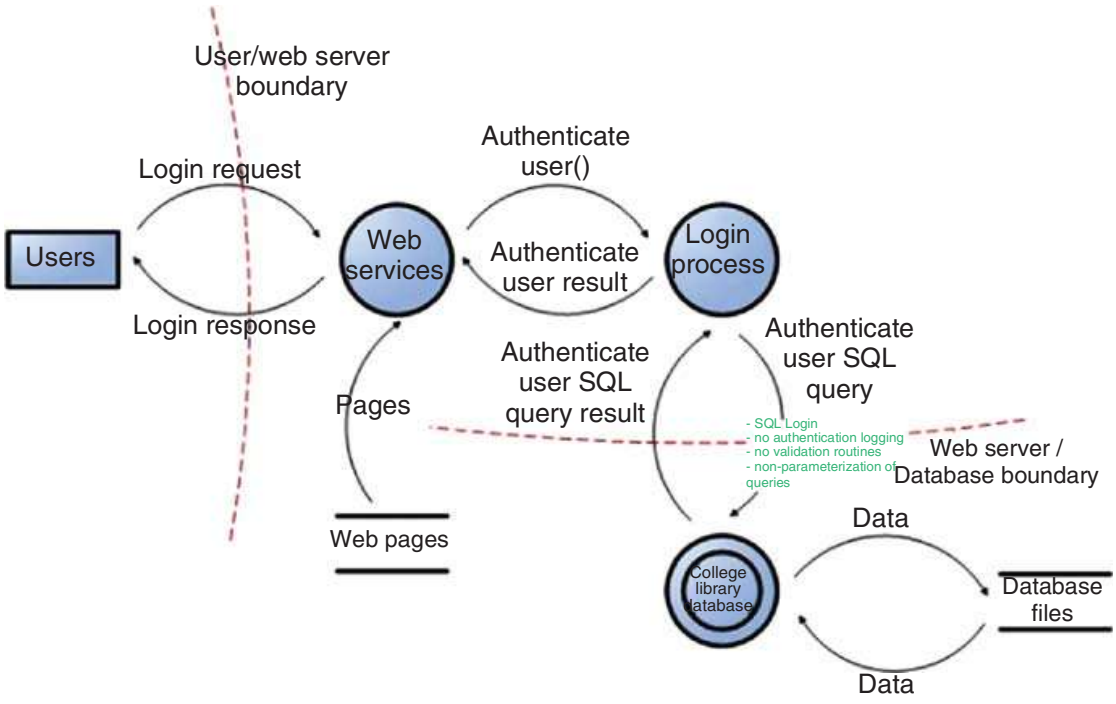


Figure 7.28 Missing Architectural Countermeasures among Application Components

The importance of conducting architectural reviews in this stage is that the other forms of automated assessment tools will not consider design flaws or simply poor architecture. As shown in the aforementioned figure, absent controls (such as those in green) are easily missed by vulnerability scanners or dynamic application scanners. Analyzing the security architecture, we can potentially augment the prior list of known vulnerabilities with more holistic design weaknesses identified from this activity.

Stage V: Activity III (S5:A3) – Map Threats to Vulnerabilities Building off of the attack tree (introduced in stage IV), we can now begin to look at the aggregate of design flaws and software vulnerabilities and map them to branches on the attack tree. This "tree" is maintained as a visual representation of the relationship between vulnerabilities/design flaws and threats. The mapping also provides a relationship node to the use cases affected by the threat. After each stage thus far, a clearer depiction of how the threat affects the application component(s), associated use cases, vulnerabilities and design flaws becomes clearer. One important node in this developing attack tree is the abuse case. The abuse case maps to the use case and plans out a strategy for exploitation, without necessarily knowing an attack vector. The vulnerability node in the attack tree provides a plausible point of entry or kink in the application armor and therefore supports the abuse branch of the tree. We see both the *abuse case* and *vulnerability* layer depicted in Figure 7.29.

In order to keep the figure simple, certain threats have only one defined abuse case; however, one-to-many relationships may exist between threats and underlying

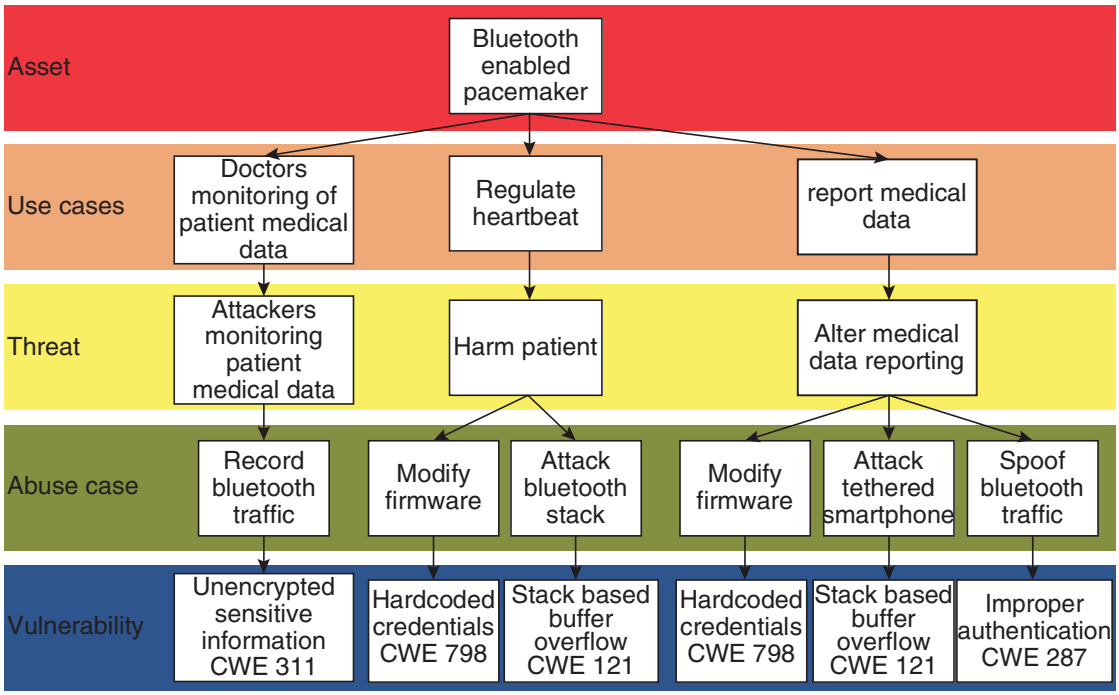


Figure 7.29 Abuse Cases & Vulnerability Branch to Attack Tree Added

branches. You may find more one-to-many relationships as you work through your own application.

This activity can be completed best in Visio and by referencing two distinct libraries: CVE lists and CWE lists. In most cases, vulnerability data from scanning reports will denote either and can be associated to the branch in the tree that corresponds to the analyzed abuse case that could exploit a vulnerability. The abuse case is aimed at being the plan of attack for a threat agent and used for the purpose of realizing one or more facets of their defined threat(s). As shown in the aforementioned figure, these threats affect the security of various associated use cases for the asset or application. The trunk of the tree is the targeted asset which may represent a physical server, a virtual host, a software application, a third-party product, or a database.

Paying attention to the vulnerability branches of the tree, we see that, for example, an abuse case of recording Bluetooth traffic may allow sensitive data to be compromised in transit. This may be something to consider architecturally as part of the product application design in Stage VII (Residual Risk Analysis).

Abuse Cases Abuse cases are supportive steps to achieving a threat's intent. They are a beneficial, yet not mandatory part of the overall attack tree. They help to depict high-level attack plans in manners that are understandable by everyone involved in the threat modeling effort. From architects to business analysts – all participants would appreciate the simplistic terminology associated with abuse cases. This simple terminology makes creating abuse cases quite easy. Simplistically, they can be a distortion of the actual use case name. For example, a use case of "Authenticate User" could have an abuse case named "Steal User Authentication." Another example is the use case of "Provide Secure Checkout Function." The distortion of this use case would lead to the abuse case of "Bypass Checkout Function." Another form of building out abuse cases is to focus on the target of the threat. If the threat's target is data, then abuse cases should be construed based on their relevance to both the application component that they are *abusing* and the threat objective. For example, a threat objective may be to bring down a Bulk Energy System (BES). The abuse cases may include (1) gaining illicit network access, (2) bypassing authentication in FTP infrastructures or (3) phishing the utility company's IT administrators. There are plenty of examples and they can be developed based on simply building the attack trees collaboratively. A brainstorming session among threat modeling participants will quickly build the necessary abuse case layer for the attack tree. Another important point to remember is that multiple abuse cases per use case, therefore it is not necessary to simply think of just one.

In Figure 7.29 (from bottom to top) the vulnerabilities or design flaws facilitate a possible abuse case. The abuse case represents the threat intent from the prior layer of the tree. The threat serves as the pinnacle of the overall threat model against the application as a whole and/or its targeted software components.

Insufficient Vulnerability Data In the case of a new application, historical vulnerability data may not be present. If this is the case, a vulnerability database can be

queried for relevancy based on application component name/version. Common Vulnerability Enumeration (CVE) and/or Common Weakness Enumeration (CWE) lists are provided in multiple formats by the MITRE (www.mitre.org) organization as well as included in various vulnerability scanners. Those scanners may or may not have the vulnerability data in a searchable format, but if they do, vulnerabilities may be pulled from the tool and mapped based on the abuse cases. Security advisories/bulletins on employed frameworks or software components are also a good source for vulnerability data to pull from. A special focus open frameworks and client-side plug-ins, if part of the technology scope (stage II), should be regarded. Specific vulnerability briefings on any employed application components are also helpful. Additional sources for vulnerability data that could be used in the absence of prior vulnerability assessment reports are as follows:

- Open Source Vulnerability Database – http://www.osvdb.org.
- Oracle (http://www.oracle.com/technetwork/topics/security/alerts-086861.html).
- Microsoft (http://technet.microsoft.com/en-us/security/dn481339).
- US-CERT (http://www.us-cert.gov/ncas/alerts/).
- Secunia (Security Advisories by Vendor http://secunia.com/community/advisories/vendor/).
- Carnegie Mellon University Vulnerability Notes Database – (http://www.kb.cert.org/vuls).
- Exploit-DB (http://www.exploit-db.com/) – Contains information on various types of exploits against various types of vulnerable software.
- Security Focus (http://www.securityfocus.com/) – Mixed back of security vulnerabilities across different technology products.
- Cisco Security Advisories (http://tools.cisco.com/security/center/mpublicationListingDetails.x?docType=CiscoSecurityAdvisory).

There is a lot of vulnerability data and without a proper tool or mechanism to consume, query, and organize relevant vulnerability data, a significant amount of time would be require to conscientiously review. For this reason, it may be better to simply address design flaws in the application/network architecture during this iteration of the application threat modeling effort. Alternatively, a targeted vulnerability research effort can be done to identify vulnerabilities for the top and most impacting threats/asset components. If the company has an internal vulnerability database, it may leverage this as a way to correlate relevant vulnerabilities based on the scope of asset components being utilized that may be vulnerable.

Stage V: Activity IV (S5:A4) – Provide Contextual Risk Analysis Based on Threat Vulnerability In this activity, we leverage previously identified and validated vulnerabilities and weaknesses and apply them to the attack tree for better context (in terms of relevance to a targeted asset or component). In doing so, this slowly builds

a prioritization model for remediation, based on relevance to use cases, asset components, and threat probabilities. The criterion around threat probabilities (introduced in stage IV), combined with confirmed vulnerabilities can begin to demonstrate the viability of successful abuse cases. Identified threats against asset components that are easy to interface with, having known vulnerabilities or design weaknesses should thus far receive the greatest attention, particularly if the use case pertaining to the asset component supports a key business objective.

Identified vulnerabilities sustain the possibilities for a threat to take place. Not all vulnerabilities are created equal however. Some vulnerabilities or design flaws greatly facilitate certain abuse cases than others. In this activity, developers, architects, certain QA professionals, and strategic third-party security testing partners will help to identify both weaknesses and vulnerabilities that affect the use cases, actors, services, and overall asset function within an application environment.

This activity helps to (1) associate vulnerabilities to assets in the threat model and (2) sustain the viability on how weaknesses and vulnerable components could facilitate the threat depicted. This stage's intrinsic value is in depicting the possibility of attack and also beginning to make an association for how certain attacks could exploit the identified vulnerability or application weakness. The following figure illustrates how the logical flow of contextual analysis should take place across both threats and vulnerabilities. This logical representation presented subsequently provides a way to consider vulnerabilities in the context of the threat, the target asset (or component) and ultimately how it ultimately affects risk levels for the application.

Figure 7.30 provides that logical analysis needed to determine (at various binary decision intervals) how a given vulnerability affects the viability of a threat. This later will translate into a higher probability for a vulnerability to become exploited as well as a greater relevancy to the threat becoming successful. As a result, it provides the opportunity for a premature risk analysis using the contextual variables of threat(s) and application component (or asset). The vulnerabilities relevance to a threat becoming successful is one risk-based consideration while the other explores the viability of the vulnerability to undermine an asset's intended use case(s) (impact). Using this knowledge, we can now factor in this analysis to how targeted vulnerability testing should take place.

Stage V: Activity V (S5:A5) Conduct Targeted Vulnerability Testing This stage is simply not about performing vulnerability testing or having one performed by a third-party group. As most IT and Security professionals know, vulnerability reports can be riddled with false positives and extend beyond a manageable response for remediation. Additionally, scope creep is a common problem for vulnerability tests conducted across many large enterprises and even small businesses. Since much of the scope targets are focused on ranges and network blocks, many of those conducting vulnerability tests have no clue on what they are truly testing in terms of asset use. For this reason, *vulnerability trimming*, or selecting vulnerabilities for tailored testing, based on threat relevance and application component, should be applied. This allows for targeted vulnerability testing to evaluate components that are within the application scope and in support of the overall threat model. The overall threat model

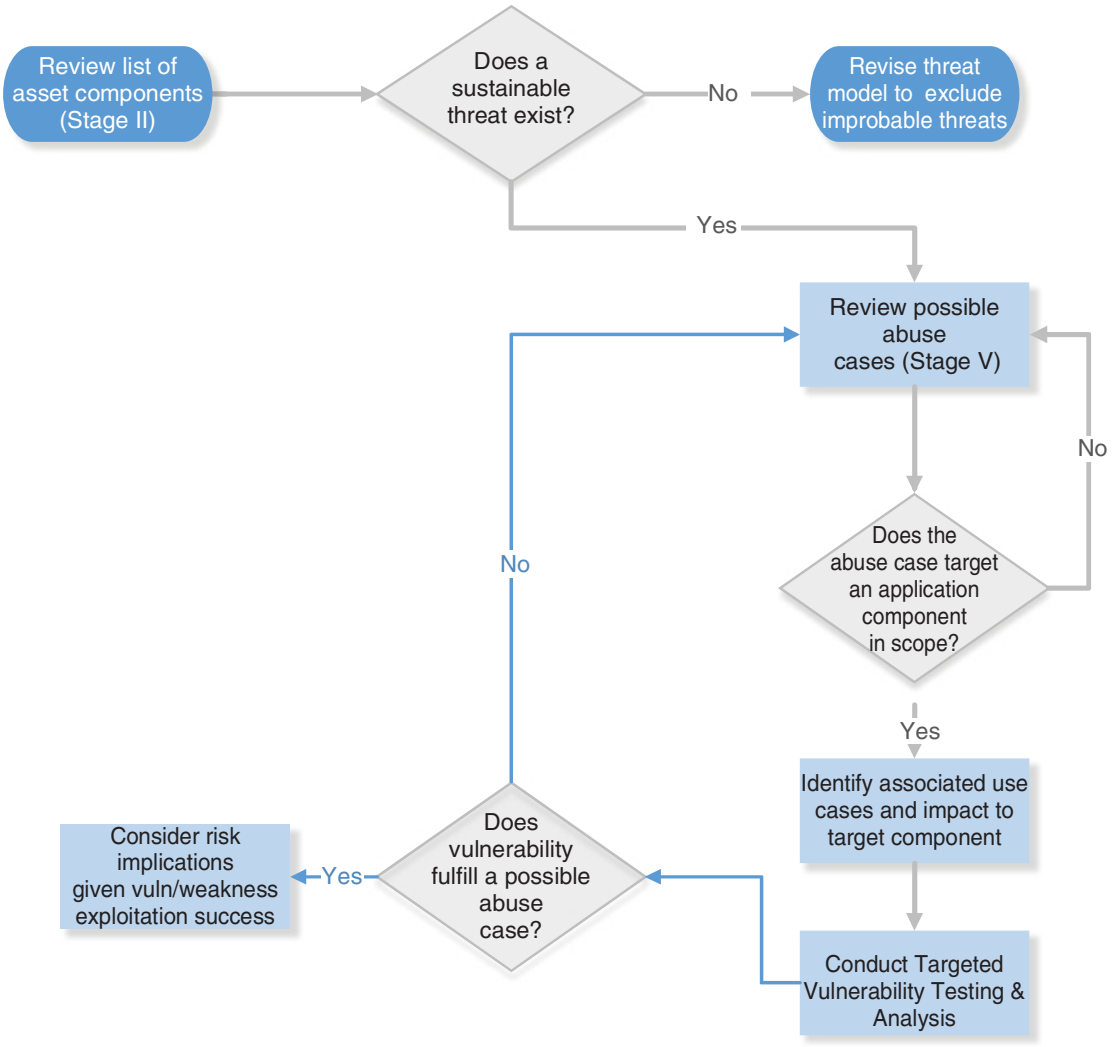


Figure 7.30 Logical Flow Considering Threats to Assets to Vulnerabilities

will have a threat and series of abuse cases that will look to exploit vulnerabilities or design flaws. The tester will have the ability to use CVE and CWE names and description tags in order to appropriately launch vulnerability/weakness checks that support the viability of a threat model. For this activity, we will look to identifying how vulnerability trimming can take place in order that targeted vulnerability scans can be achieved.

Conducting vulnerability scans is largely fulfilled by various active and passive network/application scanners. There may already be a number of tools within your enterprise or security operations team that may be poised to complete such types of vulnerability scanning. In order for targeted vulnerability scans to be properly leveraged, the following scanning techniques or review exercises should be applied.

1. For existing vulnerability reports or scans, if possible, narrow down the scope of components with prior vulnerabilities in order to use as a criteria for scoping out a targeted scan.

2. If those assets or application components have not been scanned, then they should be now. The enumeration and scoping exercises that took place in stage II (Technical Scope) greatly assist in incorporating assets or application components that require greater vulnerability analysis and that are most relevant to the application environment.
   a. From a platform or asset level, use nmap to scan the hosts within the application environment (no CIDR notations or ranges).
   b. Fingerprint services, ports, and OS versions for assets in scope using nmap. Those services, ports, and underlying operating system are all components that are in scope for testing.
   c. Use network-based vulnerability scanners to select the specific component or set of components that are in scope, based on the threat model.
   d. For example, we may want to focus a specific vulnerability test on the components that support the "Forgot Password" feature on a website. The multitiered components would be a client-side technology such as a JAR file or a compiled Flash objective. It also includes the IIS web server and any backend application components at the application or data tiers. See following screenshot and highlighted area for how targeted use cases tied to application components can be tested (Figure 7.31).
   In the aforementioned example, we see that the *forgotpw* page is called in order to retrieve a password for a given user. This use case may be part of

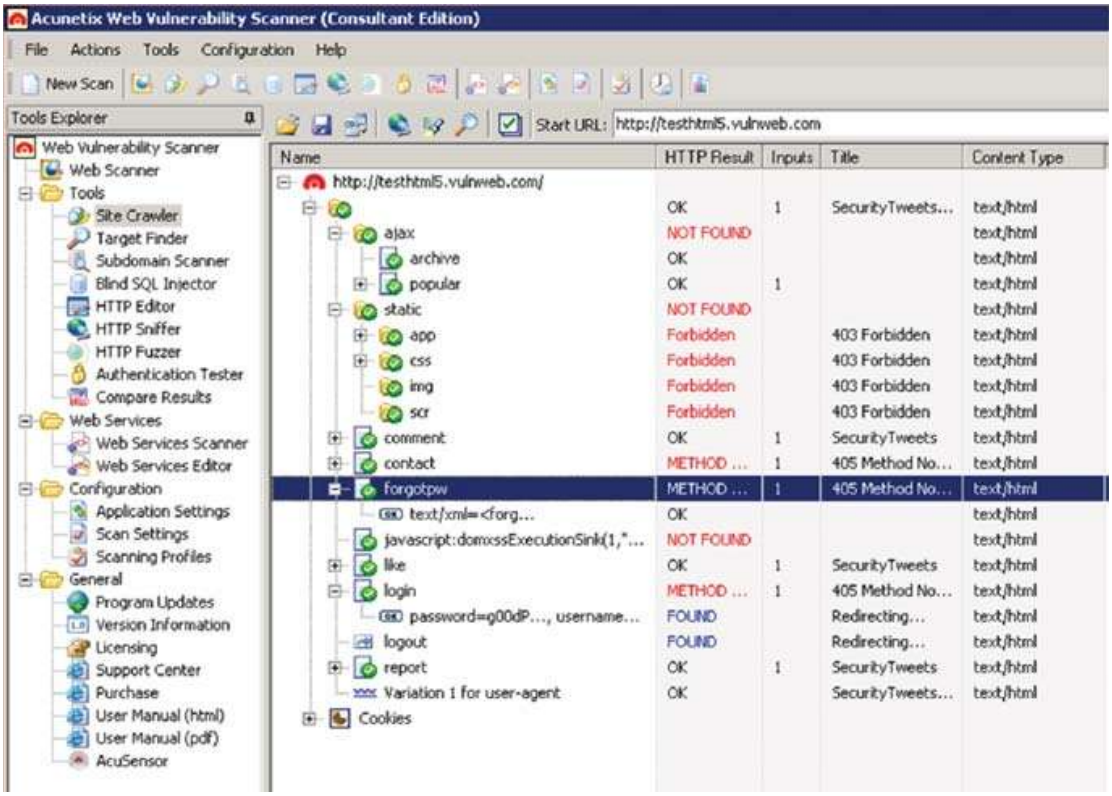


Figure 7.31 Targeted Application Testing in Web Applications

TABLE 7.17 Labeling Relevant Threat Modeling Variables during Targeted Assessment Efforts

Threat Modeling Label	Application Component
Target (asset)	Web portal
Use case	Password reset
Threat	Steal credentials
Abuse case	SQL injection (Time based)
Vulnerability 1	No input validation
Vulnerability 2	No parameterization of queries
Weakness 1	No restrictions to password reset attempts
Weakness 2	Insecure transport layer for sensitive data

a targeted threat in a given threat model. This being the case, we use the context of *component relevancy* and *threat relevancy* to determine that this page and underlying use case should be scanned. Broken up into clear labels, vulnerability testing against this use case should be considered as follows (Table 7.17).

e. Various types of vulnerability scanners will allow for customer scan types or scan modules to be selected. This provides a more targeted approach to not simply select an entire node or range of nodes to scan.

f. If application-based scans are being conducted, one can spider or traverse the branches of a site or individual application component as part of the scan. Application components in the client or presentation tier will relate to technologies that may require static or dynamic application scanners. For example, static source code analyzer may be used to review client-side code such as JavaScript. Application de-compilers will assist to reverse engineer Flash, PowerBuilder, Silverlight, or Ajax files into decipherable code. Once such client-based technologies have been decompiled, they may in turn reveal Application tier components such as exposed web service namespaces, from which now we can leverage web-based proxies to test these exposed tiers for web, mobile, or cloud-based applications. Some of these APIs may in turn leverage messaging frameworks that require testing. Beyond the testing, this activity also helps to reconcile against the activities performed to enumerate the technical components for the application in stage II.

Note: For testing in this activity, it is suggested to use authenticated scans rather than unauthenticated scans. This will ensure that false positives are not tainting the results of this targeting scanning effort.

3. A list of all in-scope assets (from a system level) should be scanned and vulnerabilities correlated to the threats, use cases, and abuse cases denoted in the "living" attack tree. The attack tree is referred to as "living" since it should be continuously updated upon revealing more branches of vulnerabilities. Ensuring that the following relevant platforms are scanned (either manually or via automated tools) is important in order to ensure that *vulnerability trimming* does not exclude relevant components identified by these scans.

a. Distributed servers
b. Midrange servers
c. Personal computing devices
d. Mobile devices
e. Network infrastructure equipment
f. Network appliances
g. Application proxies

Multiple scenarios for targeted testing can be depicted under this section; however, it would really all repeat the same basis formula of simply identifying what vulnerabilities are relevant to two key items: (1) threat attempted to be realized and (2) the target asset or application component. These two variables determine whether or not a given vulnerability is essential for inclusion. The end goal realized by this activity is a tremendous amount of time savings in both the actual vulnerability scanning as well as the review of vulnerability scan results. Last, it makes the results much more topical to the threat model and evolving attack tree that is being built across the PASTA process.

A summary of all the activities for this stage, as well as inputs and outputs, is depicted in Figure 7.32.

Stage VI – Attack Modeling & Simulation (AMS)

This stage's key objective is to complete the attack tree. The attack tree is a centerpiece for application threat modeling because it organizes all of the information around threats, target assets, abuse cases, vulnerabilities, weaknesses, and the exploits that profit from all of the above. In PASTA, this stage will continue to build from its prior stages in the manner just mentioned. Being a risk-centric threat modeling approach, we will determine the probability for a vulnerability to be exploited. As a result, the prior version of the risk equation introduced in stage IV (Threat Analysis) will now encompass another probability coefficient that is tied to the vulnerability variable. The formula will now look as follows:

$$(R)\text{isk} = \frac{(T)\text{hreat}_{(P)\text{robability}} \times (V)\text{ulnerability}_{(P)\text{robability}} \times (I)\text{mpact}}{(C)\text{ountermeasures}}$$

In the last stage, we identified relevant vulnerabilities that relate to both the desired threat as well as the targeted asset. In this stage, we want to model the different types of attacks that would exploit the relevant vulnerabilities for the application, all aimed at realizing the various threat objectives. From a well-defined attack tree, we are able to see the scope of assets, actors, services, and other entities defined in the other stages that came before stage IV (Threat Analysis). The attack tree completion will ultimately reveal different layers of attacks; all mapped to preexisting vulnerabilities that facilitate the exploitation of data, credentials, or simply online reliability of the application or system. The attack tree then becomes a key threat modeling artifact to exemplify which viability of a threat and the affected application components that would succumb to the attacks that achieve said threat.

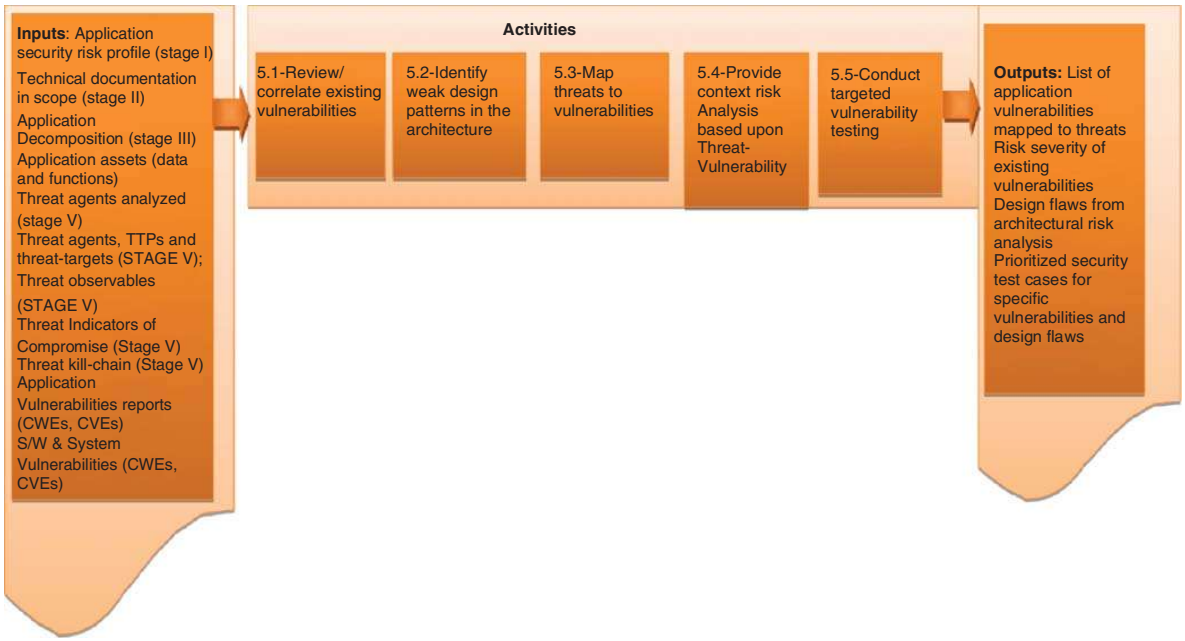


Figure 7.32 PASTA Risk-Centric Threat Modeling – Stage V – (WVA) Weakness and Vulnerability Analysis

Activities in this stage are focused on discovering the viability of exploiting identified vulnerabilities and weaknesses around the application. Thus far, the application threat model has depicted what technology footprint the application is to use as well as what is potentially flawed with its innate software, network, or design specifications. This stage aims to determine how viable successful attacks are in exploiting identified and confirmed vulnerabilities in the application. Typically, penetration testing would need to be exercised against the product system to validate previously found weaknesses. For this reason, all of these activities are aimed at running in parallel to the development phase of a generic SDLC life cycle and are to be performed in various degrees. As such, the key activities for this stage are summarized as follows:

- S6:A1 – Analyze possible attack scenarios.
- S6:A2 – Update the attack library/vectors and the control framework.
- S6:A3 – Identify the attack surface and enumerate the attack vectors.
- S6:A4 – Assess the probability and impact of each attack scenario.
- S6:A5 – Derive a set of cases to test existing countermeasures.
- S6:A6 – Conduct attack driven security tests and simulations.

Key artifacts generated from this stage include a fully matured and layered attack tree that visualizes how asset components are affected by identified threats. The following sections illustrate procedures and supporting artifacts that pertain to this stage of PASTA.

Stage VI: Activity I (S6:A1) Analyze the Attack Scenarios The goal for this activity revolves around enumerating and analyzing possible attack scenarios. In doing so, we have to consider the target asset(s) and related threat components. Additionally, we want to build off of the selected threats that have been substantiated by our work under stage IV of the PASTA methodology. Doing so, we can brainstorm what type of attack scenarios could be achieved given the threat intelligence we have of current threats affecting similar application or system environments, as well as the context of what the target asset performs or provides in terms of value to an adversarial group or figure.

Within this in mind, building attack scenarios begins with a re-examination of the threat hierarchy. The progression in an asset-centric threat modeling methodology begins with the asset under this hierarchy. The threat modeling team needs to identify attack branches that stem from the target asset using the following rationale (Figure 7.33).

As shown in attack trees, the threat is prefaced by the use case and followed by an abuse case which provides a counter to the use case's objective. The more of these layers depicted, the clearer the threat model becomes in terms of how attacks become viable and realistic to defend against.

In the following brief example, we take a threat scenario for a restaurant chain whose Point of Sale (POS) system is the subject of threat to capture credit card data.



Figure 7.33 Linearly Thinking about Attack Patterns

TABLE 7.18 Attack Considerations for POS at Restaurants

Target Asset	Point of Sale (POS) Device
Asset use case	Process card payments at restaurants
Threat scenario	Install malware to capture card data
Abuse case	Use authorized credentials to install malware
Vulnerability	Users have elevated accounts on POS systems
Attack scenario 1	Deliberate install of malware onto POS by employee
Attack vector 1	Sneaker-net; physical access to POS; Download malware over Internet
Attack scenario 2	Unintentional install of malware onto POS by employee
Attack vector 2	Physical access to POS; Drive by download on allowed Internet site
Attack scenario 3	Compromised update/configuration path to include rogue malware as part of update from system administrator
Attack scenario 3	Update server compromised and unchecked installer introduced malware to target POS.

Using the following given variables, we will proceed to analyze high-level attack scenarios.

In Table 7.18, we see a simple example of how this activity should be carried out. We begin with the logical root of the asset or target subject. This may also be a component of the overall asset used within the application model. The simple example depicted was intended to provide a generic creation of possible attack scenarios that can be considered given other threat characteristics as shown by the aforementioned table. The end goal is to arrive at a list of possible attack scenarios that are operationally possible as well as technically feasible given the known vulnerabilities in the environment.

Stage VI: Activity II (S6:A2) Update Attack Library/Vectors and Control Framework For this activity, we focus on ensuring that our list of attacks as well as possible control measures is vast enough in order to properly build a threat model – a key objective for this stage. The goal of this activity is simply to search for a comprehensive library of attack patterns that traverse a diverse range of attack vectors, or channels of attack. Similarly, we would need to properly develop or maintain a list of possible controls, as part of a broader control framework. This list of controls should equally be vast enough to encompass a diverse set of attack surfaces and attack vectors.

It may be difficult to enumerate different types of attacks if you are not familiar with attack patterns. Attack patterns represent a sequence or nonsequential collection of abusive actions against a target. Each pattern represents a collective of abuse cases for an attack. It is important to ensure that a wide variety of attack patterns are available from which to query and assign to the attack tree. In light of this, it is critical to have a comprehensive and up-to-date attack library. One of the best

and comprehensive attack libraries is the CAPEC attack library from MITRE.[7] The CAPEC library boasts approximately 400 attack patterns or CAPECs in their latest (2014) iteration of their library. The library itself has been made searchable by various other sites, namely Security Database,[8] on online PHP application that has made the entire CAPEC v.1.5 version searchable by CAPEC ID. This is truly the most comprehensive threat library out there today; however, if you would like to create your own, that is suitable as well, as long as a unique identifier can be associated with each pattern and that associated control responses are also aligned with the specific control pattern. Homemade attack libraries can be made by amalgamating various sources of information in order to build a custom attack library. Sources may be derived from OWASP,[9] WASC,[10] PTES,[11] or even various resources from the SANS Reading Room.[12] The tool, ThreatModeler™,[13] is a free threat modeling tool that incorporates the CAPEC attack library and allows the threat modeling team to assign attack patterns to threats defined in stage IV. Other tools previously mentioned, such as TAM, do not incorporate all of the CAPEC libraries so mapping attacks to parent node threats will not be as effective using TAM.

If your product team would like to get the CAPEC library directory and parse the entire library, both the data schema and the XML data are available for download at the MITRE site (http://capec.mitre.org/about/documents.html).

It is not suggested that attack libraries become aggregated and correlated separately as this will be very time-consuming. If an internal library is being developed, attack patterns should be added, normalized, and assigned unique category identifiers for ease of cross reference and lookup. Separately, several different pen testing frameworks and tools have the CAPEC library built into the product. Depending on the solution, it may be possible to query this attack library. Ultimately, developing or obtaining an attack library from external sources is simply the initial means for having a diversified pool of possible attack patterns to consider in the context of the threat and the overall threat model.

The benefit about the CAPEC library is that it also comes with an associated "Solutions & Mitigations" & "Controls" section, which essentially translates to the possible countermeasure to resist the attack pattern. A snapshot of what this looks like is as follows and taken from www.security-database.com (Figure 7.34).

CAPEC greatly facilitates the matching of attack patent to possible control measures. For this reason, many manufacturers of pen testing products have adopted the CAPEC library into their own tool solution. These tools are thereby able to ingest the XML CAPEC content and related elements that depict attack characteristics as well as related countermeasures. This eliminates the need of having a distinct attack

[7] http://capec.mitre.org/about/index.html.
[8] http://www.security-database.com/capec.php.
[9] https://www.owasp.org/index.php/Category:Attack.
[10] http://projects.webappsec.org/w/page/13246978/Threat%20Classification.
[11] http://www.pentest-standard.org/index.php/Main_Page.
[12] http://www.sans.org/reading-room.
[13] http://myappsecurity.com/threatmodeler/download.

Security Controls

ID	type	Security Control Description
1	Preventative	Avoid providing any indication regarding the validity of user IDs upon failed login attempts. Provide a simple error message such as: "Login failed. Try again or contact your administrator" regardless of why a login attempt fails.

Figure 7.34 Snapshot of Related Control from CAPEC ID in Library

library from a library of possible countermeasures, particularly when building both an organic attack library and control framework.

Stage VI: Activity III (S6:A3) Identify the Attack Surface and Enumerate the Attack Vectors In this chapter, we use the approach to identify the full attack surface for the scope of the threat model using the visualization of attack trees. The attack surface will encompass each of the possible attacks that can exploit the vulnerabilities identified from the prior stage. Attack trees provide a visual representation of the relationships among attacks and their vulnerability counterparts, along with the preceding contributing factors that sustain the attack (abuse case, threat, etc.).

A key goal in this activity is to ensure that the attack tree is finalized. We have begun with the target asset and listed underlying functional components (actors, use cases, services, processes, etc.), as well as dysfunctional components (threats, abuse cases, weaknesses, vulnerabilities, attacks, attack patterns, etc.). The final tree can be represented in multiple ways. Traditionally, they are represented via Visio schematic-like drawings, such as the ones previously referenced.

A diagram showing a simplified attack tree is included subsequently. For this exercise, we imagine modeling threats and attacks around a wearable or implantable medical device – a pacemaker. This product is listed in the top row of the attack tree. Please note that for simplicity's sake we have limited this threat model to a single asset and a narrow range of vulnerabilities with which attack mappings will take place.

Prior to introducing the attack tree for the pacemaker, the following are some simple considerations to keep vigilant when developing your own attack tree.

1. (Asset) The application we are using to build the attack tree is a pacemaker.
2. (Use Cases) The base functionality of the pacemaker is to monitor the patient's heart rate, regulate their heart rate, and then connect via Bluetooth to a smart phone in order to send heart rate information to the patient's primary care physician.
3. (Threat) Next, we consider how this functionality could be misused. In this row, we see threats that could occur if this device deviated from the intended use case. For example, if the heart rate regulation abilities functioned in a fashion other than intended, it could cause harm to the patient.
4. (Abuse Case) In the next row, we see what abuse cases could lead to this. Let us focus on attacking the embedded Bluetooth stack.

5. (Vulnerability) In the next row, we will enumerate specific technical vulnerabilities that could foster the aforementioned use case; in this case, it is a buffer overflow within the Bluetooth module's firmware.
6. (Attack Patterns) Next, we list out attack patterns or methodologies that an attacker would utilize with this specific type of technical vulnerability. Here we see that if they opted to create a buffer overflow for the Bluetooth module, their attack pattern would target hardware components. If this were not meant to be a simple and streamlined example, we could add other attack patterns such as a memory corruption attack pattern.
7. (Impact) Finally, we must consider what the ultimate impact of a successful attack would be. In this case, the attacker would have gained full control over the pacemaker's heartbeat regulation capabilities, ultimately allowing them to harm the patient as suggested by the threat row.

The following attack tree builds from the prior tree (Figure 7.29) in the vulnerability stage by adding the CAPEC attack patterns that are relevant for exploiting the identified vulnerabilities/weaknesses in the application model. The various layers can be color-coded for ease of use. This use of color coordination may be used to organize the layers of the attack tree by a criteria determine to be useful and beneficial by the threat modeling team (Figure 7.35).

In reviewing the completed attack tree, we see that the revealed attack surface is inclusive of a PAN (Personal Area Network), given the Bluetooth sniffing attack. We also see that the attack surface encompasses embedded system or software components (CAPEC 440, 401, and 440) that may be vulnerable for alteration. Last, we have a mobile device platform that can serve as an extension to the attack surface given its interoperability with the target asset – the pacemaker. The attack surface encompasses at least three different vectors of attack that could realize the vulnerabilities identified by the threat model. One vector is over near field communications enabled by the Bluetooth technology on both the phone and the pacemaker itself. Another vector is in the development of the pacemaker itself, therefore related to the manufacturing process by the manufacturer. The last is over a mobile device that interfaces with the pacemaker software. Understanding the attack surface, as well as channels of attack, provides a great visualization to share among those engaged in the PASTA process. The completed attack tree provides a great artifact for depicting how attack scenarios profit from the underlying components that facilitate the goals of a defined threat. In the next section, we will address the probabilities associated with each of the identified attack patterns that are part of this threat model.

Stage VI:Activity IV (S6:A4) Assess the Probability and Impact of Each Attack Scenario One key goal in this activity is to review the attack scenarios provided thus far and determine probability for success around exploitation. This pseudo-probabilistic analysis is useful in identifying where the most urgent part of threat mitigation should take place. Using the CAPEC library, some of the elements for each CAPEC provide information on the severity and the difficulty associated with exploiting a target weakness and/or vulnerability. Revisiting the attack tree

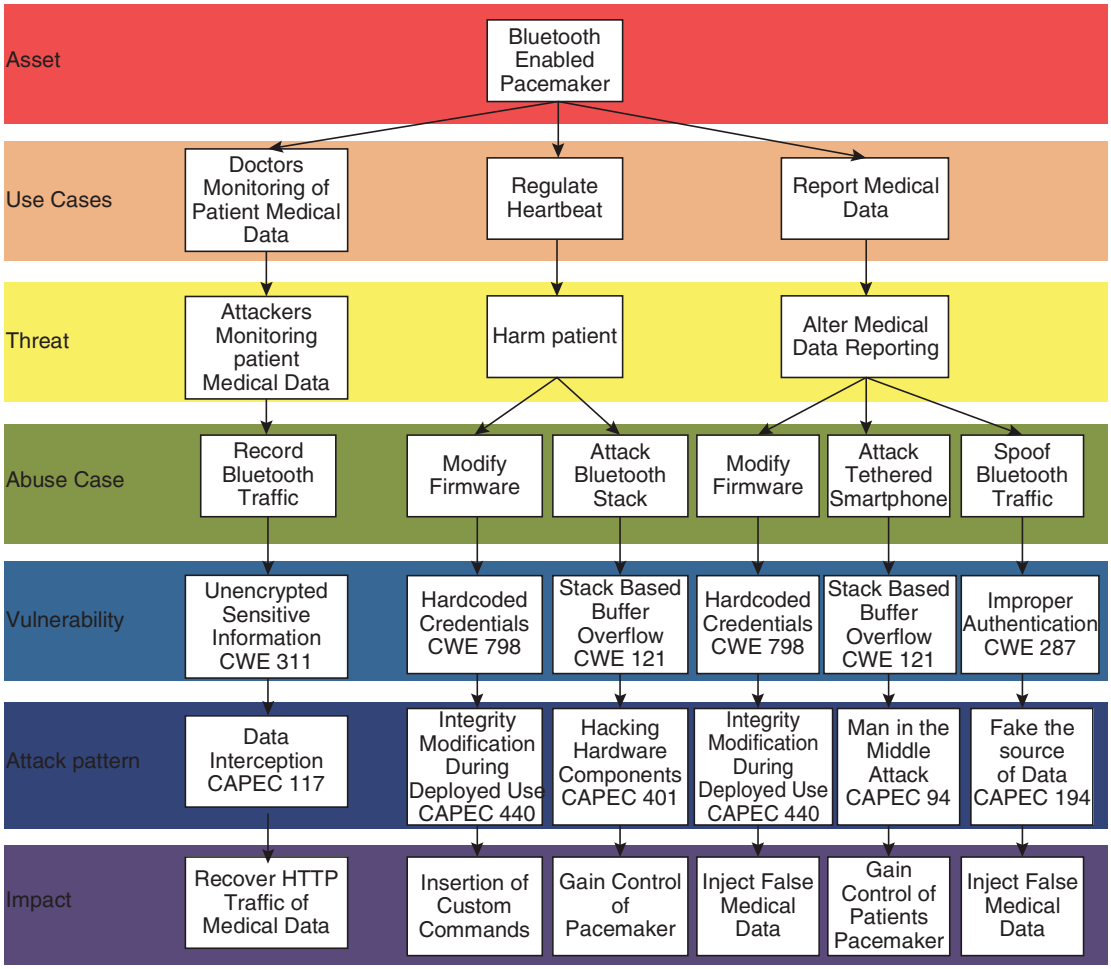


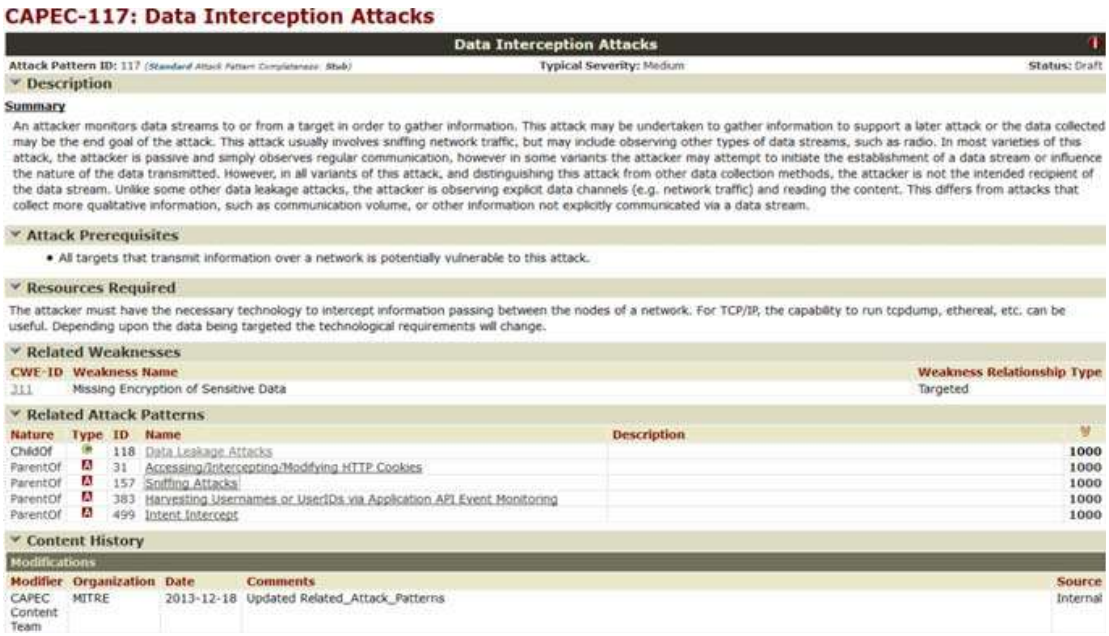
Figure 7.35 Completed Attack Tree

from the prior activity, we take a look at CAPEC-117 (Data Interception). That CAPEC has associated prerequisites and other CAPECs that go with it. See, in the following figure, the other CAPEC associations as well as the weakness (CWE) IDs that are associated with this CAPEC ID of 117[14] (Figure 7.36).

Determining the probability will hinge upon the following criteria for each attack pattern listed on the attack tree. These conditions are as follows:

1. *Attack prerequisites:* Are these prerequisites surmountable for most attackers, based on time allotments, complexity of attack, cost of attack (resources, tools, etc.)?
2. *Weakness/Vulnerability Maturity:* Is the maturity of this weakness or vulnerability widely disclosed, exploited in the wild or is it a new, isolated vulnerability with very little information around PoC?
3. *Hackability:* Can you or your team exploit the vulnerability/weakness? Partially? Fully? This is one of the better indicators and one that will be exercised

[14]https://capec.mitre.org/data/definitions/117.html.



CAPEC-117: Data Interception Attacks

Data Interception Attacks

Attack Pattern ID: 117 (Standard Attack Pattern Completeness: Stub) Typical Severity: Medium Status: Draft

Description

Summary

An attacker monitors data streams to or from a target in order to gather information. This attack may be undertaken to gather information to support a later attack or the data collected may be the end goal of the attack. This attack usually involves sniffing network traffic, but may include observing other types of data streams, such as radio. In most varieties of this attack, the attacker is passive and simply observes regular communication, however in some variants the attacker may attempt to initiate the establishment of a data stream or influence the nature of the data transmitted. However, in all variants of this attack, and distinguishing this attack from other data collection methods, the attacker is not the intended recipient of the data stream. Unlike some other data leakage attacks, the attacker is observing explicit data channels (e.g. network traffic) and reading the content. This differs from attacks that collect more qualitative information, such as communication volume, or other information not explicitly communicated via a data stream.

Attack Prerequisites

- All targets that transmit information over a network is potentially vulnerable to this attack.

Resources Required

The attacker must have the necessary technology to intercept information passing between the nodes of a network. For TCP/IP, the capability to run tcpdump, ethereal, etc. can be useful. Depending upon the data being targeted the technological requirements will change.

Related Weaknesses

CWE-ID	Weakness Name	Weakness Relationship Type
311	Missing Encryption of Sensitive Data	Targeted

Related Attack Patterns

Nature	Type	ID	Name	Description	
ChildOf		118	Data Leakage Attacks		1000
ParentOf		31	Accessing/Intercepting/Modifying HTTP Cookies		1000
ParentOf		157	Sniffing Attacks		1000
ParentOf		383	Harvesting Usernames or UserIDs via Application API Event Monitoring		1000
ParentOf		499	Intent Intercept		1000

Content History

Modifications

Modifier	Organization	Date	Comments	Source
CAPEC Content Team	MITRE	2013-12-18	Updated Related_Attack_Patterns	Internal

Figure 7.36 MITRE CAPEC Library Snapshot – CAPEC 117

in the next activity. If the attack pattern can be successfully executed by your testing team, then it is a good chance that a determined hacker or hacker syndicate will be able to do so with far greater ease, due to greater amount of time and resources that they have to conduct such exploitive measures.

Another simpler approach would be to apply estimates to probabilities based on contextual information on what we know about threat model components. In consideration of this approach, the following areas can be evaluated.

1. *Probability of threat agents, motives, and abuse cases against target assets*
   a. *Probability Coefficient:* The team should assign a probability for the threat to be conducted successfully based on analysis from stage IV (Threat Analysis) and the success of Stage VI (Attack Enumeration). Moreover, the probability could factor in the maturity of the threats and/or attacks within the industry and operational areas where the product application will be implemented.
      i. The probability coefficient should between 10% and 90% as a standard deviation of 10% is assumed on both ends automatically in the formula used in Stage VII. This probability should factor in a subjective analysis on the motives for launching the threat.
      ii. *Example:* Bluetooth Traffic Interception Probability = 25%.
2. *Maturity of vulnerability discovered*
   a. *Probability Coefficient:* The team should assign a probability for the vulnerability that is associated with threat identified in stage IV (Threat Analysis) and base it on the maturity of the vulnerability.

   i. The probability coefficient should between 10% and 90% as a standard deviation of 10% is assumed on both ends automatically in the formula used in Stage VII. The percentage reflects the probability that this vulnerability is mature and widely known.
   ii. *Example:* Unencrypted Data Prior to Transport (Design Flaw) = 90%.
3. *Severity Rating (Threat | Vulnerability | Impact)*
   a. *Threat:* Using a scale of 1-4, denote the severity rating (1 = Low | 2 = Moderate | 3 = High| 4 = Critical) if the threat were realized.
   b. *Vulnerability:* Using a scale of 1-4, denote the severity rating (1 = Low | 2 = Moderate | 3 = High| 4 = Critical) of the identified vulnerability (excluding all other considerations such as accessibility, prerequisites, etc.).
   c. *Impact Level:* Using a scale of 1-4, denote the severity rating (1 = Low | 2 = Moderate | 3 = High| 4 = Critical) of the impact to the business or product objectives if the threat were realized.

Another key objective under this activity is to determine impact. Reverting back to the attack tree figure from the prior page, we see an impact row. The impact is tied to the assumption that the attack pattern will be successful and now we must calculate what the adverse effects are to the business objectives for the application as well as the specific use case affected by the attack pattern. Revisiting CAPEC 117, we see the affected use case of "Doctors Monitoring of Patient Medical Data." The immediate impact of this may not be financial, but may ultimately affect patient safety and privacy. As such, the successful exploitation of this use case needs to be regarded by the threat modeling team in the following manner:

1. Is the successful exploit one that introduces immediate financial loss (one can use the Single Loss Expectancy (SLE) calculation for this, which equals Asset Value (AV) x Exposure Factor (EF). Under this formula, we make EF 100% if the exploit is feasible and proven. The AV in this case is really the patient data. How much is that worth by itself? If needed, this SLE value can then be annualized to an ALE (Annual Loss Expectancy) value, which is simply the SLE value multiplied times the expected frequency of occurrence.
   This is generally an easy criterion to measure. The data by itself may be worth nothing financially; however, knowledge that this attack has taken place must yield speculation that at larger more drastic affect to patient safety is almost certain to follow as most threat agents would not be able to profit by the vital information obtained from the pacemaker device, by itself.
2. Is the successful exploit one that introduces a drawn out financial impact? (e.g. – bad press coverage, class action lawsuits, etc.) How much is it worth in the context of what may follow as a subsequent form of attack that supports a larger threat? This is difficult to perform an accurate valuation on and can largely be based on industry precedence of similar post-exploitation events.

The following chapter will provide more examples via use cases to further exemplify how impact and probabilities can be derived from the attack model.

Stage VI:Activity V (S6:A5) Derive a Set of Attack Cases to Test Existing Countermeasures Now that an attack tree has been formed, it is an opportune time to see if the control framework that is in place can provide some level of inherent threat mitigation. In Stages I and II, we were introduced to controls that could be aligned with the application design. These controls would reflect the control framework for the application. In this activity, we want to select any possible controls that could limit or eliminate any aspect of the threat. This includes the opportunity for an abuse case, the existence of a vulnerability or design weakness or the attack itself. Reverting back to the CAPEC 117 example, if there are features of the pacemaker device that provide for enhanced security, but may not be enabled, then this may be a consideration. Such an inherent security control may already be present in the manufactured medical device but may not have actually been enabled. This is a frequent issue across various frameworks and even third-party services. It is therefore important during Stage II to have a review of all possible inherent control options are determine and enumerated from the technology components identified to be in scope for the application solution.

Once a list of possible inherent countermeasures is identified, the countermeasures should be tested for their effectiveness in limiting the effects of the exploit. Such exploit testing is conducted in the next activity for this stage. It's important to note that the controls selected as part of this activity are already present and don't constitute new countermeasures that need to be developed by the application or system development team.

Stage VI:Activity VI (S6:A6) Conduct Attack Driven Security Tests & Simulations The key objective for this activity is to demonstrate attack viability by denoting the probability and severity level of the attacks defined in the attack tree. While this has been a challenge among security and IT professionals, this activity will help illustrate which attack patterns may be successful. These values will later be used as part of the overall residual risk formula that is introduced and calculated in Stage VII.

The threat modeler will define the scope for a targeted pen test. No pen test will simply be one with a broad scope. In this stage, the activity should focus only on the vulnerabilities identified from the prior stage. Since the vulnerabilities found in the prior stage were topical to the threats enumerated in stage IV, the workflow of leveraging outputs from the prior stages becomes quite apparent.

When pen testing, the penetration testing team can either look up attack patterns in their respective tools that would exploit the associated vulnerabilities from Stage V. This matching can be done by using the CAPEC number or by using a keyword search, such as "Data Interception," or finding synonyms that reflect the intended attack goal. Testing tools are slowly adopting the CAPEC IDs (which are comparable to the CVE and CWE IDs). These references can be used to identify what attack patterns can be tested. Alternatively, penetration testing team members can look up the vulnerability that corresponds to the attack, by referencing that information again in the vulnerability branch layer of the attack tree. CWEs and CVEs can also be searched in tools such as Metasploit, Armitage, Web Inspect, Hailstorm, and AppScan. In lieu

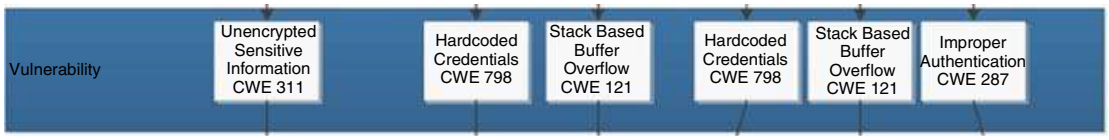


Figure 7.37 Vulnerability Portion of Attack Tree

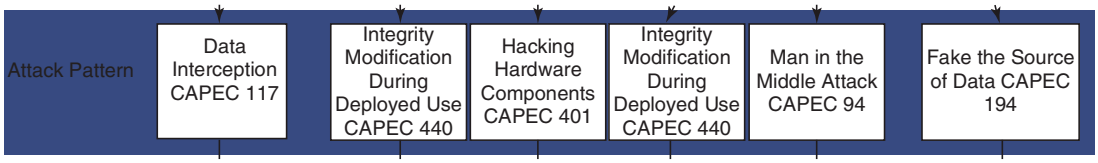


Figure 7.38 Attack Pattern Portion of Attack Tree

of doing batched scanning, these targeted scans along with manual pen tests can be conducted (Figure 7.37).

As an example, some attack module searches can be conducted on the following sites using some of the methods depicted earlier. A list of a few of these sites that can be referenced by the penetration testing team is as follows.

- http://www.exploit-db.com/
- http://www.metasploit.com/modules/framework/search
- http://www.commonexploits.com/

More sophisticated attacks can be achieved by allowing the penetration testing team to develop customized attacks that fulfill the goals of those listed under the attack pattern branch of the attack tree (Figure 7.38).

It is important to note that attacks do not necessarily have to succeed in order for an attack pattern on the attack tree to have some merit. Since results can vary depending on the rules of engagement for the penetration test, the allotted time, and the sophistication of the attack, an unsuccessful attack should simply indicate that the probability level should be throttled down when performing the overall risk analysis on each threat and determining what countermeasures to use and develop.

Most penetration testing software allows you to select attack modules to deliver to a target host. These modules or individual attack scripts, within a penetration testing framework (such as Metasploit Pro, Armitage, or Kali Linux), allow responsible testing members to conduct the testing in accordance with the attack patterns that match those listed under the attack tree. Reverting back to the recently completed attack tree, we can begin to find how these attack patterns can be developed as targeted attacks or using a penetration testing framework.

In Chapter 8, we will go over more specific use cases that provide a wider range of diversity for conducting the Attack Modeling phase. We will also introduce the notion of kill chains as a substitute for attack trees in order that greater options for modeling attacks can be considered.

A visual summary to stage VI is depicted in Figure 7.39.

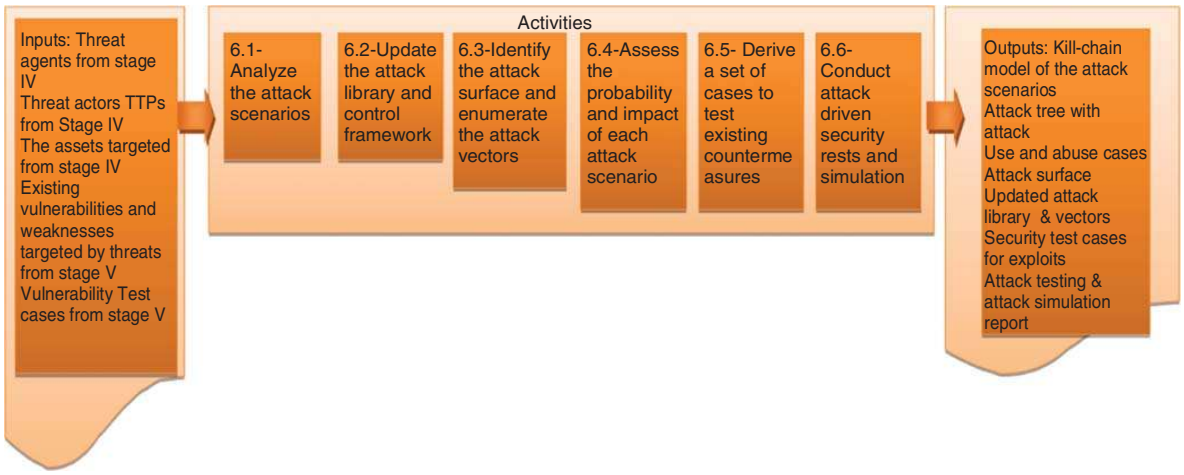


Figure 7.39 PASTA Risk-Centric Threat Modeling – Stage VI – (AMS) Attack Modeling and Simulation

Stage VII – Residual Risk Analysis & Management (RAM)

Stage VII of the PASTA methodology is focused on mitigating threats that matter to the application, product team, and overall business. This is achieved by applying all types of countermeasures that are both effective and topical to the threats and attacks depicted under the PASTA threat model. This approach saves time, is based on evidence or substantiated signs of threat patterns, and fosters a greater understanding of how security impacts application use cases and, to a greater extent, business objectives. The following are the key activities to be performed under this stage.

Stage VII:Activity I (S7:A1) Calculate the Risk of each Threat. This is an activity that borrows from prior efforts around threat analysis and risk evaluation, namely, the portion of the risk formula that deals with probability. As part of this activity, the risk professionals will have the ability to review the threat model and supportive attack trees in order to see how viable identified threat patterns are. Stage IV of the PASTA methodology provides insight to risk professionals as to the prevalence of the threat and the viability of the threat taking place against the target application or system. For each threat in the attack tree or trees that are depicted, a percentage weight of probability should be assigned based on the following conditions:

1. Internal Threat Data
   a. Do logs show that this threat has taken place before?
   b. Is there evidence that shows signs of this threat taking place?
   c. Where in the architecture have key threat data sources been pulled from? High-impact business areas? Confidential areas?
2. External Threat Intelligence
   a. What similar threat events have affected other companies in your industry?
   b. Where have similar threats been seen (across industries)?
   c. Have the threats been focused on architectures that are currently used by the organization and specifically the application undergoing the threat model?
3. Viability of Attacks
   a. Are testing efforts able to successfully exploit weaknesses/vulnerabilities that support this threat?
   b. Is the architecture containing the threat target in a readily accessible area (physically or logically?)
   c. Is the sophistication level low for exploitation of weaknesses/vulnerabilities, related to this attack(s)?

Using these three areas, attack patterns or the associated CAPECS can be assigned a probability percentage of success. This would provide a coefficient that could be used and averaged as a threat probability. Other coefficients of probability are also possible to be placed on the weakness or vulnerability (CWE & CVE respectively) nodes on the attack tree. Bruce Schneier actually has a pretty simplistic, yet effective

way to associate branches of a tree with either a probability percentage of occurrence as well as a financial value of impact.[15] Related to the probability percentages, this can be helpful, especially after having tested the application and determine the viability of the attack given test results against known weaknesses or vulnerabilities.

The attack tree, threat analysis from stage IV, and the security testing results should provide a good basis for which to determine the probability of attack layers, supporting a threat, to become viable patterns that yield an overall successful attack.

Stage VII: Activity II (S7:A2) Identify the Countermeasures The objective for this activity to determine the right amount of risk mitigation through the use of agreed upon countermeasures to be developed or designed by the appropriate roles. Software engineers and architects must work together to determine how unacceptable risk levels will be addressed by the implementation of newly proposed countermeasures.

This activity will revisit the threat model and the associated attack tree in order to see what countermeasures can be implemented via architectural design or via software updates to either the codebase or product application. A visual depiction of how countermeasures can be identified for a specific use case using use-abuse case analysis is depicted in Figure 7.40.

As part of this exercise, the threat modeling team can begin to enumerate a vast list of countermeasures that have associated costs and benefits so that product

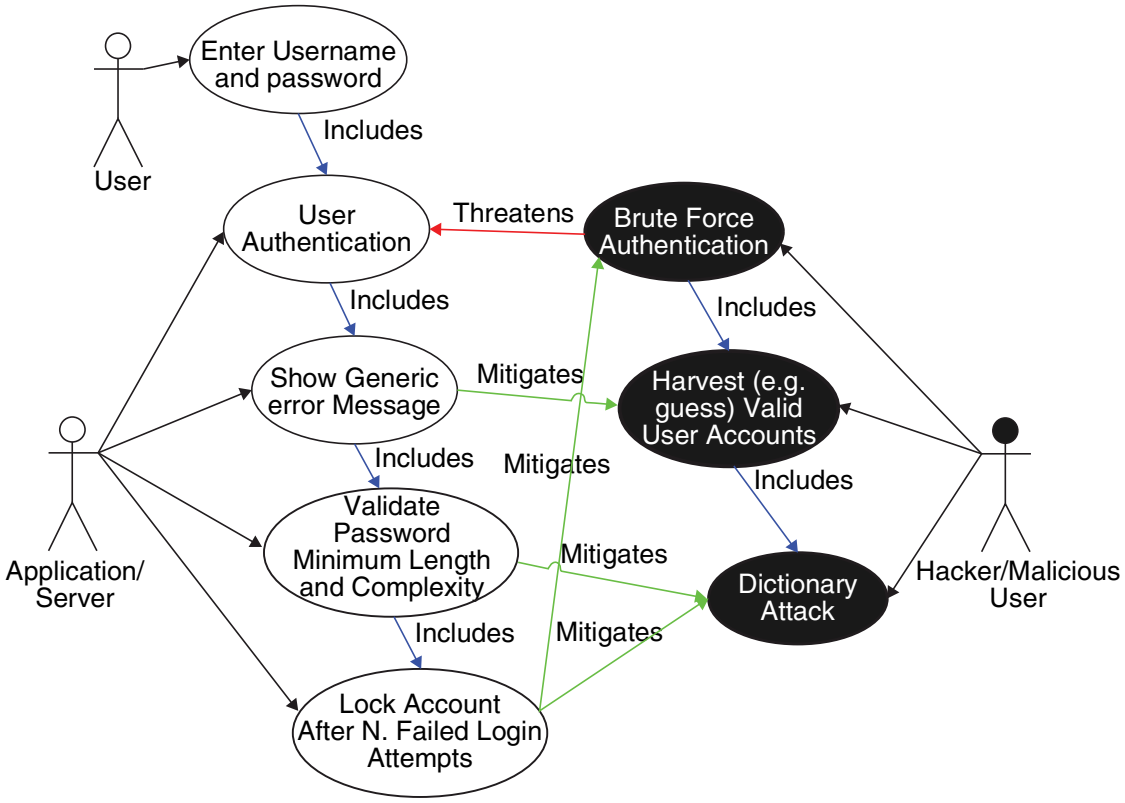


Figure 7.40 Visualization of Attack and Countermeasures

[15]https://www.schneier.com/paper-attacktrees-ddj-ft.html#rf1.

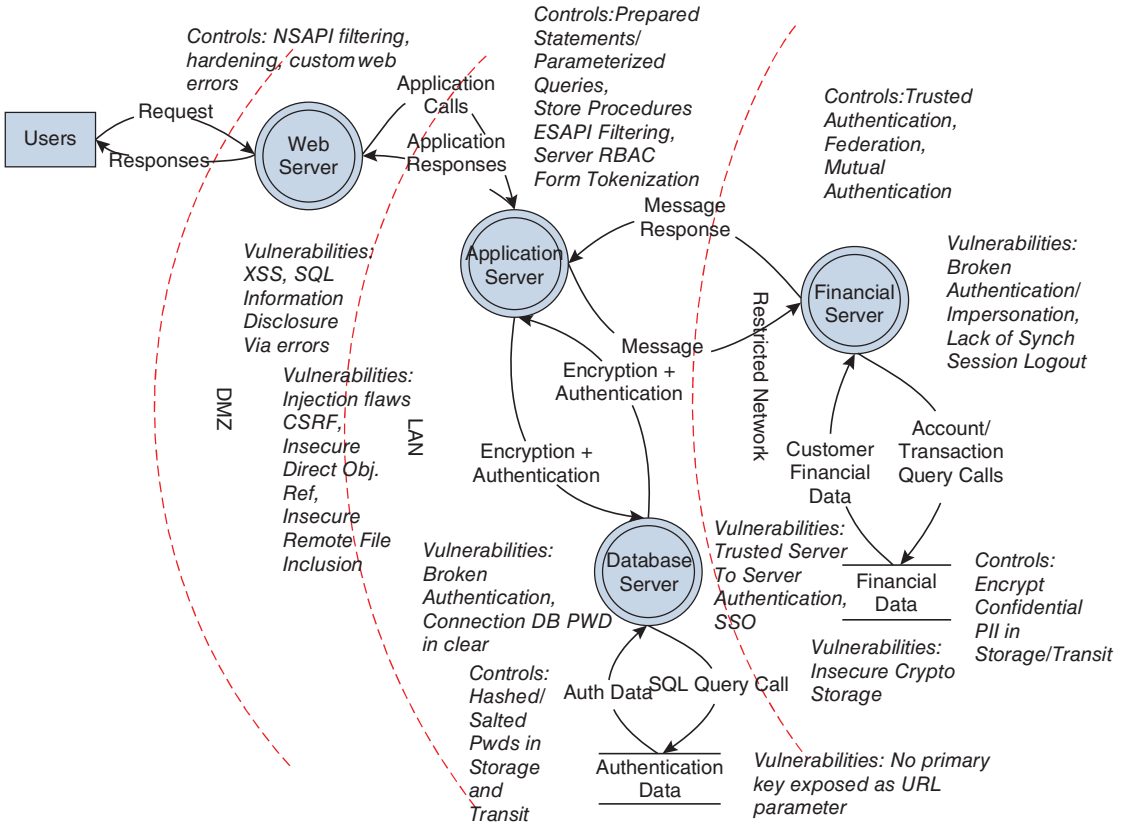


Figure 7.41 Data Flow Diagram With Architectural Risk Analysis of Vulnerabilities and Controls

management groups can be informed on how to make the best decision regarding what countermeasures to develop.

The assignment of architectural countermeasures can be done using a DFD. Many of the threats and vulnerabilities that affect data endpoints could be mitigated architecturally. As a result, exercises based on the following figure can provide a visual indication to the product team on where countermeasures could be placed. (Figure 7.41).

Beyond DFDs, the attack trees can also provide an alternative for depicting the right countermeasures that relate to attack mitigation and ultimately, threat mitigation. The bottom row of the figure that follows details appropriate countermeasures for the threats that have been discussed in the prior activities. Going back to our branch of the tree that shows the attack on the Bluetooth module with a stack-based buffer overflow, we can see that the proper way to deal with this type of threat is to perform security testing that will lead to the identification and elimination of the vulnerability. Even though it is not specifically applicable in this situation, it may make sense to deploy multiple countermeasures. For instance, if we were trying to deploy countermeasures around a cross-site scripting vulnerability in a web application, we could perform security testing around the application environment to identify and eliminate the vulnerability. Additionally, we could also deploy a web application firewall to help prevent the exploitation of the vulnerability. Depending on the

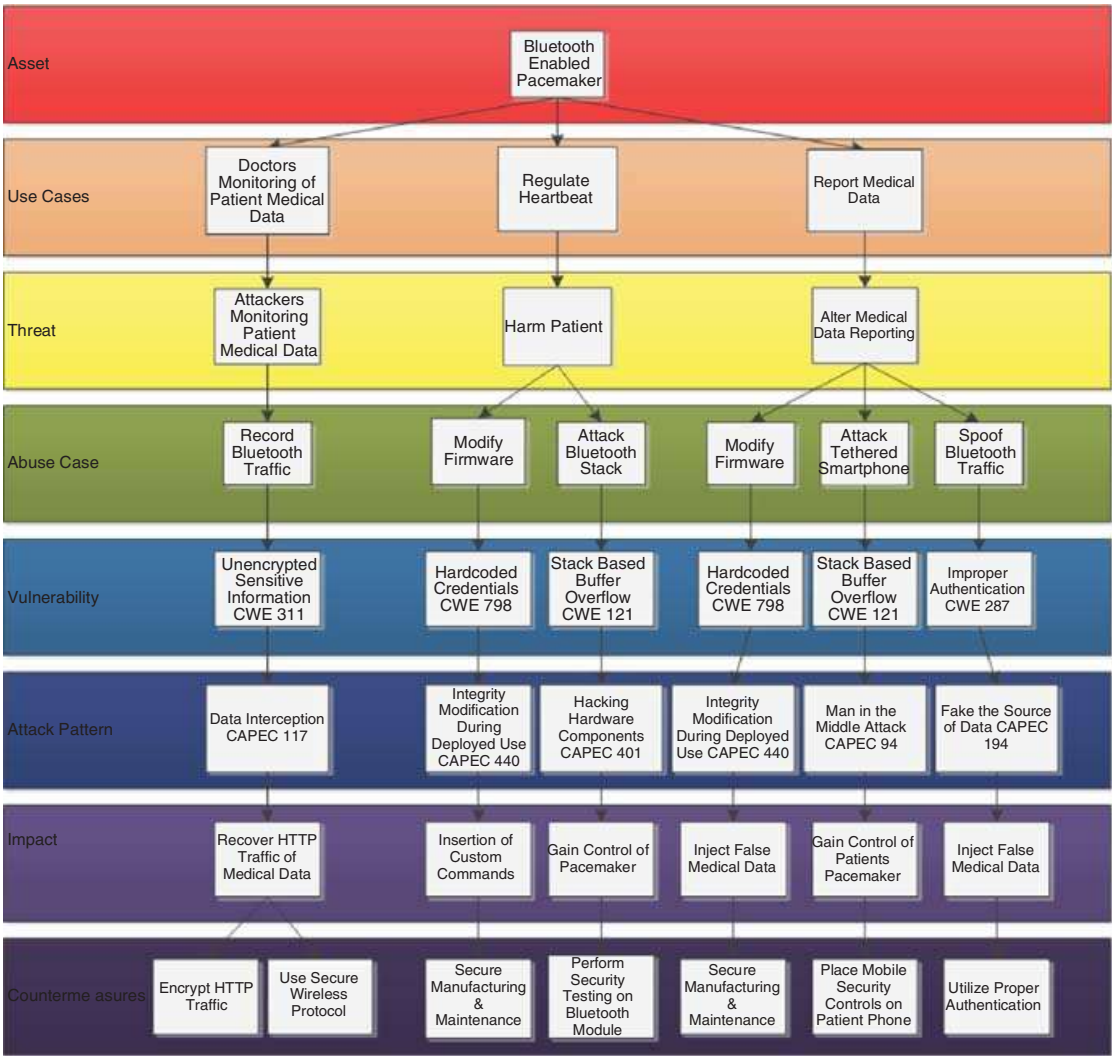


Figure 7.42 Completed Attack Tree w/Countermeasures

severity of the cross-site scripting issue and the criticality of the application, we could even recommend removing affected sections of the code base, or taking the entire application out of the production environment.

A representation of the attack tree, inclusive of the countermeasures that will be developed and mapped to the attack tree, is exemplified in Figure 7.42.

Stage VII: Activity III (S7:A3) Calculate the Residual Risks The objective for this activity revolves around risk analysis. The asset-centric approach in this guide addresses risk a bit differently by considering other variables that are not traditionally covered in risk analysis. These other variables are as follows.

- *Probabilities:* Probability coefficients can be assigned to threats and vulnerabilities in order to determine the likelihood that the vulnerability could be exploited under the observed conditions and based on its publicity. Additionally, the probability associated with the threat should be considered. Is it easy to execute the threat within the application's environment?

- *Countermeasures:* The countermeasures that currently provide some degree of protection against the threat and vulnerability also need to be factored into the risk analysis. Was the penetration testing team given information or certain advantages that allowed them to successfully exploit the application environment?

In consideration of these variables, this risk-centric threat modeling step defines a residual risk. This residual risk is what risk is left upon factoring existing countermeasures into the overall risk analysis. The new residual risk formula would be depicted:

$$\text{Residual Risk} = [(\text{tp} * \text{vp})/\text{c}] * i$$

Step 1 – Severity Rating

Threat Level (1–4): Determine a threat level for each identified threat.

$$(1 = \text{Low} \mid 2 = \text{Moderate} \mid 3 = \text{High} \mid 4 = \text{Critical})$$

This level of analysis is ported over from stage IV of PASTA. We can translate the qualitative assessment given here or use the probabilistic analysis provided in stage IV of this chapter.

Vulnerability Level (1–4): Determine how critical the vulnerability is in the context of the application or the affected use case.

$$(1 = \text{Low} \mid 2 = \text{Moderate} \mid 3 = \text{High} \mid 4 = \text{Critical})$$

This will use the probabilistic analysis associated with the exploitability of the weakness or attack. This was discussed under stage VI of the PASTA methodology. Alternatively, a quantitative percentage can be assigned based on the qualitative values listed earlier.

Impact Level (1–4): Determine what the business impact would be if the threat was successful.

$$(1 = \text{Low} \mid 2 = \text{Moderate} \mid 3 = \text{High} \mid 4 = \text{Critical})$$

It is best to unitize the impact as discussed under stage I of the PASTA methodology and Stage VI as well where impact level were calculated based on successfully completed attacks. The information provided by the business analyst will ultimately help to further support information from business impact analyses (BIAs) completed around the application environment. This is important to the overall threat modeling process as it helps to identify where countermeasures should be developed by and between actor(s) making an application call to a data source. The threat modeler or facilitator of threat modeling activities should facilitate the activities that take place in stage II in order to ensure that the activities revealed in the RACI diagram per role are completed. Within this stage, the business analyst will help to define the properties of the application

data and confirm the supportive technology as well as new technology center to the application environment.

Step 2 – Probability Coefficient Using a scale from 10% to 90%, assign a percentage for the probability that:

1. The threat will take place.
2. The vulnerability is widely known and can be exploited.
3. The probability that if the threat is imminent, the impact level would be as severe as previously rated.

☞ Note: A scale of 10–90% is used to incorporate a standard deviation percentage since the probability value for an occurrence can never be guaranteed at 100% or 0%.

Step 3 – Number of Countermeasures and the Effectiveness of the Countermeasures Considering what countermeasures exist in the environment that may not have been tested as part of the exploitation phase of the penetration testing OR considering some proposed countermeasures that could be put into place, list the number of countermeasures that would deter each of the following (a countermeasure can deter or resist more than one of the following choices (examples are also included) (Table 7.19).

The effectiveness of the countermeasures listed can range from 10% to 90%, as nothing would prove to be 100% effective. Since these are subjective values, it is best to use percentages that are rounded to 5 or 10 percentage points.

Additional examples and use cases of risk calculation are provided in Chapter 8.

Stage VII:Activity IV (S7:A4) – Recommend Strategy to Manage Risks The objective for this activity is to update risk profiles associated with the system or application, in order to have a current account of both risk and risk strategy.

Compliance and risk management teams should work together with threat modeling efforts, as part of a joint working session, in order to update the risk profile of the application and capture the risk values. These risk values can be factored into any formal enterprise risk management suite (e.g. Modulo, Archer Technologies, LockPath GRC) for tracking and reporting.

TABLE 7.19 Residual Risk Analysis

Risk Component	Number of Countermeasures	Effectiveness of Countermeasures	Description of Countermeasures
Threat	2	30% (reduces threat by 30%)	Improved authentication and application whitelisting
Vulnerability	3	90% (addresses vulnerability by this amount)	Patch to software framework
Impact	0	0% (reduces impact that could be realized by this amount)	None

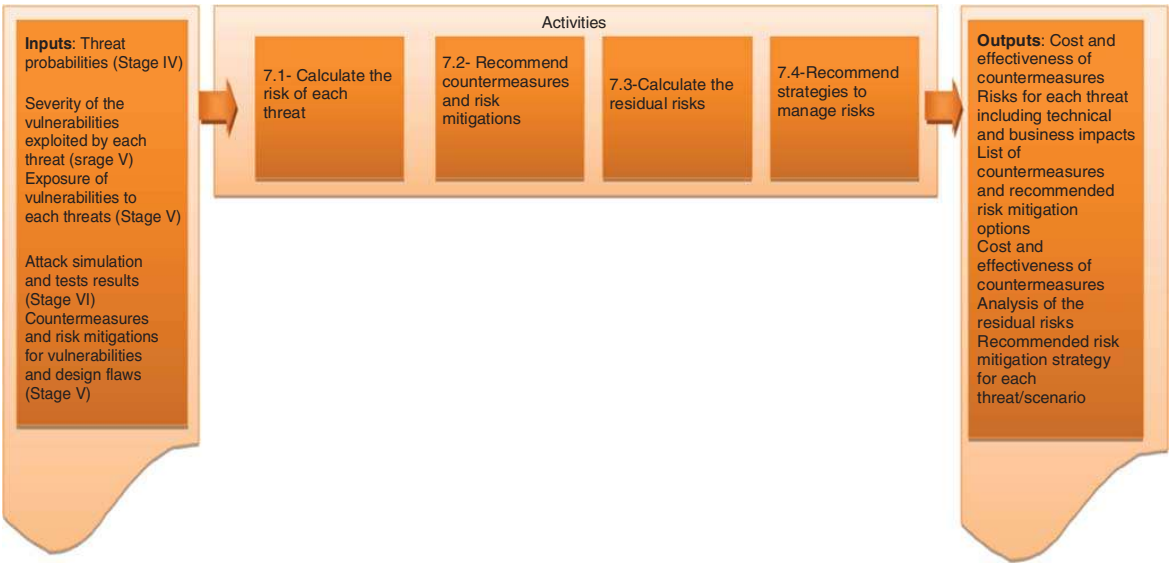


Figure 7.43 PASTA Risk-Centric Threat Modeling – Stage VII – (RAM) Risk Analysis and Management

Strategic direction around residual risks needs to come from application owners or product owners. Risk mitigation takes time and money. The risk appetite of the product group and/or overall organization is the one to ultimately decide on what countermeasures to develop. All of this strategy is based on the risk levels identified via the seven stages of PASTA as well the depicted impact revealed by the overall threat model.

As part of succinct, yet highly effective strategy to manage the residual risks identified, the following should be performed:

1. Share residual risk information and overall threat model (attack tree, DFDs, residual risk analysis) with information/application owner.
2. Risk management and compliance professionals should assist in the conveyance of all risk-related items found. They should also extract the relevant security and compliance gaps that the threat model exposes the product, to as well as the risks now linked to the organization.
3. Determine whether the risks depicted by the threat model supersede the costs of neglecting the observed threats. Threats that do not pose a risk level that is above a baseline acceptance of risk should be logged as accepted in an enterprise risk management platform.
4. Threats found to present material risk to the product or overall business should be mitigated using the proposed countermeasures revealed by the threat model. These countermeasures should be formally tracked and measured to completion.
5. The artifacts produced during the threat modeling process should be used to support a new risk profile for the application. The threat model, namely the DFDs and attack trees, should be preserved and revisited when the application environment undergoes a drastic code change and/or component change (e.g. framework, COTS, hosting/deployment model, etc.) Conditions for reevaluating the threat model should be imposed at the conclusion of Stage VII.

CHAPTER SUMMARY

Regardless of the organizational size, application threat modeling allows for a clear roadmap of risk mitigation – a message appreciated and understood by business. Even in a team of one, a well-developed threat model will save time over ad hoc security testing or control gap analysis and control audits. In the chapter that follows, we will build from the notion that security CAN be built in via a risk-centric approach to threat modeling and look at each one of these PASTA stages in greater detail. Chapter 8 also provides a broad range of use cases for how to successfully apply the PASTA methodology.

A diagram summarizing the activities, inputs, outputs for this stage is presented in Figure 7.43.